Intersection Detection of Bézier Curves Using the Implicitization Method — A Study and Implementation of Practical Methods for Solving Algebraic Equations —*

Takeshi Bashomatsu†

October 31, 2026

Abstract

This paper proposes a method for computing intersections between cubic Bézier curves with high accuracy and high performance. The Implicitization method, which algebraically determines the parameter values of intersections by converting Bézier curves into implicit form, has been known. However, a stable and efficient method for finding the real roots of the resulting algebraic equations had not been established. In this work, we narrow down candidates using Budan–Fourier's theorem, perform real-root isolation using the VAG method—an application of Vincent's theorem—and then compute real roots using the bisection method and Ridder's method. Compared with the DKA method, which is generally used to solve high-degree algebraic equations, the proposed approach allows the search domain for roots to be restricted to the real parameter range of the Bézier curves and is not affected by roots outside this range. When multiple roots occur depending on the shape of the Bézier curves or the intersection pattern, iterative root-finding methods such as the bisection method and Ridder's method cannot be applied directly. This issue is identified, and a solution is devised by extending the VAG method to also handle root finding; its effectiveness is confirmed. A trial implementation of the proposed method is carried out in a C++/Qt environment. It is verified that intersections between arbitrarily specified cubic Bézier curves can be computed correctly, and that the required memory usage and computation time are at a practically acceptable level. The proposed method is not limited to Bézier curves and is expected to be applicable to other cubic parametric curves as well.

1 Background and Related Work

Bézier curves are parametric curves widely used in computer graphics, GUI rendering, CAD, and related fields. There is a strong need to compute intersections between curves.

*This article has been reconstructed based on content previously published in the following post:
Qiita - "Intersection Detection of Bézier Curves Using the Implicitization Method
— A Study and Implementation of Practical Methods for Solving Algebraic Equations —"
https://qiita.com/kumanomi1000/items/bf69f2c32d52cc13b93b

†bashomatsutakeshi@gmail.com

As an existing intersection-detection technique, there is the polyline approximation method, in which curves are subdivided into small line segments and collision detection is performed. In rendering processes for displaying Bézier curves on the screen, drawing APIs such as OpenGL do not directly support curve rendering, so Bézier curves are approximated by polylines at higher levels (e.g., Qt GUI or Cairo). Because this polyline information can be reused for collision detection, intersection detection via polyline approximation is a reasonable approach. In rendering processing, the accuracy of the intersection points is sufficient if they are on the order of a pixel or a fraction of that.

In contrast, in CAD and similar applications, approximating Bézier curves by polylines solely for intersection detection is inefficient, and the required accuracy is much higher, so making polyline approximation method is not suitable. For such use cases, the subdivision method has been employed. This method exploits the property that a Bézier curve is contained within the polygon formed by its control points. By detecting intersections between such polygons, the parameter ranges of the curves that contain intersections are narrowed down, and the same process is iteratively applied to subdivided curves. Bézier clipping, proposed by T. W. Sederberg and Tomoyuki Nishita, is a well-known example of this approach [1]. However, this method may have poor convergence or may be difficult to guarantee accuracy depending on the shape of the Bézier curve and/or the intersection pattern.

T. W. Sederberg and colleagues proposed the "Implicitization" method, which converts Bézier curves into implicit form, transforms the intersection problem into an algebraic equation (a ninth-degree equation for intersections between two cubic Bézier curves), and solves it [2] [3]. In the Implicitization method, a parametric Bézier curve is converted into an implicit function, and the parametric equations of the other curve are substituted into it to obtain an algebraic equation (the implicit equation). This method is less affected by the shape of the Bézier curve and/or the intersection pattern, and is expected to be able to determine the intersection position efficiently and with high accuracy. However, a suitable method for solving the implicit equation had not been shown. Therefore, this study aims to develop an efficient and stable method for solving the implicit equation based on the Implicitization approach and to realize a practical implementation for computing Bézier curve intersections.

Since a ninth-degree equation cannot be solved algebraically, approximate solutions must be obtained using iterative root-finding methods. The DKA method, an extension of the Newton–Raphson method for higher-degree equations, is commonly used. The DKA method handles complex roots and places initial values evenly on a circle in the complex plane that encloses all roots (Aberth's initial values), then iteratively refines them. To prevent multiple approximations from converging to the same root, a repulsive force between approximations is incorporated into the recurrence formula. The radius of the initial circle must be set appropriately using Aberth's method. If the radius is too small, unstable behavior occurs during iteration; if it is too large, convergence becomes slow.

Initially, we attempted to find the real roots of the implicit equation using the DKA method. The DKA implementation was constructed as an evaluation worksheet in Excel, referring to [4]. The radius of the initial circle was computed using the program in [5]. For samples in which all roots were real numbers in the interval $(0, 1)$, good convergence was achieved; **Ex.3** in Table 3 corresponds to this case. (blue text indicates roots at intersections)

Table 1: Roots of implicit equation of **Ex.3**
(case in which DKA method is easily applied)

Roots of $F_0(u) = 0$	Roots of $F_1(t) = 0$
0.0192470	0.0705391
0.0549916	0.0972454
0.122057	0.151318
0.320989	0.423317
0.442531	0.504785
0.488156	0.616234
0.900476	0.858628
0.944674	0.947328
0.963072	0.966598

The radius of the initial circles by Aberth's method were small, 0.891145 and 0.815073, and the DKA method converged after about 15 iterations. However, when the intersection pattern caused the equation to include roots far outside the parameter range of the Bézier curve (roots irrelevant to intersection detection), the initial circle became extremely large, significantly degrading convergence. **Ex.1** corresponds to this case. (red text indicates roots that negatively affect convergence)

Table 2: Roots of implicit equation of **Ex.1**
(case in which DKA method is difficult to apply)

Roots of $F_0(u) = 0$	Roots of $F_1(t) = 0$
-567116	-248197
0.557665	-1.03331
1.61807	0.446194
5.16799	1.86453
5.79814	2.90335
-0.570942 -0.800873I	-213844-446352I
-0.570942+0.800873I	-213844+446352I
283560-491136I	0.534608-0.882298I
283560+491136I	0.534608+0.882298I

The radius of the initial circles by Aberth's method were huge, 624194 and 518608, and the DKA method did not converge even after 50 iterations. Based on these results, we concluded that the DKA method is unsuitable for the present application.

Alexander Reshetov of NVIDIA proposed an approach for ray tracing of hair and fur in PBRT, which are modeled using many Bézier curves. His approach first isolates real roots, identifies intervals containing exactly one real root, and then applies an iterative root-finding method [6].

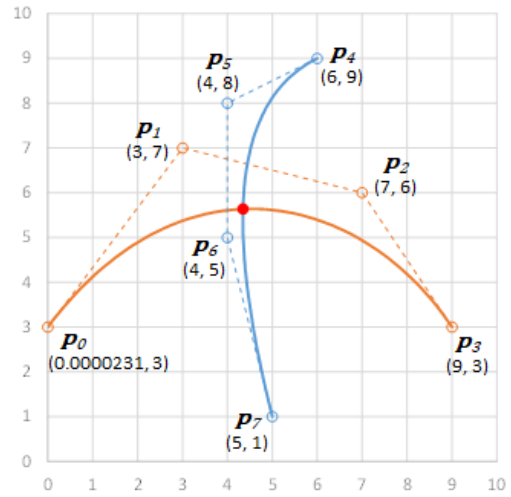
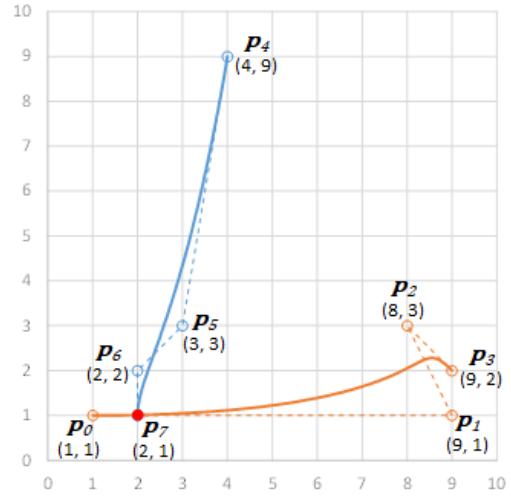
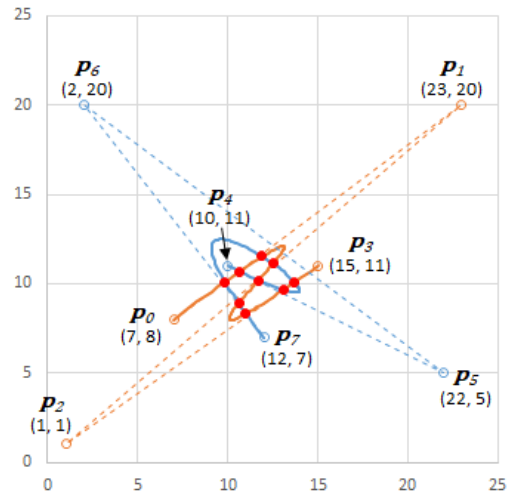
- Finds the intersection of a cubic Bézier curve and a ray (straight line). Bézier curves must be treated as having thickness, and the root of its derivative (a quintic function) is found to determine the position where the square of the distance between the curve and the line (a sextic function) is smallest.

- To find the root, the Boudin-Fourier theorem is used to narrow down the search, and Vincent's theorem is used to isolate the real root, then a root-finding algorithm is applied with as few iterations as possible.

- By combining techniques such as narrowing down the range where roots exist based on the spatial relationship between the Bézier curve and the line, a combination of the Ridder's method (without iterations, one time) and the secant method can achieve sufficient approximation accuracy.
- Significant improvements in both performance and accuracy compared to existing methods (broken-line approximation).

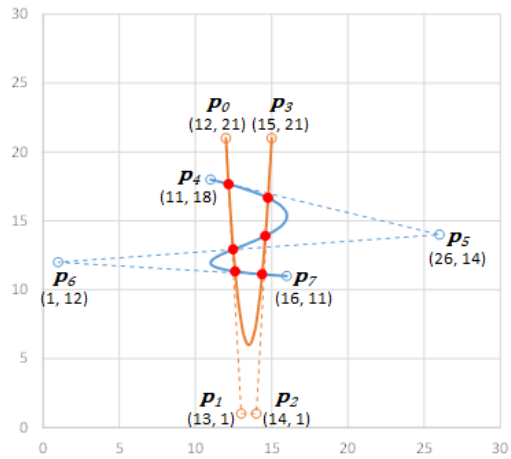
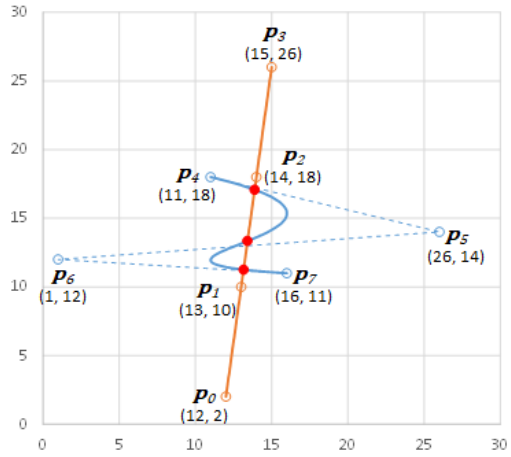
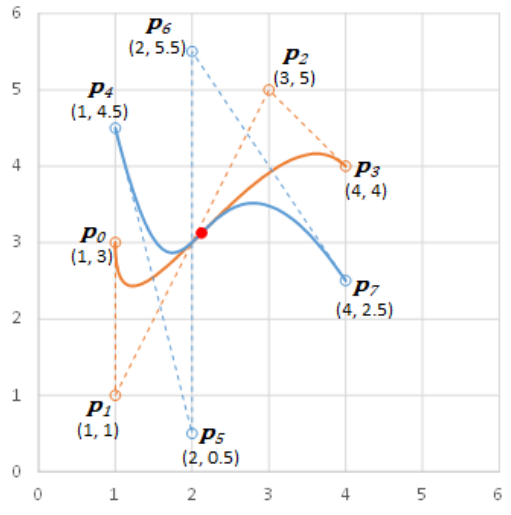
Inspired by this work, we aim to apply a similar approach to detecting intersections between Bézier curves. Among the algorithms derived from Vincent's theorem—VAS, VCA, and VAG—we select the VAG (Vincent–Alesina–Galuzzi) method due to its ease of implementation. For iterative root finding on isolated intervals, we first narrow the interval using the bisection method and then apply Ridder's method. Since the bisection method and Ridder's method do not handle multiple roots, we extend the VAG method to address this issue.

Table 3: Examples of Bézier curves intersection and properties of the implicit equations The orange curve is the Bézier curve $\boldsymbol{B}_0$ defined by control points $\boldsymbol{p}_0, \boldsymbol{p}_1, \boldsymbol{p}_2, \boldsymbol{p}_3$, and the blue curve is the Bézier curve $\boldsymbol{B}_1$ defined by control points $\boldsymbol{p}_4, \boldsymbol{p}_5, \boldsymbol{p}_6, \boldsymbol{p}_7$.

Ex.1		Intersects at one point Roots with large absolute value are included
Ex.2		Intersects at one point Roots close to the upper bound of the parameter's range are included
Ex.3		Intersects at nine points All roots are real numbers and lie within the parameter's range

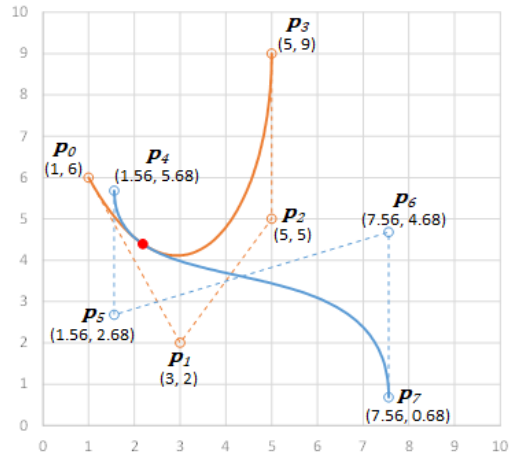
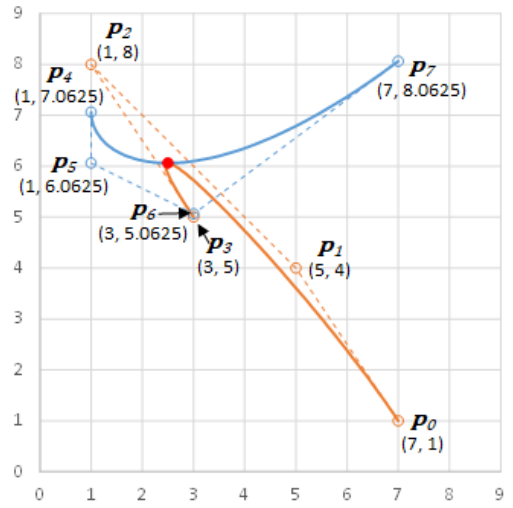
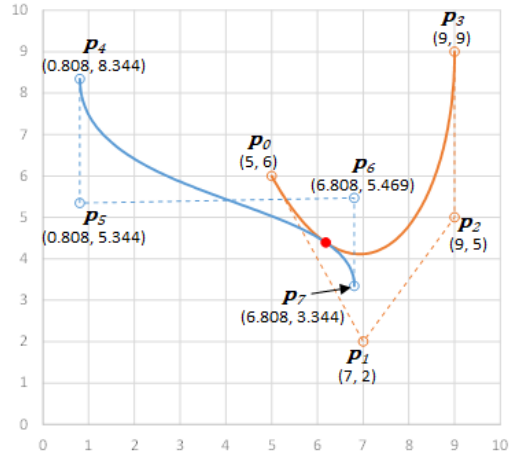
Continued on next page

(Continued)

Ex.4		$\boldsymbol{B}_0$ is quadratic Implicit equations are 6th-degree
Ex.5		$\boldsymbol{B}_0$ is linear Implicit equations are 3rd-degree
Ex.6		Intersects at one point first derivatives coincide at the intersection Multiple root with multiplicity of 3 is included

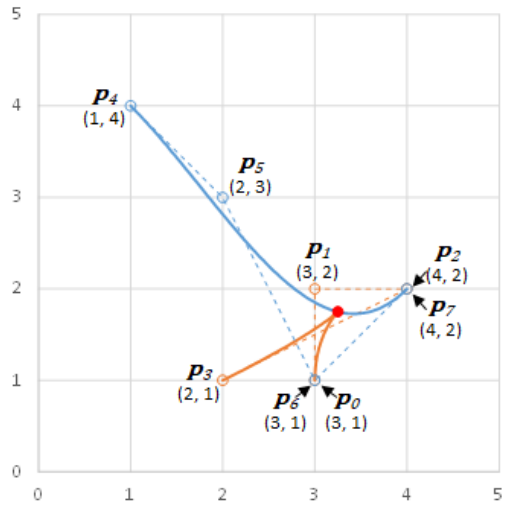
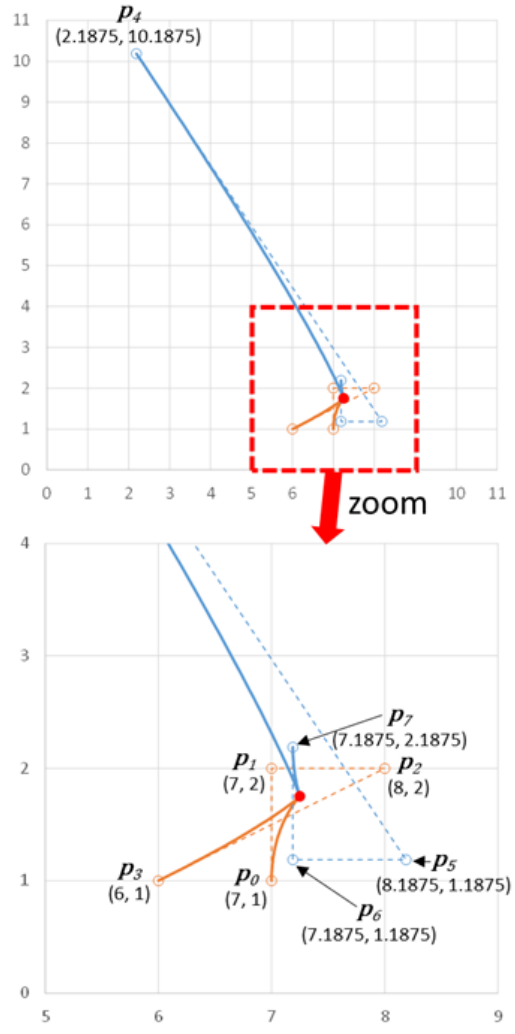
Continued on next page

(Continued)

Ex.7		Intersects at one point first derivatives coincide at the intersection Multiple root with multiplicity of 3 is included
Ex.8		Contacts at one point first derivatives coincide at the contact point Multiple root with multiplicity of 2 is included
Ex.9		Contacts at one point first derivatives coincide at the contact point Multiple root with multiplicity of 2 is included

Continued on next page

(Continued)

Ex.10		A cusp point contacts Multiple root with multiplicity of 2 is included
Ex.11		Two cusp points contact each other Multiple root with multiplicity of 4 is included

2 Adopted Method

2.1 Target Curves

For the Bézier curve $\boldsymbol{B}_0$ defined by control points $\boldsymbol{p}_0, \boldsymbol{p}_1, \boldsymbol{p}_2, \boldsymbol{p}_3$, the coordinates (x, y) of an arbitrary point on the curve are given by the following matrix-form parametric functions [7]:

$$x = f_0(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \boldsymbol{R}_{bz} \begin{bmatrix} p_{0x} \\ p_{1x} \\ p_{2x} \\ p_{3x} \end{bmatrix} = a_0 t^3 + a_1 t^2 + a_2 t + a_3$$

$$y = g_0(t) = \begin{bmatrix} t^3 & t^2 & t & 1 \end{bmatrix} \boldsymbol{R}_{bz} \begin{bmatrix} p_{0y} \\ p_{1y} \\ p_{2y} \\ p_{3y} \end{bmatrix} = b_0 t^3 + b_1 t^2 + b_2 t + b_3$$

Similarly, for the Bézier curve $\boldsymbol{B}_1$ defined by control points $\boldsymbol{p}_4, \boldsymbol{p}_5, \boldsymbol{p}_6, \boldsymbol{p}_7$, the coordinates (x, y) are defined as follows:

$$x = f_1(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \boldsymbol{R}_{bz} \begin{bmatrix} p_{4x} \\ p_{5x} \\ p_{6x} \\ p_{7x} \end{bmatrix} = c_0 u^3 + c_1 u^2 + c_2 u + c_3$$

$$y = g_1(u) = \begin{bmatrix} u^3 & u^2 & u & 1 \end{bmatrix} \boldsymbol{R}_{bz} \begin{bmatrix} p_{4y} \\ p_{5y} \\ p_{6y} \\ p_{7y} \end{bmatrix} = d_0 u^3 + d_1 u^2 + d_2 u + d_3$$

where, $\boldsymbol{R}_{bz}$ is the coefficient matrix specific to cubic Bézier curves:

$$\boldsymbol{R}_{bz} = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

2.2 Requirement Definition: Parameter Range

Assuming composite Bézier curves composed of multiple connected segments, the parameter range of the Bézier curves (t and u) is defined as the half-open interval $[0, 1)$. Since Budan–Fourier's theorem and Vincent's theorem apply to open intervals, separate logic is added to determine whether 0 is a root.

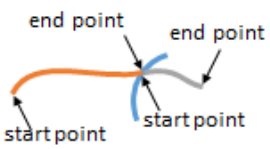


Figure 1: An example of composite Bézier curve and intersection

If the Bézier curve is isolated, with a closed interval $[0, 1]$, additional logic is required to determine whether 1 is a root.

2.3 Requirement Definition: Classification of Intersection Patterns and Roots of the Implicit Equation

The following classification is based on empirical observations and requires further analytical investigation:

- Even if curves do not intersect, the implicit equation has real roots in $[0, 1]$ if the curves intersect when extended.
- If the first derivatives of both curves coincide at an intersection point, the implicit equation has a triple root.
- If the curves contact at a single point with matching first derivatives, the implicit equation has a double root.
- If a cusp point contacts a curve, the implicit equation has a double root.
- If two cusp points contact, the implicit equation has a quadruple root.

Table 4 summarizes these cases, where “ true ” and “ false ” indicate the presence or absence of real roots in $[0, 1]$.

Table 4: Intersection patterns and types of roots of implicit equations

Intersection pattern (orange: curve #0, blue: curve #1)		Curve #0	Curve #1	Multi-plicity	Example
	No intersection	false	false		-
	No intersection, intersects if curve #1 is extended to left	true	false		-
	Intersects at one point	true	true	1	**Ex.1** **Ex.2**
	Intersects at one point, first derivatives coincide at the intersection	true	true	3	**Ex.6** **Ex.7**
	Contacts at one point, first derivatives coincide at the contact point	true	true	2	**Ex.8** **Ex.9**
	A cusp point contacts	true	true	2	**Ex.10**
	Two cusp points contact each other	true	true	4	**Ex.11**

2.4 Procedure

1. Convert $x = f_0(t)$ and $y = g_0(t)$ of curve $\boldsymbol{B}_0$ into the implicit form $F_0(x, y) = 0$.
2. Substitute $x = f_1(u)$, $y = g_1(u)$ of curve $\boldsymbol{B}_1$ into $F_0(x, y) = 0$ to obtain the implicit equation $F_0(u) = 0$.
3. Find the real roots of $F_0(u) = 0$ for $u \in (0, 1)$.
4. Substitute the roots into $x = f_1(u)$, $y = g_1(u)$ to obtain candidate intersection points on $\boldsymbol{B}_1$.
5. Convert $x = f_1(u)$ and $y = g_1(u)$ of curve $\boldsymbol{B}_1$ into the implicit form $F_1(x, y) = 0$.
6. Substitute $x = f_0(t)$, $y = g_0(t)$ of curve $\boldsymbol{B}_0$ into $F_1(x, y) = 0$ to obtain the implicit equation $F_1(t) = 0$.
7. Find the real roots of $F_1(t) = 0$ for $t \in (0, 1)$.
8. Substitute the roots into $x = f_0(t)$, $y = g_0(t)$ to obtain candidate intersection points on $\boldsymbol{B}_0$.
9. Match candidate points; paired points are intersections.

2.5 Implicitization of Bézier Curves

From the parametric representation:
$x = f_0(t) = a_0t^3 + a_1t^2 + a_2t + a_3$,
$y = g_0(t) = b_0t^3 + b_1t^2 + b_2t + b_3$
of the curve $\boldsymbol{B}_0$, we obtain the implicit representation:
$F_0(x, y) = 0$.
Specifically, referring to [8], the parameter t is eliminated using the Eliminate function of Wolfram Alpha, and the resulting expression is rearranged into the form of the implicit function $F_0(x, y) = 0$. The notebook and execution results are shown in Appendix A.

The implicit representation of the curve $\boldsymbol{B}_0$ obtained in this way is as follows.

- Implicit representation when the curve $\boldsymbol{B}_0$ is cubic:

$$
\begin{aligned}
F_0(x,y) = {} & a_3^3b_0^3 - a_2a_3^2b_0^2b_1 + a_1a_3^2b_0b_1^2 - a_0a_3^2b_1^3 + a_2^2a_3b_0^2b_2 - 2a_1a_3^2b_0^2b_2 \\
& -a_1a_2a_3b_0b_1b_2 + 3a_0a_3^2b_0b_1b_2 + a_0a_2a_3b_1^2b_2 + a_1^2a_3b_0b_2^2 - 2a_0a_2a_3b_0b_2^2 \\
& -a_0a_1a_3b_1b_2^2 + a_0^2a_3b_2^3 - a_2^3b_0^2b_3 + 3a_1a_2a_3b_0^2b_3 - 3a_0a_3^2b_0^2b_3 + a_1a_2^2b_0b_1b_3 \\
& -2a_1^2a_3b_0b_1b_3 - a_0a_2a_3b_0b_1b_3 - a_0a_2^2b_1^2b_3 + 2a_0a_1a_3b_1^2b_3 - a_1^2a_2b_0b_2b_3 \\
& +2a_0a_2^2b_0b_2b_3 + a_0a_1a_3b_0b_2b_3 + a_0a_1a_2b_1b_2b_3 - 3a_0^2a_3b_1b_2b_3 - a_0^2a_2b_2^2b_3 \\
& +a_1^3b_0b_3^2 - 3a_0a_1a_2b_0b_3^2 + 3a_0^2a_3b_0b_3^2 - a_0a_1^2b_1b_3^2 + 2a_0^2a_2b_1b_3^2 + a_0^2a_1b_2b_3^2 \\
& -a_0^3b_3^3 \\
& +(-3a_3^2b_0^3 + 2a_2a_3b_0^2b_1 - 2a_1a_3b_0b_1^2 + 2a_0a_3b_1^3 - a_2^2b_0^2b_2 + 4a_1a_3b_0^2b_2 \\
& +a_1a_2b_0b_1b_2 - 6a_0a_3b_0b_1b_2 - a_0a_2b_1^2b_2 - a_1^2b_0b_2^2 + 2a_0a_2b_0b_2^2 + a_0a_1b_1b_2^2 \\
& -a_0^2b_2^3 - 3a_1a_2b_0^2b_3 + 6a_0a_3b_0^2b_3 + 2a_1^2b_0b_1b_3 + a_0a_2b_0b_1b_3 - 2a_0a_1b_1^2b_3 \\
& -a_0a_1b_0b_2b_3 + 3a_0^2b_1b_2b_3 - 3a_0^2b_0b_3^2)x \\
& +(a_2^3b_0^2 - 3a_1a_2a_3b_0^2 + 3a_0a_3^2b_0^2 - a_1a_2^2b_0b_1 + 2a_1^2a_3b_0b_1 + a_0a_2a_3b_0b_1 + a_0a_2^2b_1^2 \\
& -2a_0a_1a_3b_1^2 + a_1^2a_2b_0b_2 - 2a_0a_2^2b_0b_2 - a_0a_1a_3b_0b_2 - a_0a_1a_2b_1b_2 + 3a_0^2a_3b_1b_2 \\
& +a_0^2a_2b_2^2 - 2a_1^3b_0b_3 + 6a_0a_1a_2b_0b_3 - 6a_0^2a_3b_0b_3 + 2a_0a_1^2b_1b_3 - 4a_0^2a_2b_1b_3
\end{aligned}
$$

$$
\begin{aligned}
&\quad -2a_0^2a_1b_2b_3 + 3a_0^3b_3^2)y \\
&+(3a_3b_0^3 - a_2b_0^2b_1 + a_1b_0b_1^2 - a_0b_1^3 - 2a_1b_0^2b_2 + 3a_0b_0b_1b_2 - 3a_0b_0^2b_3)x^2 \\
&+(3a_1a_2b_0^2 - 6a_0a_3b_0^2 - 2a_1^2b_0b_1 - a_0a_2b_0b_1 + 2a_0a_1b_1^2 + a_0a_1b_0b_2 - 3a_0^2b_1b_2 \\
&\quad +6a_0^2b_0b_3)xy \\
&+(a_1^3b_0 - 3a_0a_1a_2b_0 + 3a_0^2a_3b_0 - a_0a_1^2b_1 + 2a_0^2a_2b_1 + a_0^2a_1b_2 - 3a_0^3b_3)y^2 \\
&-b_0^3x^3 + 3a_0b_0^2x^2y - 3a_0^2b_0xy^2 + a_0^3y^3 \quad = 0
\end{aligned}
$$

Depending on the shape of the curve, the degree of the Bézier curve may be reduced to a quadratic curve, i.e., when $a_0 = 0$ and $b_0 = 0$. In this case, the left-hand side of the implicit function becomes 0, and the roots cannot be obtained. For this case, an implicit representation for quadratic curves is used.

- Implicit representation when the curve $\boldsymbol{B}_0$ is quadratic:
  $$
  \begin{aligned}
  F_0(x,y) &= a_3^2b_1^2 - a_2a_3b_1b_2 + a_1a_3b_2^2 + a_2^2b_1b_3 - 2a_1a_3b_1b_3 - a_1a_2b_2b_3 + a_1^2b_3^2 \\
  &\quad +(-2a_3b_1^2 + a_2b_1b_2 - a_1b_2^2 + 2a_1b_1b_3)x + (-a_2^2b_1 + 2a_1a_3b_1 + a_1a_2b_2 - 2a_1^2b_3)y \\
  &\quad +b_1^2x^2 - 2a_1b_1xy + a_1^2y^2 \quad = 0
  \end{aligned}
  $$

Furthermore, when the curve is linear, i.e., when $a_0 = 0$, $a_1 = 0$, $b_0 = 0$ and $b_1 = 0$, an implicit representation for linear curves is used.

- Implicit representation when the curve $\boldsymbol{B}_0$ is linear:
  $$
  F_0(x,y) = -a_3b_2 + a_2b_3 + b_2x - a_2y \quad = 0
  $$

By substituting the parametric functions $x = f_1(u)$ and $y = g_1(u)$ of curve $\boldsymbol{B}_1$ into the implicit function $F_0(x,y) = 0$ of curve $\boldsymbol{B}_0$, we obtain an implicit equation $F_0(u) = 0$ with parameter u. The real roots of this equation are the candidate parameters of the intersection points. Table 5 through 7 present the formulas for determining the coefficients of each term of the implicit equation $F_0(u) = 0$. These were derived using SageMath (Notebook: Appendix B).

Table 5: Coefficients of each term of the implicit equation $F_0(u) = 0$ (a 9th-degree algebraic equation) when $\boldsymbol{B}_0$ is cubic

Term	Coefficient
u^9	$-b_0^3c_0^3 + 3a_0b_0^2c_0^2d_0 - 3a_0^2b_0c_0d_0^2 + a_0^3d_0^3$
u^8	$-3b_0^3c_0^2c_1 + 6a_0b_0^2c_0c_1d_0 - 3a_0^2b_0c_1d_0^2 + 3(a_0b_0^2c_0^2 - 2a_0^2b_0c_0d_0$ $+a_0^3d_0^2)d_1$
u^7	$-3b_0^3c_0c_1^2 - 3b_0^3c_0^2c_2 - 3a_0^2b_0c_2d_0^2 - 3(a_0^2b_0c_0 - a_0^3d_0)d_1^2 + 3(a_0b_0^2c_1^2$ $+2a_0b_0^2c_0c_2)d_0 + 6(a_0b_0^2c_0c_1 - a_0^2b_0c_1d_0)d_1 + 3(a_0b_0^2c_0^2 - 2a_0^2b_0c_0d_0$ $+a_0^3d_0^2)d_2$
u^6	$-b_0^3c_1^3 - 6b_0^3c_0c_1c_2 - 3b_0^3c_0^2c_3 - 3a_0^2b_0c_1d_1^2 + a_0^3d_1^3 + (3a_3b_0^3 - a_2b_0^2b_1$ $+a_1b_0b_1^2 - a_0b_1^3 - 3a_0b_0^2b_3 - (2a_1b_0^2 - 3a_0b_0b_1)b_2)c_0^2 + (a_0^2a_1b_2$ $-3a_0^3b_3 - 3a_0^2b_0c_3 + (a_1^3 - 3a_0a_1a_2 + 3a_0^2a_3)b_0 - (a_0a_1^2 - 2a_0^2a_2)b_1)d_0^2$ $+(6a_0b_0^2c_1c_2 + 6a_0b_0^2c_0c_3 + (2a_0a_1b_1^2 + 6a_0^2b_0b_3 + 3(a_1a_2 - 2a_0a_3)b_0^2$ $-(2a_1^2 + a_0a_2)b_0b_1 + (a_0a_1b_0 - 3a_0^2b_1)b_2)c_0)d_0 + 3(a_0b_0^2c_1^2$ $+2a_0b_0^2c_0c_2 - 2a_0^2b_0c_2d_0)d_1 + 6(a_0b_0^2c_0c_1 - a_0^2b_0c_1d_0 - (a_0^2b_0c_0$ $-a_0^3d_0)d_1)d_2 + 3(a_0b_0^2c_0^2 - 2a_0^2b_0c_0d_0 + a_0^3d_0^2)d_3$
u^5	$-3b_0^3c_1^2c_2 - 3b_0^3c_0c_2^2 - 6b_0^3c_0c_1c_3 - 3a_0^2b_0c_2d_1^2 + 2(3a_3b_0^3 - a_2b_0^2b_1$ $+a_1b_0b_1^2 - a_0b_1^3 - 3a_0b_0^2b_3 - (2a_1b_0^2 - 3a_0b_0b_1)b_2)c_0c_1 - 3(a_0^2b_0c_0$ $-a_0^3d_0)d_2^2 + (3a_0b_0^2c_2^2 + 6a_0b_0^2c_1c_3 + (2a_0a_1b_1^2 + 6a_0^2b_0b_3 + 3(a_1a_2$ $-2a_0a_3)b_0^2 - (2a_1^2 + a_0a_2)b_0b_1 + (a_0a_1b_0 - 3a_0^2b_1)b_2)c_1)d_0$ $+(6a_0b_0^2c_1c_2 + 6a_0b_0^2c_0c_3 + (2a_0a_1b_1^2 + 6a_0^2b_0b_3 + 3(a_1a_2 - 2a_0a_3)b_0^2$ $-(2a_1^2 + a_0a_2)b_0b_1 + (a_0a_1b_0 - 3a_0^2b_1)b_2)c_0 + 2(a_0^2a_1b_2 - 3a_0^3b_3$ $-3a_0^2b_0c_3 + (a_1^3 - 3a_0a_1a_2 + 3a_0^2a_3)b_0 - (a_0a_1^2 - 2a_0^2a_2)b_1)d_0)d_1$ $+3(a_0b_0^2c_1^2 + 2a_0b_0^2c_0c_2 - 2a_0^2b_0c_2d_0 - 2a_0^2b_0c_1d_1 + a_0^3d_1^2)d_2$ $+6(a_0b_0^2c_0c_1 - a_0^2b_0c_1d_0 - (a_0^2b_0c_0 - a_0^3d_0)d_1)d_3$
u^4	$-3b_0^3c_1c_2^2 + (3a_3b_0^3 - a_2b_0^2b_1 + a_1b_0b_1^2 - a_0b_1^3 - 3a_0b_0^2b_3 - (2a_1b_0^2$ $-3a_0b_0b_1)b_2)c_1^2 + 2(3a_3b_0^3 - a_2b_0^2b_1 + a_1b_0b_1^2 - a_0b_1^3 - 3a_0b_0^2b_3$ $-(2a_1b_0^2 - 3a_0b_0b_1)b_2)c_0c_2 + (a_0^2a_1b_2 - 3a_0^3b_3 - 3a_0^2b_0c_3 + (a_1^3$ $-3a_0a_1a_2 + 3a_0^2a_3)b_0 - (a_0a_1^2 - 2a_0^2a_2)b_1)d_1^2 - 3(a_0^2b_0c_1 - a_0^3d_1)d_2^2$ $-3(b_0^3c_1^2 + 2b_0^3c_0c_2)c_3 + (6a_0b_0^2c_2c_3 + (2a_0a_1b_1^2 + 6a_0^2b_0b_3 + 3(a_1a_2$ $-2a_0a_3)b_0^2 - (2a_1^2 + a_0a_2)b_0b_1 + (a_0a_1b_0 - 3a_0^2b_1)b_2)c_2)d_0 + (3a_0b_0^2c_2^2$ $+6a_0b_0^2c_1c_3 + (2a_0a_1b_1^2 + 6a_0^2b_0b_3 + 3(a_1a_2 - 2a_0a_3)b_0^2 - (2a_1^2$ $+a_0a_2)b_0b_1 + (a_0a_1b_0 - 3a_0^2b_1)b_2)c_1)d_1 + (6a_0b_0^2c_1c_2 + 6a_0b_0^2c_0c_3$ $-6a_0^2b_0c_2d_1 + (2a_0a_1b_1^2 + 6a_0^2b_0b_3 + 3(a_1a_2 - 2a_0a_3)b_0^2 - (2a_1^2$ $+a_0a_2)b_0b_1 + (a_0a_1b_0 - 3a_0^2b_1)b_2)c_0 + 2(a_0^2a_1b_2 - 3a_0^3b_3 - 3a_0^2b_0c_3$ $+(a_1^3 - 3a_0a_1a_2 + 3a_0^2a_3)b_0 - (a_0a_1^2 - 2a_0^2a_2)b_1)d_0)d_2 + 3(a_0b_0^2c_1^2$ $+2a_0b_0^2c_0c_2 - 2a_0^2b_0c_2d_0 - 2a_0^2b_0c_1d_1 + a_0^3d_1^2 - 2(a_0^2b_0c_0 - a_0^3d_0)d_2)d_3$

Continued on next page

(Continued)

u^3	$-b_0^3c_2^3-3b_0^3c_0c_3^2-3a_0^2b_0c_2d_2^2+a_0^3d_2^3+2(3a_3b_0^3-a_2b_0^2b_1+a_1b_0b_1^2$ $-a_0b_1^3-3a_0b_0^2b_3-(2a_1b_0^2-3a_0b_0b_1)b_2)c_1c_2-3(a_0^2b_0c_0-a_0^3d_0)d_3^2$ $-(3a_3^2b_0^3-2a_2a_3b_0^2b_1+2a_1a_3b_0b_1^2-2a_0a_3b_1^3+a_0^2b_2^3+3a_0^2b_0b_3^2$ $-(a_0a_1b_1-(a_1^2-2a_0a_2)b_0)b_2^2+(a_0a_2b_1^2+(a_2^2-4a_1a_3)b_0^2-(a_1a_2$ $-6a_0a_3)b_0b_1)b_2+(2a_0a_1b_1^2+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1$ $+(a_0a_1b_0-3a_0^2b_1)b_2)b_3)c_0-2(3b_0^3c_1c_2-(3a_3b_0^3-a_2b_0^2b_1+a_1b_0b_1^2$ $-a_0b_1^3-3a_0b_0^2b_3-(2a_1b_0^2-3a_0b_0b_1)b_2)c_0)c_3+(a_0^2a_2b_2^2+3a_0^3b_3^2$ $+3a_0b_0^2c_3^2+(a_2^3-3a_1a_2a_3+3a_0a_3^2)b_0^2-(a_1a_2^2-(2a_1^2+a_0a_2)a_3)b_0b_1$ $+(a_0a_2^2-2a_0a_1a_3)b_1^2+((a_1^2a_2-2a_0a_2^2-a_0a_1a_3)b_0-(a_0a_1a_2$ $-3a_0^2a_3)b_1)b_2-2(a_0^2a_1b_2+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2$ $-2a_0^2a_2)b_1)b_3+(2a_0a_1b_1^2+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2$ $+a_0a_2)b_0b_1+(a_0a_1b_0-3a_0^2b_1)b_2)c_3)d_0+(6a_0b_0^2c_2c_3+(2a_0a_1b_1^2$ $+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0$ $-3a_0^2b_1)b_2)c_2)d_1+(3a_0b_0^2c_2^2+6a_0b_0^2c_1c_3+(2a_0a_1b_1^2+6a_0^2b_0b_3$ $+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0-3a_0^2b_1)b_2)c_1$ $+2(a_0^2a_1b_2-3a_0^3b_3-3a_0^2b_0c_3+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2$ $-2a_0^2a_2)b_1)d_1)d_2+(6a_0b_0^2c_1c_2+6a_0b_0^2c_0c_3-6a_0^2b_0c_2d_1+(2a_0a_1b_1^2$ $+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0$ $-3a_0^2b_1)b_2)c_0+2(a_0^2a_1b_2-3a_0^3b_3-3a_0^2b_0c_3+(a_1^3-3a_0a_1a_2$ $+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)d_0-6(a_0^2b_0c_1-a_0^3d_1)d_2)d_3$
u^2	$-3b_0^3c_1c_3^2+(3a_3b_0^3-a_2b_0^2b_1+a_1b_0b_1^2-a_0b_1^3-3a_0b_0^2b_3-(2a_1b_0^2$ $-3a_0b_0b_1)b_2)c_2^2+(a_0^2a_1b_2-3a_0^3b_3-3a_0^2b_0c_3+(a_1^3-3a_0a_1a_2$ $+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)d_2^2-3(a_0^2b_0c_1-a_0^3d_1)d_3^2-(3a_3^2b_0^3$ $-2a_2a_3b_0^2b_1+2a_1a_3b_0b_1^2-2a_0a_3b_1^3+a_0^2b_2^3+3a_0^2b_0b_3^2-(a_0a_1b_1-(a_1^2$ $-2a_0a_2)b_0)b_2^2+(a_0a_2b_1^2+(a_2^2-4a_1a_3)b_0^2-(a_1a_2-6a_0a_3)b_0b_1)b_2$ $+(2a_0a_1b_1^2+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0$ $-3a_0^2b_1)b_2)b_3)c_1-(3b_0^3c_2^2-2(3a_3b_0^3-a_2b_0^2b_1+a_1b_0b_1^2-a_0b_1^3$ $-3a_0b_0^2b_3-(2a_1b_0^2-3a_0b_0b_1)b_2)c_1)c_3+(a_0^2a_2b_2^2+3a_0^3b_3^2+3a_0b_0^2c_3^2$ $+(a_2^3-3a_1a_2a_3+3a_0a_3^2)b_0^2-(a_1a_2^2-(2a_1^2+a_0a_2)a_3)b_0b_1+(a_0a_2^2$ $-2a_0a_1a_3)b_1^2+((a_1^2a_2-2a_0a_2^2-a_0a_1a_3)b_0-(a_0a_1a_2-3a_0^2a_3)b_1)b_2$ $-2(a_0^2a_1b_2+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)b_3$ $+(2a_0a_1b_1^2+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1$ $+(a_0a_1b_0-3a_0^2b_1)b_2)c_3)d_1+(6a_0b_0^2c_2c_3+(2a_0a_1b_1^2+6a_0^2b_0b_3$ $+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0$ $-3a_0^2b_1)b_2)c_2)d_2+(3a_0b_0^2c_2^2+6a_0b_0^2c_1c_3-6a_0^2b_0c_2d_2+3a_0^3d_2^2$ $+(2a_0a_1b_1^2+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1$ $+(a_0a_1b_0-3a_0^2b_1)b_2)c_1+2(a_0^2a_1b_2-3a_0^3b_3-3a_0^2b_0c_3+(a_1^3$ $-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)d_1)d_3$

Continued on next page

(Continued)

u	$-3b_0^3c_2c_3^2+2(3a_3b_0^3-a_2b_0^2b_1+a_1b_0b_1^2-a_0b_1^3-3a_0b_0^2b_3-(2a_1b_0^2$ $-3a_0b_0b_1)b_2)c_2c_3-3(a_0^2b_0c_2-a_0^3d_2)d_3^2-(3a_3^2b_0^3-2a_2a_3b_0^2b_1$ $+2a_1a_3b_0b_1^2-2a_0a_3b_1^3+a_0^2b_2^3+3a_0^2b_0b_3^2-(a_0a_1b_1-(a_1^2$ $-2a_0a_2)b_0)b_2^2+(a_0a_2b_1^2+(a_2^2-4a_1a_3)b_0^2-(a_1a_2-6a_0a_3)b_0b_1)b_2$ $+(2a_0a_1b_1^2+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0$ $-3a_0^2b_1)b_2)b_3)c_2+(a_0^2a_2b_2^2+3a_0^3b_3^2+3a_0b_0^2c_3^2+(a_2^3-3a_1a_2a_3$ $+3a_0a_3^2)b_0^2-(a_1a_2^2-(2a_1^2+a_0a_2)a_3)b_0b_1+(a_0a_2^2-2a_0a_1a_3)b_1^2$ $+((a_1^2a_2-2a_0a_2^2-a_0a_1a_3)b_0-(a_0a_1a_2-3a_0^2a_3)b_1)b_2-2(a_0^2a_1b_2$ $+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)b_3+(2a_0a_1b_1^2$ $+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0$ $-3a_0^2b_1)b_2)c_3)d_2+(6a_0b_0^2c_2c_3+(2a_0a_1b_1^2+6a_0^2b_0b_3+3(a_1a_2$ $-2a_0a_3)b_0^2-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0-3a_0^2b_1)b_2)c_2+2(a_0^2a_1b_2$ $-3a_0^3b_3-3a_0^2b_0c_3+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2$ $-2a_0^2a_2)b_1)d_2)d_3$
1	$a_3^3b_0^3-a_2a_3^2b_0^2b_1+a_1a_3^2b_0b_1^2-a_0a_3^2b_1^3+a_0^2a_3b_2^3-a_0^3b_3^3-b_0^3c_3^3$ $+a_0^3d_3^3-(a_0a_1a_3b_1-(a_1^2-2a_0a_2)a_3b_0)b_2^2+(a_0^2a_1b_2+(a_1^3$ $-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)b_3^2+(3a_3b_0^3-a_2b_0^2b_1$ $+a_1b_0b_1^2-a_0b_1^3-3a_0b_0^2b_3-(2a_1b_0^2-3a_0b_0b_1)b_2)c_3^2+(a_0^2a_1b_2$ $-3a_0^3b_3-3a_0^2b_0c_3+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2-2a_0^2a_2)b_1)d_3^2$ $+(a_0a_2a_3b_1^2+(a_2^2a_3-2a_1a_3^2)b_0^2-(a_1a_2a_3-3a_0a_3^2)b_0b_1)b_2-(a_0^2a_2b_2^2$ $+(a_2^3-3a_1a_2a_3+3a_0a_3^2)b_0^2-(a_1a_2^2-(2a_1^2+a_0a_2)a_3)b_0b_1+(a_0a_2^2$ $-2a_0a_1a_3)b_1^2+((a_1^2a_2-2a_0a_2^2-a_0a_1a_3)b_0-(a_0a_1a_2$ $-3a_0^2a_3)b_1)b_2)b_3-(3a_3^2b_0^3-2a_2a_3b_0^2b_1+2a_1a_3b_0b_1^2-2a_0a_3b_1^3$ $+a_0^2b_2^3+3a_0^2b_0b_3^2-(a_0a_1b_1-(a_1^2-2a_0a_2)b_0)b_2^2+(a_0a_2b_1^2+(a_2^2$ $-4a_1a_3)b_0^2-(a_1a_2-6a_0a_3)b_0b_1)b_2+(2a_0a_1b_1^2+3(a_1a_2-2a_0a_3)b_0^2$ $-(2a_1^2+a_0a_2)b_0b_1+(a_0a_1b_0-3a_0^2b_1)b_2)b_3)c_3+(a_0^2a_2b_2^2+3a_0^3b_3^2$ $+3a_0b_0^2c_3^2+(a_2^3-3a_1a_2a_3+3a_0a_3^2)b_0^2-(a_1a_2^2-(2a_1^2+a_0a_2)a_3)b_0b_1$ $+(a_0a_2^2-2a_0a_1a_3)b_1^2+((a_1^2a_2-2a_0a_2^2-a_0a_1a_3)b_0-(a_0a_1a_2$ $-3a_0^2a_3)b_1)b_2-2(a_0^2a_1b_2+(a_1^3-3a_0a_1a_2+3a_0^2a_3)b_0-(a_0a_1^2$ $-2a_0^2a_2)b_1)b_3+(2a_0a_1b_1^2+6a_0^2b_0b_3+3(a_1a_2-2a_0a_3)b_0^2-(2a_1^2$ $+a_0a_2)b_0b_1+(a_0a_1b_0-3a_0^2b_1)b_2)c_3)d_3$

Table 6: Coefficients of each term of the implicit equation $F_0(u) = 0$ (a 6th-degree algebraic equation) when $\boldsymbol{B}_0$ is quadratic

Term	Coefficient
u^6	$b_1^2c_0^2 - 2a_1b_1c_0d_0 + a_1^2d_0^2$
u^5	$2b_1^2c_0c_1 - 2a_1b_1c_1d_0 - 2(a_1b_1c_0 - a_1^2d_0)d_1$
u^4	$b_1^2c_1^2+2b_1^2c_0c_2-2a_1b_1c_2d_0-2a_1b_1c_1d_1+a_1^2d_1^2-2(a_1b_1c_0-a_1^2d_0)d_2$
u^3	$2b_1^2c_1c_2+2b_1^2c_0c_3-2a_1b_1c_2d_1-(2a_3b_1^2-a_2b_1b_2+a_1b_2^2-2a_1b_1b_3)c_0$ $+(a_1a_2b_2 - 2a_1^2b_3 - 2a_1b_1c_3 - (a_2^2 - 2a_1a_3)b_1)d_0 - 2(a_1b_1c_1$ $-a_1^2d_1)d_2 - 2(a_1b_1c_0 - a_1^2d_0)d_3$
u^2	$b_1^2c_2^2 + 2b_1^2c_1c_3 - 2a_1b_1c_2d_2 + a_1^2d_2^2 - (2a_3b_1^2 - a_2b_1b_2 + a_1b_2^2$ $-2a_1b_1b_3)c_1 + (a_1a_2b_2 - 2a_1^2b_3 - 2a_1b_1c_3 - (a_2^2 - 2a_1a_3)b_1)d_1$ $-2(a_1b_1c_1 - a_1^2d_1)d_3$
u	$2b_1^2c_2c_3 - (2a_3b_1^2 - a_2b_1b_2 + a_1b_2^2 - 2a_1b_1b_3)c_2 + (a_1a_2b_2 - 2a_1^2b_3$ $-2a_1b_1c_3 - (a_2^2 - 2a_1a_3)b_1)d_2 - 2(a_1b_1c_2 - a_1^2d_2)d_3$
1	$a_3^2b_1^2 - a_2a_3b_1b_2 + a_1a_3b_2^2 + a_1^2b_3^2 + b_1^2c_3^2 + a_1^2d_3^2 - (a_1a_2b_2 - (a_2^2$ $-2a_1a_3)b_1)b_3 - (2a_3b_1^2 - a_2b_1b_2 + a_1b_2^2 - 2a_1b_1b_3)c_3 + (a_1a_2b_2$ $-2a_1^2b_3 - 2a_1b_1c_3 - (a_2^2 - 2a_1a_3)b_1)d_3$

Table 7: Coefficients of each term of the implicit equation $F_0(u) = 0$ (a 3rd-degree algebraic equation) when $\boldsymbol{B}_0$ is linear

Term	Coefficient
u^3	$b_2c_0 - a_2d_0$
u^2	$b_2c_1 - a_2d_1$
u	$b_2c_2 - a_2d_2$
1	$-a_3b_2 + a_2b_3 + b_2c_3 - a_2d_3$

2.6 Real-Root Isolation

2.6.1 Narrowing Down Using Budan–Fourier's Theorem

In practical applications of Bézier curve intersection detection, it is assumed that most Bézier curves either do not intersect or intersect at only one point, and that curves with two or more intersection points are extremely rare. Therefore, it is expected that narrowing down will significantly reduce the average computational cost.

For narrowing down, Budan–Fourier's theorem, which has low computational complexity, is used. Budan–Fourier's theorem for the equation $p(x) = 0$ consists of the following two statements [9]:

- Budan's statement: using the difference in the number of sign variations of the coefficients of the transformed polynomials. (the difference between the number of sign variations in the coefficients of $p(x + a)$ and $p(x + b)$)

- Fourier's statement: using the difference in the number of sign variations of the derivative sequences. (the difference between the number of sign variations of $\{p(a), p^{(1)}(a), p^{(2)}(a), \ldots, p^{(9)}(a)\}$ at $x = a$ and $\{p(b), p^{(1)}(b), p^{(2)}(b), \ldots, p^{(9)}(b)\}$ at $x = b$)

In this study, both methods were computed and confirmed to produce identical results during verification on desk, and the former method, which is easier to design, was adopted in the trial implementation.

To target the entire Bézier curve, calculate the difference in the number of sign variations when $a = 0$ and $b = 1$, and subsequent processing is decided accordingly:

- No sign variations $\implies$ No real roots in the interval $(0, 1)$ $\implies$ No intersections.
- Number of sign variations is 1 $\implies$ There is one real root in the interval $(0, 1)$ $\implies$ The entire interval is treated as an isolated interval, and an iterative root-finding method is performed.
- Otherwise $\implies$ Real-root isolation is performed using Vincent's theorem.

For the polynomial
$p(x) = e_0x^9 + e_1x^8 + e_2x^7 + e_3x^6 + e_4x^5 + e_5x^4 + e_6x^3 + e_7x^2 + e_8x + e_9$
the transformed polynomial for $b = 1$ is:
$$p(x+1) = e_0(x+1)^9 + e_1(x+1)^8 + e_2(x+1)^7 + e_3(x+1)^6 + e_4(x+1)^5 + e_5(x+1)^4 + e_6(x+1)^3 + e_7(x+1)^2 + e_8(x+1) + e_9$$
, and the coefficients of each degree of the transformed polynomial $p(x+1)$ are presented in Table 8. These were derived using SageMath (Notebook: Appendix C).

Table 8: Coefficients of transformed polynomial $p(x + 1)$

Term	Coefficient
x^9	e_0
x^8	$9e_0 + e_1$
x^7	$36e_0 + 8e_1 + e_2$
x^6	$84e_0 + 28e_1 + 7e_2 + e_3$
x^5	$126e_0 + 56e_1 + 21e_2 + 6e_3 + e_4$
x^4	$126e_0 + 70e_1 + 35e_2 + 15e_3 + 5e_4 + e_5$
x^3	$84e_0 + 56e_1 + 35e_2 + 20e_3 + 10e_4 + 4e_5 + e_6$
x^2	$36e_0 + 28e_1 + 21e_2 + 15e_3 + 10e_4 + 6e_5 + 3e_6 + e_7$
x	$9e_0 + 8e_1 + 7e_2 + 6e_3 + 5e_4 + 4e_5 + 3e_6 + 2e_7 + e_8$
1	$e_0 + e_1 + e_2 + e_3 + e_4 + e_5 + e_6 + e_7 + e_8 + e_9$

2.6.2 Real-Root Isolation Using Vincent's Theorem

Using the VAG method (Vincent–Alesina–Galuzzi), an application of Vincent's theorem, intervals containing exactly one real root are determined [9].

For an interval (a, b), the transformed polynomial of $p(x)$ is

$$f(x) = (x+1)^{deg(p)} p\left(\frac{a+bx}{1+x}\right)$$

The number of sign variations in the coefficient sequence of $f(x)$ is evaluated:

- If 0 $\implies$ no real roots $\implies$ interval (a, b) is discarded
- If 1 $\implies$ exactly one real root $\implies$ interval (a, b) is output as an isolated interval
- If 2 or more $\implies$ at least one real root $\implies$ split the interval (a, b) at the midpoint m, and recursively apply the VAG method to (a, m) and (m, b)

Since the entire Bézier curve is considered, the initial values are set to $a = 0$ and $b = 1$. For the polynomial

$p(x) = e_0x^9 + e_1x^8 + e_2x^7 + e_3x^6 + e_4x^5 + e_5x^4 + e_6x^3 + e_7x^2 + e_8x + e_9$,

the coefficients of each degree of the transformed polynomial $f(x)$ are presented in Table 9. These were derived using SageMath (Notebook: Appendix D).



Figure 2: An example in which the implicit equation $F_0(u) = 0$ from **Ex.3** is subjected to iterative interval subdivision using the VAG method for real-root isolation
Red box: discarded intervals with no real roots
Blue box: intervals that contain exactly one real root and for which isolation is complete

Table 9: Coefficients of transformed polynomial $f(x) = (x+1)^{deg(p)} p\left(\frac{a+bx}{1+x}\right)$

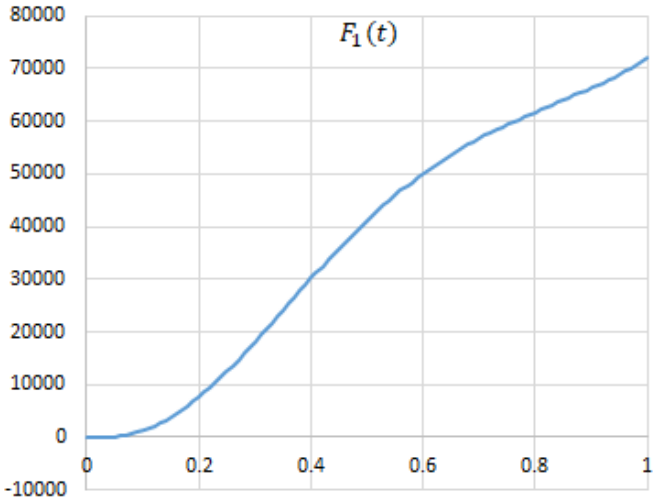
Term	Coefficient
x^9	$e_0b^9 + e_1b^8 + e_2b^7 + e_3b^6 + e_4b^5 + e_5b^4 + e_6b^3 + e_7b^2 + e_8b + e_9$
x^8	$9ae_0 + e_1)b^8 + 2(4ae_1 + e_2)b^7 + (7ae_2 + 3e_3)b^6 + 2(3ae_3 + 2e_4)b^5$ $+5(ae_4 + e_5)b^4 + 2(2ae_5 + 3e_6)b^3 + (3ae_6 + 7e_7)b^2 + ae_8 + 2(ae_7$ $+4e_8)b + 9e_9$
x^7	$(36a^2e_0 + 8ae_1 + e_2)b^7 + (28a^2e_1 + 14ae_2 + 3e_3)b^6 + 3(7a^2e_2$ $+6ae_3 + 2e_4)b^5 + 5(3a^2e_3 + 4ae_4 + 2e_5)b^4 + 5(2a^2e_4 + 4ae_5$ $+3e_6)b^3 + a^2e_7 + 3(2a^2e_5 + 6ae_6 + 7e_7)b^2 + 8ae_8 + (3a^2e_6 + 14ae_7$ $+28e_8)b + 36e_9$
x^6	$(84a^3e_0 + 28a^2e_1 + 7ae_2 + e_3)b^6 + 2(28a^3e_1 + 21a^2e_2 + 9ae_3$ $+2e_4)b^5 + 5(7a^3e_2 + 9a^2e_3 + 6ae_4 + 2e_5)b^4 + a^3e_6 + 20(a^3e_3$ $+2a^2e_4 + 2ae_5 + e_6)b^3 + 7a^2e_7 + 5(2a^3e_4 + 6a^2e_5 + 9ae_6 + 7e_7)b^2$ $+28ae_8 + 2(2a^3e_5 + 9a^2e_6 + 21ae_7 + 28e_8)b + 84e_9$
x^5	$(126a^4e_0 + 56a^3e_1 + 21a^2e_2 + 6ae_3 + e_4)b^5 + a^4e_5 + 5(14a^4e_1$ $+14a^3e_2 + 9a^2e_3 + 4ae_4 + e_5)b^4 + 6a^3e_6 + 5(7a^4e_2 + 12a^3e_3$ $+12a^2e_4 + 8ae_5 + 3e_6)b^3 + 21a^2e_7 + 5(3a^4e_3 + 8a^3e_4 + 12a^2e_5$ $+12ae_6 + 7e_7)b^2 + 56ae_8 + 5(a^4e_4 + 4a^3e_5 + 9a^2e_6 + 14ae_7$ $+14e_8)b + 126e_9$
x^4	$a^5e_4 + 5a^4e_5 + (126a^5e_0 + 70a^4e_1 + 35a^3e_2 + 15a^2e_3 + 5ae_4 + e_5)b^4$ $+15a^3e_6 + 2(28a^5e_1 + 35a^4e_2 + 30a^3e_3 + 20a^2e_4 + 10ae_5 + 3e_6)b^3$ $+35a^2e_7 + 3(7a^5e_2 + 15a^4e_3 + 20a^3e_4 + 20a^2e_5 + 15ae_6 + 7e_7)b^2$ $+70ae_8 + 2(3a^5e_3 + 10a^4e_4 + 20a^3e_5 + 30a^2e_6 + 35ae_7 + 28e_8)b$ $+126e_9$
x^3	$a^6e_3 + 4a^5e_4 + 10a^4e_5 + 20a^3e_6 + (84a^6e_0 + 56a^5e_1 + 35a^4e_2$ $+20a^3e_3 + 10a^2e_4 + 4ae_5 + e_6)b^3 + 35a^2e_7 + (28a^6e_1 + 42a^5e_2$ $+45a^4e_3 + 40a^3e_4 + 30a^2e_5 + 18ae_6 + 7e_7)b^2 + 56ae_8 + (7a^6e_2$ $+18a^5e_3 + 30a^4e_4 + 40a^3e_5 + 45a^2e_6 + 42ae_7 + 28e_8)b + 84e_9$
x^2	$a^7e_2 + 3a^6e_3 + 6a^5e_4 + 10a^4e_5 + 15a^3e_6 + 21a^2e_7 + (36a^7e_0$ $+28a^6e_1 + 21a^5e_2 + 15a^4e_3 + 10a^3e_4 + 6a^2e_5 + 3ae_6 + e_7)b^2$ $+28ae_8 + 2(4a^7e_1 + 7a^6e_2 + 9a^5e_3 + 10a^4e_4 + 10a^3e_5 + 9a^2e_6$ $+7ae_7 + 4e_8)b + 36e_9$
x	$a^8e_1 + 2a^7e_2 + 3a^6e_3 + 4a^5e_4 + 5a^4e_5 + 6a^3e_6 + 7a^2e_7 + 8ae_8$ $+(9a^8e_0 + 8a^7e_1 + 7a^6e_2 + 6a^5e_3 + 5a^4e_4 + 4a^3e_5 + 3a^2e_6 + 2ae_7$ $+e_8)b + 9e_9$
1	$a^9e_0 + a^8e_1 + a^7e_2 + a^6e_3 + a^5e_4 + a^4e_5 + a^3e_6 + a^2e_7 + ae_8 + e_9$

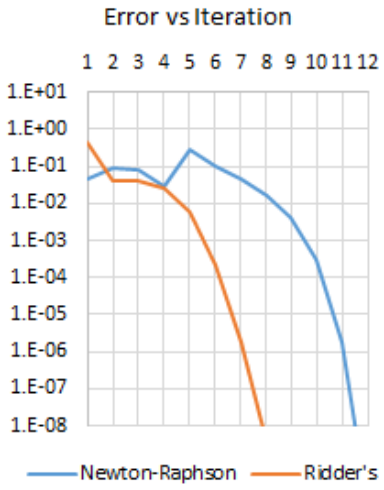
2.7 Iterative Root-Finding Methods

2.7.1 Interval Reduction Using the Bisection Method

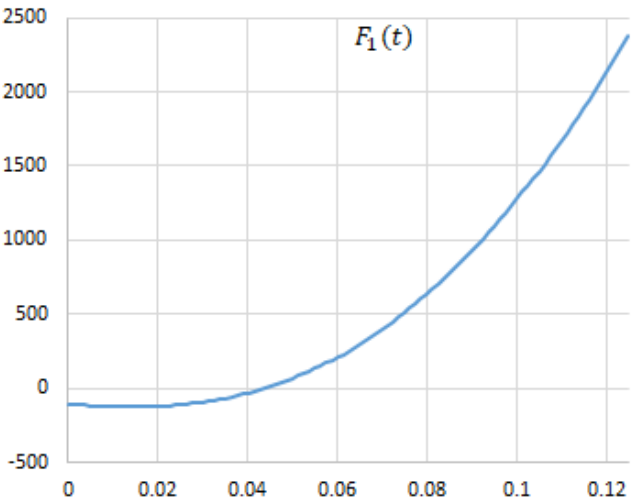
When singular points such as local minima or maxima are included within an interval, convergence may deteriorate significantly, or the Newton–Raphson method may converge to a root outside the isolated interval. Therefore, for isolated intervals that contain a unique real root, an additional step was introduced in which the bisection method is applied three times to reduce the interval to one-eighth of its original size. **Ex.2** demonstrates improved convergence due to the addition of the bisection method.

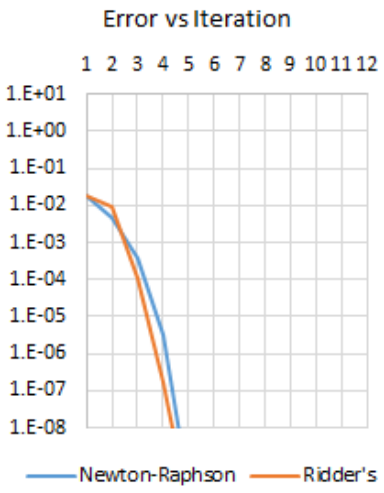
- without bisection method:





- reduced to one-eighth with bisection method:



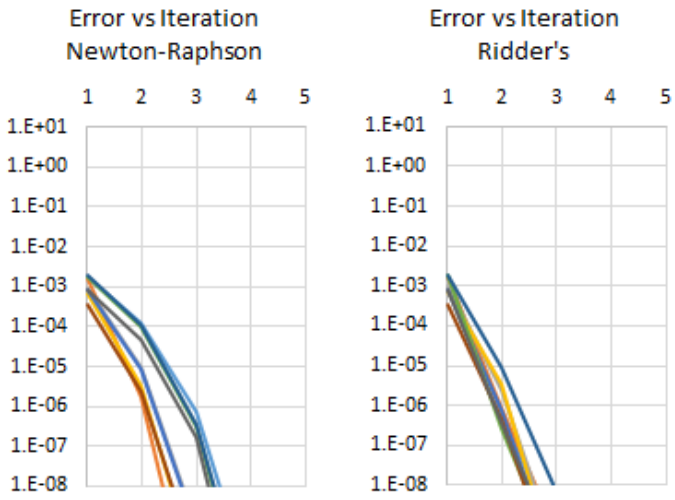


2.7.2 Root Finding Using Ridder's Method

Ridder's method is applied to the reduced isolated intervals. Ridder's method is a root-finding algorithm that iteratively approximates the root of a continuous function $f(x)$ using fixed-point iteration and exponential functions. A new point x_3 between x_0 and x_2 is obtained by the following formula [11]:

$$x_3 = x_1 + (x_1 - x_0)\frac{\mathrm{sign}[f(x_0)]f(x_1)}{\sqrt{f(x_1)^2 - f(x_0)f(x_2)}}$$

In this study, Ridder's method was selected because it allows the search interval for the root to be specified, is easy to implement, and was shown to be superior in the comparative evaluation results reported in [6]. Compared with the Newton–Raphson method, it converges faster and avoids the problem of convergence to roots outside the specified interval. The following figures show the convergence graphs for each isolated interval of the implicit equation $F_0(u) = 0$ from **Ex.3**. (Left: Newton–Raphson method, Right: Ridder's method)



Ridder's method is simpler than other root-finding algorithms such as Müller's method, yet is often regarded as having comparable performance; however, some people have doubts [12]. In the sample evaluations conducted in this study, the combination of the bisection method and Ridder's method exhibited good convergence characteristics, but further investigation is required.

2.8 Handling Multiple Roots

The bisection method and Ridder's method do not support multiple roots. The VAG method also does not support them in its original form, but it was extended in this study so that, for multiple roots, subdivision is repeatedly performed until the isolated interval becomes sufficiently short, and the midpoint of the isolated interval output by the VAG method is taken as the root. Originally, Vincent's theorem and the VAG method is intended for real-root isolation. Using it also for convergence-based root finding requires further investigation in terms of reliability and computational efficiency.

The pseudocode of the VAG method from reference [10] was extended as follows to support multiple roots (the additions are shown in red):

1. $var \leftarrow$ the number of sign variations of $(x+1)^{deg(p)}p(\frac{a+bx}{1+x})$
2. if $var = 0$ then RETURN $\varnothing$;
3. if $var = 1$ then RETURN $\{(a, b)\}$;
4. $m \leftarrow \frac{1}{2}(a+b)$
5. if $b - a < M$ then RETURN $\{(m - A, m + A)\}$;
6. if $p(m) \neq 0$ then
7. RETURN VAG(p, (a, m)) $\cup$ VAG(p, (m, b))
8. else
9. $var \leftarrow$ the number of sign variations of $(x+1)^{deg(p)}p(\frac{m-A+(m+A)x}{1+x})$
10. if $var = 1$ then
11. RETURN VAG(p, (a, m)) $\cup$ $\{[m, m]\}$ $\cup$ VAG(p, (m, b))
12. else
13. RETURN VAG(p, $(a, m - A)$) $\cup$ $\{[m - A, m + A]\}$ $\cup$ VAG(p, $(m + A, b)$)
14. end
15. end

- A is half the length of the isolated interval for multiple roots, M is the minimum interval length.
- If the function value $p(m)$ at the midpoint m of the interval is zero, that is, when m is a root, and the number of roots in the interval $(m-A, m+A)$ is one, then the isolated interval $[m, m]$ is output. If the number of roots is two or more, the isolated interval $(m-A, m+A)$ is output. Note that among the examples of multiple roots in Table 3, for **Ex.6**, **Ex.8**, **Ex.10**, and **Ex.11**, the isolated interval is determined based on the latter condition.
- Even if an interval (a, b) contains two or more roots, if the interval length $b - a$ is shorter than M, the subdivision process is terminated and an isolated interval $(m-A, m+A)$ containing multiple roots is output. Note that among the examples of multiple roots in Table 3, for **Ex.7** and **Ex.9**, the isolated interval is determined based on this condition.
- If the output isolated interval has zero length, or if the interval contains two or more roots, subsequent processing (bisection method, Ridder's method) is skipped. In the latter case, the midpoint of the interval is taken as the root.

The reason for allowing M and A to be set independently is to balance the accuracy of multiple-root computation with tolerance for floating-point errors. Table 10 presents the results obtained using SageMath to compute the coefficients of the transformed polynomial in the VAG method for the root $u = 0.75$ (multiplicity 4) in **Ex.11** (contact between cusp points), for each interval $(0, 0.75 - A)$, $[0.75 - A, 0.75 + A]$ and $(0.75 + A, 1)$. To obtain the expected numbers of sign variations: $\{0, 4, 0\}$, the isolated interval needed to be relatively large (with $A \geq 10^{-3}$). Considering the coefficient values, this is presumed to be due to the influence of floating-point arithmetic errors, but further analysis is required.

Table 10: Coefficient values and number of sign variations of the transformed polynomial with respect to margin A in **Ex.11**

<table>
<tr><th>A</th><th>coefficient values
and number of sign variations
of $(x+1)^{deg(p)}p(\frac{a+bx}{1+x})$</th></tr>
<tr><td>10^{-6}</td><td>
<table>
<tr><td>a</td><td>0</td><td>0.749999</td><td>0.750001</td></tr>
<tr><td>b</td><td>0.749999</td><td>0.750001</td><td>1</td></tr>
<tr><td rowspan="11">coefficient</td><td>-5.82E-11</td><td>2.33E-10</td><td>-3.24E+01</td></tr>
<tr><td>-4.66E-10</td><td>0</td><td>-1.83E+02</td></tr>
<tr><td>-5.40E-08</td><td>0</td><td>-4.11E+02</td></tr>
<tr><td>-2.62E-02</td><td>-1.49E-08</td><td>-4.64E+02</td></tr>
<tr><td>-4.92E+03</td><td>0</td><td>-2.63E+02</td></tr>
<tr><td>-3.44E+04</td><td>0</td><td>-6.08E+01</td></tr>
<tr><td>-1.02E+05</td><td>0</td><td>-9.72E-04</td></tr>
<tr><td>-1.65E+05</td><td>1.49E-08</td><td>-7.45E-09</td></tr>
<tr><td>-1.48E+05</td><td>0</td><td>-1.86E-09</td></tr>
<tr><td>-5.81E+04</td><td>0</td><td>2.33E-10</td></tr>
<tr><td>sign variation</td><td>0</td><td>2</td><td>1</td></tr>
</table>
</td></tr>
<tr><td>10^{-5}</td><td>
<table>
<tr><td>a</td><td>0</td><td>0.74999</td><td>0.75001</td></tr>
<tr><td>b</td><td>0.74999</td><td>0.75001</td><td>1</td></tr>
<tr><td rowspan="11">coefficient</td><td>3.49E-10</td><td>0</td><td>-3.24E+01</td></tr>
<tr><td>0</td><td>0</td><td>-1.83E+02</td></tr>
<tr><td>-5.24E-06</td><td>7.45E-09</td><td>-4.11E+02</td></tr>
<tr><td>-2.62E-01</td><td>0</td><td>-4.64E+02</td></tr>
<tr><td>-4.92E+03</td><td>-8.94E-08</td><td>-2.63E+02</td></tr>
<tr><td>-3.45E+04</td><td>0</td><td>-6.08E+01</td></tr>
<tr><td>-1.02E+05</td><td>-1.49E-08</td><td>-9.72E-03</td></tr>
<tr><td>-1.65E+05</td><td>-1.49E-08</td><td>-5.96E-07</td></tr>
<tr><td>-1.48E+05</td><td>-3.73E-09</td><td>3.73E-09</td></tr>
<tr><td>-5.81E+04</td><td>-2.33E-10</td><td>0</td></tr>
<tr><td>sign variation</td><td>1</td><td>1</td><td>1</td></tr>
</table>
</td></tr>
<tr><td>10^{-4}</td><td>
<table>
<tr><td>a</td><td>0</td><td>0.7499</td><td>0.7501</td></tr>
<tr><td>b</td><td>0.7499</td><td>0.7501</td><td>1</td></tr>
<tr><td rowspan="11">coefficient</td><td>-2.33E-10</td><td>-2.33E-10</td><td>-3.24E+01</td></tr>
<tr><td>-4.75E-08</td><td>0</td><td>-1.83E+02</td></tr>
<tr><td>-5.25E-04</td><td>-1.49E-08</td><td>-4.12E+02</td></tr>
<tr><td>-2.63E+00</td><td>2.98E-08</td><td>-4.64E+02</td></tr>
<tr><td>-4.94E+03</td><td>0</td><td>-2.64E+02</td></tr>
<tr><td>-3.45E+04</td><td>0</td><td>-6.12E+01</td></tr>
<tr><td>-1.02E+05</td><td>2.98E-08</td><td>-9.74E-02</td></tr>
<tr><td>-1.65E+05</td><td>0</td><td>-5.84E-05</td></tr>
<tr><td>-1.48E+05</td><td>3.73E-09</td><td>-1.30E-08</td></tr>
<tr><td>-5.81E+04</td><td>0</td><td>-4.66E-10</td></tr>
<tr><td>sign variation</td><td>0</td><td>1</td><td>0</td></tr>
</table>
</td></tr>
<tr><td>10^{-3}</td><td>
<table>
<tr><td>a</td><td>0</td><td>0.749</td><td>0.751</td></tr>
<tr><td>b</td><td>0.749</td><td>0.751</td><td>1</td></tr>
<tr><td rowspan="11">coefficient</td><td>-1.54E-08</td><td>-1.58E-08</td><td>-3.24E+01</td></tr>
<tr><td>-4.69E-05</td><td>-1.86E-08</td><td>-1.83E+02</td></tr>
<tr><td>-5.30E-02</td><td>7.45E-08</td><td>-4.14E+02</td></tr>
<tr><td>-2.67E+01</td><td>8.94E-08</td><td>-4.70E+02</td></tr>
<tr><td>-5.12E+03</td><td>-5.96E-08</td><td>-2.70E+02</td></tr>
<tr><td>-3.51E+04</td><td>-5.96E-08</td><td>-6.48E+01</td></tr>
<tr><td>-1.03E+05</td><td>8.94E-08</td><td>-9.95E-01</td></tr>
<tr><td>-1.66E+05</td><td>7.45E-08</td><td>-5.88E-03</td></tr>
<tr><td>-1.48E+05</td><td>-1.30E-08</td><td>-1.56E-05</td></tr>
<tr><td>-5.81E+04</td><td>-1.56E-08</td><td>-1.58E-08</td></tr>
<tr><td>sign variation</td><td>0</td><td>4</td><td>0</td></tr>
</table>
</td></tr>
</table>

2.9 Intersection Determination

The candidate parameter t is substituted into $x = f_0(t)$ and $y = g_0(t)$ to obtain the coordinates of the candidate intersection point on curve $\boldsymbol{B}_0$. Similarly, the candidate parameter u is substituted into $x = f_1(u)$ and $y = g_1(u)$ to obtain the coordinates of the candidate intersection point on curve $\boldsymbol{B}_1$. Pairs that correspond to each other — i.e., those whose Euclidean distance is sufficiently small — are determined to be intersection points.

3 Implementation

This algorithm was implemented in C++/Qt. The source code is shown in Appendix E. Ridder's method was implemented in C++ by translating the Python code example provided in [13].

For numerical stability, the following measures were taken:

- Control point coordinates are affine-transformed so that their value range becomes $[0, 1]$ before intersection detection. Intersection coordinates are then computed from the original control point coordinates and the intersection parameters.
- For polynomial evaluation of degree four or higher, the implementation was rewritten to use repeated multiplication and addition. For example,
  $p(x) = e_0x^9 + e_1x^8 + e_2x^7 + e_3x^6 + e_4x^5 + e_5x^4 + e_6x^3 + e_7x^2 + e_8x + e_9$
  was implemented as follows:

```
double p_(double *e, double x){
  return (((((((e[0]*x+e[1])*x+e[2])*x+e[3])*x+e[4])*x+e[5])*x+e[6])*x
      +e[7])*x+e[8])*x+e[9];
}
```

- Judgments such as whether a value is zero were implemented with tolerance for floating-point errors. Each threshold value was experimentally determined using sample cases, and further evaluation is required.
- The VAG method was implemented using recursive calls according to the pseudocode, requiring stack frames proportional to recursion depth. In the present implementation, the stack usage of vVAG was measured to be at most 4624 bytes (using MSVC 2022).

4 Results

- Operation was verified using multiple examples, and correct intersection points were confirmed.
- For root finding, convergence using Ridder's method was sufficiently fast and stable.
- When the implicit equation contained multiple roots, the number of VAG iterations increased, but remained within a practical range.

5 Discussion

The strength of the implicitization method lies in its ability to algebraically compute intersections even in complex crossing patterns or cases involving twisting and overlapping, which tend to be unstable with conventional geometric methods.

In addition, since numerical procedures are clearly separated (real-root isolation $\Longrightarrow$ real-root extraction $\Longrightarrow$ intersection determination), the approach offers high portability and generality of implementation. It also has potential applicability to other cubic parametric curves (such as B-splines and Catmull–Rom splines), and even to surfaces.

On the other hand, practical issues require careful attention, including handling multiple roots, coefficient scaling, floating-point errors, and the stability of root-finding algorithms. In particular, the handling of multiple roots is based on a simple heuristic and is insufficient in terms of accuracy guarantees and theoretical rigor.

6 Conclusion and Future Work

This paper presented a practical method for Bézier curve intersection detection using the implicitization approach, along with its implementation and verification. As a result, it was shown that the method can handle intersection patterns that are difficult for conventional geometric approaches, demonstrating the usefulness of an algebraic approach.

Future work includes the following:

- Classification of curve intersection patterns and root types of implicit equations through analytical approaches
- Further investigation of root-finding methods (Müller's method and others, or combinations of multiple methods)
- More rigorous handling of multiple and closely spaced roots
- Improved robustness against numerical and rounding errors
- Application to real graphics/CAD/GUI systems and evaluation of performance
- Extension to other parametric curves and surfaces

References

[1] T.W. Sederberg, T. Nishita. *Curve intersection using Bézier clipping.* Computer-Aided Design, Vol.22, Issue 9, Nov.1990, pp.538-549.
http://nishitalab.org/user/nis/cdrom/cad/CAGD90Curve.pdf

[2] Thomas W. Sederberg. *Computer Aided Geometric Design*, pp.191-208. Brigham Young University ScholarsArchive – Faculty Publications.
https://scholarsarchive.byu.edu/facpub/1/

[3] T.W Sederberg, D.C Anderson, R.N Goldman. *Implicit representation of parametric curves and surfaces.* Computer Vision, Graphics, and Image Processing, Vol.28, Issue 1, Oct.1984, pp.72-84.
https://www.sciencedirect.com/science/article/abs/pii/0734189X84901403

[4] ゴッドフット企画エクセルで操る！デュラン・ケルナー・アバース法 (DKA 法) による高次代数方程式の解計算
https://godfoot.world.coocan.jp/DKA-Method.htm

[5] PukiWiki for PBCGLab – 非線形方程式の根
https://pbcglab.jp/cgi-bin/wiki/?%E9%9D%9E%E7%B7%9A%E5%BD%A2%E6%96%B9%E7%A8%8B%E5%BC%8F%E3%81%AE%E6%A0%B9

[6] Alexander Reshetov. *Exploiting Budan-Fourier and Vincent's Theorems for Ray Tracing 3D Bézier Curves.* Proceedings of HPG '17, Los Angeles, CA, USA, July 28-30, 2017
https://research.nvidia.com/publication/2017-07_exploiting-budan-fourier-and-vincents-theorems-ray-tracing-3d-bezier-curves

[7] Gerald Farin. *Curves and Surfaces for Computer Aided Geometric Design: A Practical Guide.* Academic Press, ISBN 0-12-249050-9, p.42, 1988.

[8] Mathematics Stack Exchange – Quadratic Bezier curves representation as implicit quadratic equation
https://math.stackexchange.com/questions/438743/quadratic-bezier-curves-representation-as-implicit-quadratic-equation

[9] Wikipedia – Budan's theorem
https://en.wikipedia.org/wiki/Budan%27s_theorem

[10] Wikipedia – Vincent's theorem
https://en.wikipedia.org/wiki/Vincent%27s_theorem

[11] Wikipedia – Ridders' method
https://en.wikipedia.org/wiki/Ridders%27_method

[12] Mathematics Stack Exchange – Why does Ridders' method work as well as it does?
https://math.stackexchange.com/questions/1596654/why-does-ridders-method-work-as-well-as-it-does

[13] Jaan Kiusalaas. *Numerical Methods in Engineering with Python (2nd ed.).* Cambridge University Press, ISBN 978-0-521-19132-6, pp.147–148, 2010.

Appendix A Converts a parametric function of a curve to an implicit function (for cubic)

Listing 1: input (Wolfram Alpha)

```
Collect[Expand[Eliminate[{a0*t^3+a1*t^2+a2*t+a3==x,b0*t^3+b1*t^2+b2*t+b3==y
   },t]],{x,y}]
```

Listing 2: output

```
y (-6 a0^2 a3 b0 b3 + 3 a0^2 a3 b1 b2 - a0 a1 a3 b0 b2 - 2 a0 a1 a3 b1^2 +
   a0 a2 a3 b0 b1 + 3 a0 a3^2 b0^2 + 2 a1^2 a3 b0 b1 - 3 a1 a2 a3 b0^2) + 3
    a0^2 a3 b0 b3^2 + 3 a0^2 a3 b0 y^2 - 3 a0^2 a3 b1 b2 b3 + a0^2 a3 b2^3
   + x (6 a0 a3 b0^2 b3 - 6 a0 a3 b0^2 y - 6 a0 a3 b0 b1 b2 + 2 a0 a3 b1^3
   + 4 a1 a3 b0^2 b2 - 2 a1 a3 b0 b1^2 + 2 a2 a3 b0^2 b1 - 3 a3^2 b0^3) +
   a0 a1 a3 b0 b2 b3 + 2 a0 a1 a3 b1^2 b3 - a0 a1 a3 b1 b2^2 - a0 a2 a3 b0
   b1 b3 - 2 a0 a2 a3 b0 b2^2 + a0 a2 a3 b1^2 b2 - 3 a0 a3^2 b0^2 b3 + 3 a0
    a3^2 b0 b1 b2 - a0 a3^2 b1^3 - 2 a1^2 a3 b0 b1 b3 + a1^2 a3 b0 b2^2 + 3
    a1 a2 a3 b0^2 b3 - a1 a2 a3 b0 b1 b2 - 2 a1 a3^2 b0^2 b2 + a1 a3^2 b0
   b1^2 + a2^2 a3 b0^2 b2 - a2 a3^2 b0^2 b1 + a3^3 b0^3 + 3 a3 b0^3 x^2 =
   a0^3 b3^3 - a0^3 y^3 + x (y (-6 a0^2 b0 b3 + 3 a0^2 b1 b2 - a0 a1 b0 b2
   - 2 a0 a1 b1^2 + a0 a2 b0 b1 + 2 a1^2 b0 b1 - 3 a1 a2 b0^2) + 3 a0^2 b0
   b3^2 + 3 a0^2 b0 y^2 - 3 a0^2 b1 b2 b3 + a0^2 b2^3 + a0 a1 b0 b2 b3 + 2
   a0 a1 b1^2 b3 - a0 a1 b1 b2^2 - a0 a2 b0 b1 b3 - 2 a0 a2 b0 b2^2 + a0 a2
    b1^2 b2 - 2 a1^2 b0 b1 b3 + a1^2 b0 b2^2 + 3 a1 a2 b0^2 b3 - a1 a2 b0
   b1 b2 + a2^2 b0^2 b2) - a0^2 a1 b2 b3^2 - 2 a0^2 a2 b1 b3^2 + a0^2 a2 b2
   ^2 b3 + y (-3 a0^3 b3^2 + 2 a0^2 a1 b2 b3 + 4 a0^2 a2 b1 b3 - a0^2 a2 b2
   ^2 - 2 a0 a1^2 b1 b3 - 6 a0 a1 a2 b0 b3 + a0 a1 a2 b1 b2 + 2 a0 a2^2 b0
   b2 - a0 a2^2 b1^2 + 2 a1^3 b0 b3 - a1^2 a2 b0 b2 + a1 a2^2 b0 b1 - a2^3
   b0^2) + y^2 (3 a0^3 b3 - a0^2 a1 b2 - 2 a0^2 a2 b1 + a0 a1^2 b1 + 3 a0
   a1 a2 b0 - a1^3 b0) + a0 a1^2 b1 b3^2 + x^2 (3 a0 b0^2 b3 - 3 a0 b0^2 y
   - 3 a0 b0 b1 b2 + a0 b1^3 + 2 a1 b0^2 b2 - a1 b0 b1^2 + a2 b0^2 b1) + 3
   a0 a1 a2 b0 b3^2 - a0 a1 a2 b1 b2 b3 - 2 a0 a2^2 b0 b2 b3 + a0 a2^2 b1^2
    b3 + a1^3 (-b0) b3^2 + a1^2 a2 b0 b2 b3 - a1 a2^2 b0 b1 b3 + a2^3 b0^2
   b3 + b0^3 x^3
```

Manually rearrange into the form of implicit function $F(x, y) = 0$.

Listing 3: verification (Sage Math)

```
var('a0 a1 a2 a3')
var('b0 b1 b2 b3')
x=a0*t^3+a1*t^2+a2*t+a3
y=b0*t^3+b1*t^2+b2*t+b3
F3=a3^3*b0^3-a2*a3^2*b0^2*b1+a1*a3^2*b0*b1^2-a0*a3^2*b1^3+a2^2*a3*b0^2*b2-2*
   a1*a3^2*b0^2*b2-a1*a2*a3*b0*b1*b2+3*a0*a3^2*b0*b1*b2+a0*a2*a3*b1^2*b2+a1
   ^2*a3*b0*b2^2-2*a0*a2*a3*b0*b2^2-a0*a1*a3*b1*b2^2+a0^2*a3*b2^3-a2^3*b0
   ^2*b3+3*a1*a2*a3*b0^2*b3-3*a0*a3^2*b0^2*b3+a1*a2^2*b0*b1*b3-2*a1^2*a3*b0
   *b1*b3-a0*a2*a3*b0*b1*b3-a0*a2^2*b1^2*b3+2*a0*a1*a3*b1^2*b3-a1^2*a2*b0*
   b2*b3+2*a0*a2^2*b0*b2*b3+a0*a1*a3*b0*b2*b3+a0*a1*a2*b1*b2*b3-3*a0^2*a3*
   b1*b2*b3-a0^2*a2*b2^2*b3+a1^3*b0*b3^2-3*a0*a1*a2*b0*b3^2+3*a0^2*a3*b0*b3
   ^2-a0*a1^2*b1*b3^2+2*a0^2*a2*b1*b3^2+a0^2*a1*b2*b3^2-a0^3*b3^3+(-3*a3^2*
   b0^3+2*a2*a3*b0^2*b1-2*a1*a3*b0*b1^2+2*a0*a3*b1^3-a2^2*b0^2*b2+4*a1*a3*
```

```
   b0^2*b2+a1*a2*b0*b1*b2-6*a0*a3*b0*b1*b2-a0*a2*b1^2*b2-a1^2*b0*b2^2+2*a0*
   a2*b0*b2^2+a0*a1*b1*b2^2-a0^2*b2^3-3*a1*a2*b0^2*b3+6*a0*a3*b0^2*b3+2*a1
   ^2*b0*b1*b3+a0*a2*b0*b1*b3-2*a0*a1*b1^2*b3-a0*a1*b0*b2*b3+3*a0^2*b1*b2*
   b3-3*a0^2*b0*b3^2)*x+(a2^3*b0^2-3*a1*a2*a3*b0^2+3*a0*a3^2*b0^2-a1*a2^2*
   b0*b1+2*a1^2*a3*b0*b1+a0*a2*a3*b0*b1+a0*a2^2*b1^2-2*a0*a1*a3*b1^2+a1^2*
   a2*b0*b2-2*a0*a2^2*b0*b2-a0*a1*a3*b0*b2-a0*a1*a2*b1*b2+3*a0^2*a3*b1*b2+
   a0^2*a2*b2^2-2*a1^3*b0*b3+6*a0*a1*a2*b0*b3-6*a0^2*a3*b0*b3+2*a0*a1^2*b1*
   b3-4*a0^2*a2*b1*b3-2*a0^2*a1*b2*b3+3*a0^3*b3^2)*y+(3*a3*b0^3-a2*b0^2*b1+
   a1*b0*b1^2-a0*b1^3-2*a1*b0^2*b2+3*a0*b0*b1*b2-3*a0*b0^2*b3)*x^2+(3*a1*a2
   *b0^2-6*a0*a3*b0^2-2*a1^2*b0*b1-a0*a2*b0*b1+2*a0*a1*b1^2+a0*a1*b0*b2-3*
   a0^2*b1*b2+6*a0^2*b0*b3)*x*y+(a1^3*b0-3*a0*a1*a2*b0+3*a0^2*a3*b0-a0*a1
   ^2*b1+2*a0^2*a2*b1+a0^2*a1*b2-3*a0^3*b3)*y^2-b0^3*x^3+3*a0*b0^2*x^2*y-3*
   a0^2*b0*x*y^2+a0^3*y^3
F3.simplify_full()
```

Listing 4: output

```
0
```

Appendix B Find the coefficients of each term in the implicit equation

Listing 5: for cubic (Sage Math)

```
var('u a0 a1 a2 a3 b0 b1 b2 b3 c0 c1 c2 c3 d0 d1 d2 d3')
x=c0*u^3+c1*u^2+c2*u+c3
y=d0*u^3+d1*u^2+d2*u+d3
F0=a3^3*b0^3-a2*a3^2*b0^2*b1+a1*a3^2*b0*b1^2-a0*a3^2*b1^3+a2^2*a3*b0^2*b2-2*
   a1*a3^2*b0^2*b2-a1*a2*a3*b0*b1*b2+3*a0*a3^2*b0*b1*b2+a0*a2*a3*b1^2*b2+a1
   ^2*a3*b0*b2^2-2*a0*a2*a3*b0*b2^2-a0*a1*a3*b1*b2^2+a0^2*a3*b2^3-a2^3*b0
   ^2*b3+3*a1*a2*a3*b0^2*b3-3*a0*a3^2*b0^2*b3+a1*a2^2*b0*b1*b3-2*a1^2*a3*b0
   *b1*b3-a0*a2*a3*b0*b1*b3-a0*a2^2*b1^2*b3+2*a0*a1*a3*b1^2*b3-a1^2*a2*b0*
   b2*b3+2*a0*a2^2*b0*b2*b3+a0*a1*a3*b0*b2*b3+a0*a1*a2*b1*b2*b3-3*a0^2*a3*
   b1*b2*b3-a0^2*a2*b2^2*b3+a1^3*b0*b3^2-3*a0*a1*a2*b0*b3^2+3*a0^2*a3*b0*b3
   ^2-a0*a1^2*b1*b3^2+2*a0^2*a2*b1*b3^2+a0^2*a1*b2*b3^2-a0^3*b3^3+(-3*a3^2*
   b0^3+2*a2*a3*b0^2*b1-2*a1*a3*b0*b1^2+2*a0*a3*b1^3-a2^2*b0^2*b2+4*a1*a3*
   b0^2*b2+a1*a2*b0*b1*b2-6*a0*a3*b0*b1*b2-a0*a2*b1^2*b2-a1^2*b0*b2^2+2*a0*
   a2*b0*b2^2+a0*a1*b1*b2^2-a0^2*b2^3-3*a1*a2*b0^2*b3+6*a0*a3*b0^2*b3+2*a1
   ^2*b0*b1*b3+a0*a2*b0*b1*b3-2*a0*a1*b1^2*b3-a0*a1*b0*b2*b3+3*a0^2*b1*b2*
   b3-3*a0^2*b0*b3^2)*x+(a2^3*b0^2-3*a1*a2*a3*b0^2+3*a0*a3^2*b0^2-a1*a2^2*
   b0*b1+2*a1^2*a3*b0*b1+a0*a2*a3*b0*b1+a0*a2^2*b1^2-2*a0*a1*a3*b1^2+a1^2*
   a2*b0*b2-2*a0*a2^2*b0*b2-a0*a1*a3*b0*b2-a0*a1*a2*b1*b2+3*a0^2*a3*b1*b2+
   a0^2*a2*b2^2-2*a1^3*b0*b3+6*a0*a1*a2*b0*b3-6*a0^2*a3*b0*b3+2*a0*a1^2*b1*
   b3-4*a0^2*a2*b1*b3-2*a0^2*a1*b2*b3+3*a0^3*b3^2)*y+(3*a3*b0^3-a2*b0^2*b1+
   a1*b0*b1^2-a0*b1^3-2*a1*b0^2*b2+3*a0*b0*b1*b2-3*a0*b0^2*b3)*x^2+(3*a1*a2
   *b0^2-6*a0*a3*b0^2-2*a1^2*b0*b1-a0*a2*b0*b1+2*a0*a1*b1^2+a0*a1*b0*b2-3*
   a0^2*b1*b2+6*a0^2*b0*b3)*x*y+(a1^3*b0-3*a0*a1*a2*b0+3*a0^2*a3*b0-a0*a1
   ^2*b1+2*a0^2*a2*b1+a0^2*a1*b2-3*a0^3*b3)*y^2-b0^3*x^3+3*a0*b0^2*x^2*y-3*
   a0^2*b0*x*y^2+a0^3*y^3
cf0=F0.coefficients(u,sparse=False)
```

```
cf0[9].simplify_full()
cf0[8].simplify_full()
cf0[7].simplify_full()
cf0[6].simplify_full()
cf0[5].simplify_full()
cf0[4].simplify_full()
cf0[3].simplify_full()
cf0[2].simplify_full()
cf0[1].simplify_full()
cf0[0].simplify_full()
```

Listing 6: output

```
-b0^3*c0^3 + 3*a0*b0^2*c0^2*d0 - 3*a0^2*b0*c0*d0^2 + a0^3*d0^3
-3*b0^3*c0^2*c1 + 6*a0*b0^2*c0*c1*d0 - 3*a0^2*b0*c1*d0^2 + 3*(a0*b0^2*c0^2 -
    2*a0^2*b0*c0*d0 + a0^3*d0^2)*d1
-3*b0^3*c0*c1^2 - 3*b0^3*c0^2*c2 - 3*a0^2*b0*c2*d0^2 - 3*(a0^2*b0*c0 - a0^3*
   d0)*d1^2 + 3*(a0*b0^2*c1^2 + 2*a0*b0^2*c0*c2)*d0 + 6*(a0*b0^2*c0*c1 - a0
   ^2*b0*c1*d0)*d1 + 3*(a0*b0^2*c0^2 - 2*a0^2*b0*c0*d0 + a0^3*d0^2)*d2
-b0^3*c1^3 - 6*b0^3*c0*c1*c2 - 3*b0^3*c0^2*c3 - 3*a0^2*b0*c1*d1^2 + a0^3*d1
   ^3 + (3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0^2*b3 - (2*
   a1*b0^2 - 3*a0*b0*b1)*b2)*c0^2 + (a0^2*a1*b2 - 3*a0^3*b3 - 3*a0^2*b0*c3
   + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*d0^2 +
   (6*a0*b0^2*c1*c2 + 6*a0*b0^2*c0*c3 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(
   a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*
   b2)*c0)*d0 + 3*(a0*b0^2*c1^2 + 2*a0*b0^2*c0*c2 - 2*a0^2*b0*c2*d0)*d1 +
   6*(a0*b0^2*c0*c1 - a0^2*b0*c1*d0 - (a0^2*b0*c0 - a0^3*d0)*d1)*d2 + 3*(a0
   *b0^2*c0^2 - 2*a0^2*b0*c0*d0 + a0^3*d0^2)*d3
-3*b0^3*c1^2*c2 - 3*b0^3*c0*c2^2 - 6*b0^3*c0*c1*c3 - 3*a0^2*b0*c2*d1^2 +
   2*(3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0^2*b3 - (2*a1*
   b0^2 - 3*a0*b0*b1)*b2)*c0*c1 - 3*(a0^2*b0*c0 - a0^3*d0)*d2^2 + (3*a0*b0
   ^2*c2^2 + 6*a0*b0^2*c1*c3 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*
   a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c1)*d0
    + (6*a0*b0^2*c1*c2 + 6*a0*b0^2*c0*c3 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 +
   3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*
   b1)*b2)*c0 + 2*(a0^2*a1*b2 - 3*a0^3*b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*
   a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*d0)*d1 + 3*(a0*b0^2*c1^2
   + 2*a0*b0^2*c0*c2 - 2*a0^2*b0*c2*d0 - 2*a0^2*b0*c1*d1 + a0^3*d1^2)*d2 +
   6*(a0*b0^2*c0*c1 - a0^2*b0*c1*d0 - (a0^2*b0*c0 - a0^3*d0)*d1)*d3
-3*b0^3*c1*c2^2 + (3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0
   ^2*b3 - (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c1^2 + 2*(3*a3*b0^3 - a2*b0^2*b1 +
   a1*b0*b1^2 - a0*b1^3 - 3*a0*b0^2*b3 - (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c0*c2
    + (a0^2*a1*b2 - 3*a0^3*b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*
   a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*d1^2 - 3*(a0^2*b0*c1 - a0^3*d1)*d2^2
   - 3*(b0^3*c1^2 + 2*b0^3*c0*c2)*c3 + (6*a0*b0^2*c2*c3 + (2*a0*a1*b1^2 +
   6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*
   a1*b0 - 3*a0^2*b1)*b2)*c2)*d0 + (3*a0*b0^2*c2^2 + 6*a0*b0^2*c1*c3 + (2*
   a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*
   b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c1)*d1 + (6*a0*b0^2*c1*c2 + 6*a0*b0
   ^2*c0*c3 - 6*a0^2*b0*c2*d1 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 -
   2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c0 +
    2*(a0^2*a1*b2 - 3*a0^3*b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*
```

```
   a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*d0)*d2 + 3*(a0*b0^2*c1^2 + 2*a0*b0^2*
   c0*c2 - 2*a0^2*b0*c2*d0 - 2*a0^2*b0*c1*d1 + a0^3*d1^2 - 2*(a0^2*b0*c0 -
   a0^3*d0)*d2)*d3
-b0^3*c2^3 - 3*b0^3*c0*c3^2 - 3*a0^2*b0*c2*d2^2 + a0^3*d2^3 + 2*(3*a3*b0^3 -
    a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0^2*b3 - (2*a1*b0^2 - 3*a0*b0
   *b1)*b2)*c1*c2 - 3*(a0^2*b0*c0 - a0^3*d0)*d3^2 - (3*a3^2*b0^3 - 2*a2*a3*
   b0^2*b1 + 2*a1*a3*b0*b1^2 - 2*a0*a3*b1^3 + a0^2*b2^3 + 3*a0^2*b0*b3^2 -
   (a0*a1*b1 - (a1^2 - 2*a0*a2)*b0)*b2^2 + (a0*a2*b1^2 + (a2^2 - 4*a1*a3)*
   b0^2 - (a1*a2 - 6*a0*a3)*b0*b1)*b2 + (2*a0*a1*b1^2 + 3*(a1*a2 - 2*a0*a3)
   *b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*b3)*c0 -
   2*(3*b0^3*c1*c2 - (3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*
   b0^2*b3 - (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c0)*c3 + (a0^2*a2*b2^2 + 3*a0^3*
   b3^2 + 3*a0*b0^2*c3^2 + (a2^3 - 3*a1*a2*a3 + 3*a0*a3^2)*b0^2 - (a1*a2^2
   - (2*a1^2 + a0*a2)*a3)*b0*b1 + (a0*a2^2 - 2*a0*a1*a3)*b1^2 + ((a1^2*a2 -
    2*a0*a2^2 - a0*a1*a3)*b0 - (a0*a1*a2 - 3*a0^2*a3)*b1)*b2 - 2*(a0^2*a1*
   b2 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*b3 +
    (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*
   a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c3)*d0 + (6*a0*b0^2*c2*c3 + (2*a0
   *a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0
   *b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c2)*d1 + (3*a0*b0^2*c2^2 + 6*a0*b0^2*c1
   *c3 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2
   + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c1 + 2*(a0^2*a1*b2 - 3*a0^3*
   b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0
   ^2*a2)*b1)*d1)*d2 + (6*a0*b0^2*c1*c2 + 6*a0*b0^2*c0*c3 - 6*a0^2*b0*c2*d1
    + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 +
   a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c0 + 2*(a0^2*a1*b2 - 3*a0^3*b3
    - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0
   ^2*a2)*b1)*d0 - 6*(a0^2*b0*c1 - a0^3*d1)*d2)*d3
-3*b0^3*c1*c3^2 + (3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0
   ^2*b3 - (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c2^2 + (a0^2*a1*b2 - 3*a0^3*b3 - 3*
   a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*
   b1)*d2^2 - 3*(a0^2*b0*c1 - a0^3*d1)*d3^2 - (3*a3^2*b0^3 - 2*a2*a3*b0^2*
   b1 + 2*a1*a3*b0*b1^2 - 2*a0*a3*b1^3 + a0^2*b2^3 + 3*a0^2*b0*b3^2 - (a0*
   a1*b1 - (a1^2 - 2*a0*a2)*b0)*b2^2 + (a0*a2*b1^2 + (a2^2 - 4*a1*a3)*b0^2
   - (a1*a2 - 6*a0*a3)*b0*b1)*b2 + (2*a0*a1*b1^2 + 3*(a1*a2 - 2*a0*a3)*b0^2
    - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*b3)*c1 - (3*b0^3*
   c2^2 - 2*(3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0^2*b3 -
    (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c1)*c3 + (a0^2*a2*b2^2 + 3*a0^3*b3^2 + 3*
   a0*b0^2*c3^2 + (a2^3 - 3*a1*a2*a3 + 3*a0*a3^2)*b0^2 - (a1*a2^2 - (2*a1^2
    + a0*a2)*a3)*b0*b1 + (a0*a2^2 - 2*a0*a1*a3)*b1^2 + ((a1^2*a2 - 2*a0*a2
   ^2 - a0*a1*a3)*b0 - (a0*a1*a2 - 3*a0^2*a3)*b1)*b2 - 2*(a0^2*a1*b2 + (a1
   ^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*b3 + (2*a0*
   a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*
   b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c3)*d1 + (6*a0*b0^2*c2*c3 + (2*a0*a1*b1
   ^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 +
   (a0*a1*b0 - 3*a0^2*b1)*b2)*c2)*d2 + (3*a0*b0^2*c2^2 + 6*a0*b0^2*c1*c3 -
   6*a0^2*b0*c2*d2 + 3*a0^3*d2^2 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3 + 3*(a1*a2
   - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0^2*b1)*b2)*c1
    + 2*(a0^2*a1*b2 - 3*a0^3*b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0
   ^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*d1)*d3
-3*b0^3*c2*c3^2 + 2*(3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 - 3*a0*b0
```

```
    ^2*b3 - (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c2*c3 - 3*(a0^2*b0*c2 - a0^3*d2)*d3
    ^2 - (3*a3^2*b0^3 - 2*a2*a3*b0^2*b1 + 2*a1*a3*b0*b1^2 - 2*a0*a3*b1^3 +
    a0^2*b2^3 + 3*a0^2*b0*b3^2 - (a0*a1*b1 - (a1^2 - 2*a0*a2)*b0)*b2^2 + (a0
    *a2*b1^2 + (a2^2 - 4*a1*a3)*b0^2 - (a1*a2 - 6*a0*a3)*b0*b1)*b2 + (2*a0*
    a1*b1^2 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0
    - 3*a0^2*b1)*b2)*b3)*c2 + (a0^2*a2*b2^2 + 3*a0^3*b3^2 + 3*a0*b0^2*c3^2 +
     (a2^3 - 3*a1*a2*a3 + 3*a0*a3^2)*b0^2 - (a1*a2^2 - (2*a1^2 + a0*a2)*a3)*
    b0*b1 + (a0*a2^2 - 2*a0*a1*a3)*b1^2 + ((a1^2*a2 - 2*a0*a2^2 - a0*a1*a3)*
    b0 - (a0*a1*a2 - 3*a0^2*a3)*b1)*b2 - 2*(a0^2*a1*b2 + (a1^3 - 3*a0*a1*a2
    + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*b3 + (2*a0*a1*b1^2 + 6*a0^2*
    b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 -
    3*a0^2*b1)*b2)*c3)*d2 + (6*a0*b0^2*c2*c3 + (2*a0*a1*b1^2 + 6*a0^2*b0*b3
    + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (a0*a1*b0 - 3*a0
    ^2*b1)*b2)*c2 + 2*(a0^2*a1*b2 - 3*a0^3*b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*
    a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*d2)*d3
a3^3*b0^3 - a2*a3^2*b0^2*b1 + a1*a3^2*b0*b1^2 - a0*a3^2*b1^3 + a0^2*a3*b2^3
    - a0^3*b3^3 - b0^3*c3^3 + a0^3*d3^3 - (a0*a1*a3*b1 - (a1^2 - 2*a0*a2)*a3
    *b0)*b2^2 + (a0^2*a1*b2 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2
    - 2*a0^2*a2)*b1)*b3^2 + (3*a3*b0^3 - a2*b0^2*b1 + a1*b0*b1^2 - a0*b1^3 -
     3*a0*b0^2*b3 - (2*a1*b0^2 - 3*a0*b0*b1)*b2)*c3^2 + (a0^2*a1*b2 - 3*a0
    ^3*b3 - 3*a0^2*b0*c3 + (a1^3 - 3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 -
    2*a0^2*a2)*b1)*d3^2 + (a0*a2*a3*b1^2 + (a2^2*a3 - 2*a1*a3^2)*b0^2 - (a1*
    a2*a3 - 3*a0*a3^2)*b0*b1)*b2 - (a0^2*a2*b2^2 + (a2^3 - 3*a1*a2*a3 + 3*a0
    *a3^2)*b0^2 - (a1*a2^2 - (2*a1^2 + a0*a2)*a3)*b0*b1 + (a0*a2^2 - 2*a0*a1
    *a3)*b1^2 + ((a1^2*a2 - 2*a0*a2^2 - a0*a1*a3)*b0 - (a0*a1*a2 - 3*a0^2*a3
    )*b1)*b2)*b3 - (3*a3^2*b0^3 - 2*a2*a3*b0^2*b1 + 2*a1*a3*b0*b1^2 - 2*a0*
    a3*b1^3 + a0^2*b2^3 + 3*a0^2*b0*b3^2 - (a0*a1*b1 - (a1^2 - 2*a0*a2)*b0)*
    b2^2 + (a0*a2*b1^2 + (a2^2 - 4*a1*a3)*b0^2 - (a1*a2 - 6*a0*a3)*b0*b1)*b2
     + (2*a0*a1*b1^2 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 + (
    a0*a1*b0 - 3*a0^2*b1)*b2)*b3)*c3 + (a0^2*a2*b2^2 + 3*a0^3*b3^2 + 3*a0*b0
    ^2*c3^2 + (a2^3 - 3*a1*a2*a3 + 3*a0*a3^2)*b0^2 - (a1*a2^2 - (2*a1^2 + a0
    *a2)*a3)*b0*b1 + (a0*a2^2 - 2*a0*a1*a3)*b1^2 + ((a1^2*a2 - 2*a0*a2^2 -
    a0*a1*a3)*b0 - (a0*a1*a2 - 3*a0^2*a3)*b1)*b2 - 2*(a0^2*a1*b2 + (a1^3 -
    3*a0*a1*a2 + 3*a0^2*a3)*b0 - (a0*a1^2 - 2*a0^2*a2)*b1)*b3 + (2*a0*a1*b1
    ^2 + 6*a0^2*b0*b3 + 3*(a1*a2 - 2*a0*a3)*b0^2 - (2*a1^2 + a0*a2)*b0*b1 +
    (a0*a1*b0 - 3*a0^2*b1)*b2)*c3)*d3
```

Listing 7: for quadratic (Sage Math)

```
var('u a1 a2 a3 b1 b2 b3 c0 c1 c2 c3 d0 d1 d2 d3')
x=c0*u^3+c1*u^2+c2*u+c3
y=d0*u^3+d1*u^2+d2*u+d3
F0=a3^2*b1^2-a2*a3*b1*b2+a1*a3*b2^2+a2^2*b1*b3-2*a1*a3*b1*b3-a1*a2*b2*b3+a1
    ^2*b3^2+(-2*a3*b1^2+a2*b1*b2-a1*b2^2+2*a1*b1*b3)*x+(-a2^2*b1+2*a1*a3*b1+
    a1*a2*b2-2*a1^2*b3)*y+b1^2*x^2-2*a1*b1*x*y+a1^2*y^2
cf0=F0.coefficients(u,sparse=False)
cf0[6].simplify_full()
cf0[5].simplify_full()
cf0[4].simplify_full()
cf0[3].simplify_full()
cf0[2].simplify_full()
cf0[1].simplify_full()
```

```
cf0[0].simplify_full()
```

Listing 8: output

```
b1^2*c0^2 - 2*a1*b1*c0*d0 + a1^2*d0^2
2*b1^2*c0*c1 - 2*a1*b1*c1*d0 - 2*(a1*b1*c0 - a1^2*d0)*d1
b1^2*c1^2 + 2*b1^2*c0*c2 - 2*a1*b1*c2*d0 - 2*a1*b1*c1*d1 + a1^2*d1^2 - 2*(a1
    *b1*c0 - a1^2*d0)*d2
2*b1^2*c1*c2 + 2*b1^2*c0*c3 - 2*a1*b1*c2*d1 - (2*a3*b1^2 - a2*b1*b2 + a1*b2
    ^2 - 2*a1*b1*b3)*c0 + (a1*a2*b2 - 2*a1^2*b3 - 2*a1*b1*c3 - (a2^2 - 2*a1*
    a3)*b1)*d0 - 2*(a1*b1*c1 - a1^2*d1)*d2 - 2*(a1*b1*c0 - a1^2*d0)*d3
b1^2*c2^2 + 2*b1^2*c1*c3 - 2*a1*b1*c2*d2 + a1^2*d2^2 - (2*a3*b1^2 - a2*b1*b2
     + a1*b2^2 - 2*a1*b1*b3)*c1 + (a1*a2*b2 - 2*a1^2*b3 - 2*a1*b1*c3 - (a2^2
     - 2*a1*a3)*b1)*d1 - 2*(a1*b1*c1 - a1^2*d1)*d3
2*b1^2*c2*c3 - (2*a3*b1^2 - a2*b1*b2 + a1*b2^2 - 2*a1*b1*b3)*c2 + (a1*a2*b2
    - 2*a1^2*b3 - 2*a1*b1*c3 - (a2^2 - 2*a1*a3)*b1)*d2 - 2*(a1*b1*c2 - a1^2*
    d2)*d3
a3^2*b1^2 - a2*a3*b1*b2 + a1*a3*b2^2 + a1^2*b3^2 + b1^2*c3^2 + a1^2*d3^2 - (
    a1*a2*b2 - (a2^2 - 2*a1*a3)*b1)*b3 - (2*a3*b1^2 - a2*b1*b2 + a1*b2^2 -
    2*a1*b1*b3)*c3 + (a1*a2*b2 - 2*a1^2*b3 - 2*a1*b1*c3 - (a2^2 - 2*a1*a3)*
    b1)*d3
```

Listing 9: for linear (Sage Math)

```
var('u a2 a3 b2 b3 c0 c1 c2 c3 d0 d1 d2 d3')
x=c0*u^3+c1*u^2+c2*u+c3
y=d0*u^3+d1*u^2+d2*u+d3
F0=-a3*b2+a2*b3+b2*x-a2*y
cf0=F0.coefficients(u,sparse=False)
cf0[3].simplify_full()
cf0[2].simplify_full()
cf0[1].simplify_full()
cf0[0].simplify_full()
```

Listing 10: output

```
b2*c0 - a2*d0
b2*c1 - a2*d1
b2*c2 - a2*d2
-a3*b2 + a2*b3 + b2*c3 - a2*d3
```

Appendix C Find the formula for the coefficients of the transformed polynomial of Budin's statement

transformed polynomial: $p(x+1)$

Listing 11: input (Sage Math)

```
var('x e0 e1 e2 e3 e4 e5 e6 e7 e8 e9')
x1=x+1
```

```
p=e0*x1^9+e1*x1^8+e2*x1^7+e3*x1^6+e4*x1^5+e5*x1^4+e6*x1^3+e7*x1^2+e8*x1+e9
p=p.simplify_full()
p.coefficients(x)
```

Listing 12: output

```
[[e0 + e1 + e2 + e3 + e4 + e5 + e6 + e7 + e8 + e9, 0],
 [9*e0 + 8*e1 + 7*e2 + 6*e3 + 5*e4 + 4*e5 + 3*e6 + 2*e7 + e8, 1],
 [36*e0 + 28*e1 + 21*e2 + 15*e3 + 10*e4 + 6*e5 + 3*e6 + e7, 2],
 [84*e0 + 56*e1 + 35*e2 + 20*e3 + 10*e4 + 4*e5 + e6, 3],
 [126*e0 + 70*e1 + 35*e2 + 15*e3 + 5*e4 + e5, 4],
 [126*e0 + 56*e1 + 21*e2 + 6*e3 + e4, 5],
 [84*e0 + 28*e1 + 7*e2 + e3, 6],
 [36*e0 + 8*e1 + e2, 7],
 [9*e0 + e1, 8],
 [e0, 9]]
```

Appendix D Find the formula for finding the coefficients of the transformed polynomial for the VAG method

transformed polynomial: $(x+1)^9 p\left(\frac{a+bx}{1+x}\right)$

Listing 13: input (Sage Math)

```
var('x a b e0 e1 e2 e3 e4 e5 e6 e7 e8 e9')
x1=(a+b*x)/(1+x)
p=e0*x1^9+e1*x1^8+e2*x1^7+e3*x1^6+e4*x1^5+e5*x1^4+e6*x1^3+e7*x1^2+e8*x1+e9
f=(x+1)^9*p
f=f.simplify_full()
f.coefficients(x)
```

Listing 14: output

```
[[a^9*e0 + a^8*e1 + a^7*e2 + a^6*e3 + a^5*e4 + a^4*e5 + a^3*e6 + a^2*e7 + a*
   e8 + e9,
  0],
 [a^8*e1 + 2*a^7*e2 + 3*a^6*e3 + 4*a^5*e4 + 5*a^4*e5 + 6*a^3*e6 + 7*a^2*e7 +
    8*a*e8 + (9*a^8*e0 + 8*a^7*e1 + 7*a^6*e2 + 6*a^5*e3 + 5*a^4*e4 + 4*a^3*
   e5 + 3*a^2*e6 + 2*a*e7 + e8)*b + 9*e9,
  1],
 [a^7*e2 + 3*a^6*e3 + 6*a^5*e4 + 10*a^4*e5 + 15*a^3*e6 + 21*a^2*e7 + (36*a
   ^7*e0 + 28*a^6*e1 + 21*a^5*e2 + 15*a^4*e3 + 10*a^3*e4 + 6*a^2*e5 + 3*a*
   e6 + e7)*b^2 + 28*a*e8 + 2*(4*a^7*e1 + 7*a^6*e2 + 9*a^5*e3 + 10*a^4*e4 +
    10*a^3*e5 + 9*a^2*e6 + 7*a*e7 + 4*e8)*b + 36*e9,
  2],
 [a^6*e3 + 4*a^5*e4 + 10*a^4*e5 + 20*a^3*e6 + (84*a^6*e0 + 56*a^5*e1 + 35*a
   ^4*e2 + 20*a^3*e3 + 10*a^2*e4 + 4*a*e5 + e6)*b^3 + 35*a^2*e7 + (28*a^6*
   e1 + 42*a^5*e2 + 45*a^4*e3 + 40*a^3*e4 + 30*a^2*e5 + 18*a*e6 + 7*e7)*b^2
    + 56*a*e8 + (7*a^6*e2 + 18*a^5*e3 + 30*a^4*e4 + 40*a^3*e5 + 45*a^2*e6 +
    42*a*e7 + 28*e8)*b + 84*e9,
```

```
 3],
[a^5*e4 + 5*a^4*e5 + (126*a^5*e0 + 70*a^4*e1 + 35*a^3*e2 + 15*a^2*e3 + 5*a*
    e4 + e5)*b^4 + 15*a^3*e6 + 2*(28*a^5*e1 + 35*a^4*e2 + 30*a^3*e3 + 20*a
    ^2*e4 + 10*a*e5 + 3*e6)*b^3 + 35*a^2*e7 + 3*(7*a^5*e2 + 15*a^4*e3 + 20*a
    ^3*e4 + 20*a^2*e5 + 15*a*e6 + 7*e7)*b^2 + 70*a*e8 + 2*(3*a^5*e3 + 10*a
    ^4*e4 + 20*a^3*e5 + 30*a^2*e6 + 35*a*e7 + 28*e8)*b + 126*e9,
 4],
[(126*a^4*e0 + 56*a^3*e1 + 21*a^2*e2 + 6*a*e3 + e4)*b^5 + a^4*e5 + 5*(14*a
    ^4*e1 + 14*a^3*e2 + 9*a^2*e3 + 4*a*e4 + e5)*b^4 + 6*a^3*e6 + 5*(7*a^4*e2
     + 12*a^3*e3 + 12*a^2*e4 + 8*a*e5 + 3*e6)*b^3 + 21*a^2*e7 + 5*(3*a^4*e3
    + 8*a^3*e4 + 12*a^2*e5 + 12*a*e6 + 7*e7)*b^2 + 56*a*e8 + 5*(a^4*e4 + 4*a
    ^3*e5 + 9*a^2*e6 + 14*a*e7 + 14*e8)*b + 126*e9,
 5],
[(84*a^3*e0 + 28*a^2*e1 + 7*a*e2 + e3)*b^6 + 2*(28*a^3*e1 + 21*a^2*e2 + 9*a
    *e3 + 2*e4)*b^5 + 5*(7*a^3*e2 + 9*a^2*e3 + 6*a*e4 + 2*e5)*b^4 + a^3*e6 +
     20*(a^3*e3 + 2*a^2*e4 + 2*a*e5 + e6)*b^3 + 7*a^2*e7 + 5*(2*a^3*e4 + 6*a
    ^2*e5 + 9*a*e6 + 7*e7)*b^2 + 28*a*e8 + 2*(2*a^3*e5 + 9*a^2*e6 + 21*a*e7
    + 28*e8)*b + 84*e9,
 6],
[(36*a^2*e0 + 8*a*e1 + e2)*b^7 + (28*a^2*e1 + 14*a*e2 + 3*e3)*b^6 + 3*(7*a
    ^2*e2 + 6*a*e3 + 2*e4)*b^5 + 5*(3*a^2*e3 + 4*a*e4 + 2*e5)*b^4 + 5*(2*a
    ^2*e4 + 4*a*e5 + 3*e6)*b^3 + a^2*e7 + 3*(2*a^2*e5 + 6*a*e6 + 7*e7)*b^2 +
     8*a*e8 + (3*a^2*e6 + 14*a*e7 + 28*e8)*b + 36*e9,
 7],
[(9*a*e0 + e1)*b^8 + 2*(4*a*e1 + e2)*b^7 + (7*a*e2 + 3*e3)*b^6 + 2*(3*a*e3
    + 2*e4)*b^5 + 5*(a*e4 + e5)*b^4 + 2*(2*a*e5 + 3*e6)*b^3 + (3*a*e6 + 7*e7
    )*b^2 + a*e8 + 2*(a*e7 + 4*e8)*b + 9*e9,
 8],
[e0*b^9 + e1*b^8 + e2*b^7 + e3*b^6 + e4*b^5 + e5*b^4 + e6*b^3 + e7*b^2 + e8
    *b + e9,
 9]]
```

Appendix E Implementation example source code

Listing 15: bzintersection.cpp

```
#include <cmath>
#include <QPointF>
#include <QList>
using namespace std;

#define EZERO 1e-14 // Values closer to zero than this are treated as zero
#define ABADJ 1e-3 // VAG method: half of the isolated interval length for
    multiple roots
#define ABMIN 1e-5 // VAG method: minimum isolated interval length
#define DSTMIN 1e-4 // Distance threshold for matching intersection
    candidates

#define PW3(x) ((x)*(x)*(x))
#define PW2(x) ((x)*(x))
```

```
typedef struct{
   double a; // Interval start
   double b; // Interval end
   double pa; // Function value at a
   double pb; // Function value at b
   int nr; // Number of real roots contained in the interval
}Interval;

// Compute the coefficients of a cubic Bezier curve
void coefBezier(
  QList<QPointF> &cp, // IN Bezier curve control points
  double *a,   // OUT x-coordinate coefficients
  double *b)   // OUT y-coordinate coefficients
{
   a[0]=-cp[0].x()+3*cp[1].x()-3*cp[2].x()+cp[3].x();
   a[1]=3*cp[0].x()-6*cp[1].x()+3*cp[2].x();
   a[2]=-3*cp[0].x()+3*cp[1].x();
   a[3]=cp[0].x();
   b[0]=-cp[0].y()+3*cp[1].y()-3*cp[2].y()+cp[3].y();
   b[1]=3*cp[0].y()-6*cp[1].y()+3*cp[2].y();
   b[2]=-3*cp[0].y()+3*cp[1].y();
   b[3]=cp[0].y();
}

// Compute a point on a cubic parametric curve
QPointF pointBezier(
  double *a,   // IN x-coordinate coefficients
  double *b,   // IN y-coordinate coefficients
  double t)    // IN Parameter
{
   return QPointF(a[0]*PW3(t)+a[1]*PW2(t)+a[2]*t+a[3],
               b[0]*PW3(t)+b[1]*PW2(t)+b[2]*t+b[3]);
}

// Normalize control point coordinates to the range [0, 1]
void normCP(
  QList<QPointF> &cp0, // IN Control points of curve 0
  QList<QPointF> &cp1, // IN Control points of curve 1
  QList<QPointF> &ncp0, // OUT Normalized control points of curve 0
  QList<QPointF> &ncp1) // OUT Normalized control points of curve 1
{
   double xmin=cp0[0].x();
   double xmax=cp0[0].x();
   double ymin=cp0[0].y();
   double ymax=cp0[0].y();
   foreach(QPointF p,cp0.mid(1)){
      if(xmin>p.x()) xmin=p.x();
      if(xmax<p.x()) xmax=p.x();
      if(ymin>p.y()) ymin=p.y();
      if(ymax<p.y()) ymax=p.y();
```

```
   }
   foreach(QPointF p,cp1){
      if(xmin>p.x()) xmin=p.x();
      if(xmax<p.x()) xmax=p.x();
      if(ymin>p.y()) ymin=p.y();
      if(ymax<p.y()) ymax=p.y();
   }
   double m=max(xmax-xmin,ymax-ymin);
   ncp0.clear();
   ncp1.clear();
   foreach(QPointF p,cp0)
      ncp0<<QPointF((p.x()-xmin)/m,(p.y()-ymin)/m);
   foreach(QPointF p,cp1)
      ncp1<<QPointF((p.x()-xmin)/m,(p.y()-ymin)/m);
}

// Compute the coefficients of the implicit equation F0(u) for the
   intersection of two cubic parametric curves
void coefImp0(
  double *a, // IN Coefficients of curve 0 (x,y)=f(t); a[0], b[0] are cubic
     terms
  double *b,
  double *c, // IN Coefficients of curve 1 (x,y)=g(u); c[0], d[0] are cubic
     terms
  double *d,
  double *cf) // OUT Coefficients of F0(u); cf[0] is the 9th-degree term
{
   if(abs(a[0])>=EZERO||abs(b[0])>=EZERO){
      // Curve 0 is cubic
      cf[0]=-PW3(b[0])*PW3(c[0])+3*a[0]*PW2(b[0])*PW2(c[0])*d[0]-3*PW2(a[0])
         *b[0]*c[0]*PW2(d[0])+PW3(a[0])*PW3(d[0]);
      cf[1]=-3*PW3(b[0])*PW2(c[0])*c[1]+6*a[0]*PW2(b[0])*c[0]*c[1]*d[0]-3*
         PW2(a[0])*b[0]*c[1]*PW2(d[0])+3*(a[0]*PW2(b[0])*PW2(c[0])-2*PW2(a
         [0])*b[0]*c[0]*d[0]+PW3(a[0])*PW2(d[0]))*d[1];
      cf[2]=-3*PW3(b[0])*c[0]*PW2(c[1])-3*PW3(b[0])*PW2(c[0])*c[2]-3*PW2(a
         [0])*b[0]*c[2]*PW2(d[0])-3*(PW2(a[0])*b[0]*c[0]-PW3(a[0])*d[0])*PW2
         (d[1])+3*(a[0]*PW2(b[0])*PW2(c[1])+2*a[0]*PW2(b[0])*c[0]*c[2])*d
         [0]+6*(a[0]*PW2(b[0])*c[0]*c[1]-PW2(a[0])*b[0]*c[1]*d[0])*d[1]+3*(a
         [0]*PW2(b[0])*PW2(c[0])-2*PW2(a[0])*b[0]*c[0]*d[0]+PW3(a[0])*PW2(d
         [0]))*d[2];
      cf[3]=-PW3(b[0])*PW3(c[1])-6*PW3(b[0])*c[0]*c[1]*c[2]-3*PW3(b[0])*PW2(
         c[0])*c[3]-3*PW2(a[0])*b[0]*c[1]*PW2(d[1])+PW3(a[0])*PW3(d[1])+(3*a
         [3]*PW3(b[0])-a[2]*PW2(b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b
         [1])-3*a[0]*PW2(b[0])*b[3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b
         [2])*PW2(c[0])+(PW2(a[0])*a[1]*b[2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b
         [0]*c[3]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*
         PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*PW2(d[0])+(6*a[0]*PW2(b[0])*c[1]*
         c[2]+6*a[0]*PW2(b[0])*c[0]*c[3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*
         b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a
         [2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[0])*d
         [0]+3*(a[0]*PW2(b[0])*PW2(c[1])+2*a[0]*PW2(b[0])*c[0]*c[2]-2*PW2(a
```

```
    [0])*b[0]*c[2]*d[0])*d[1]+6*(a[0]*PW2(b[0])*c[0]*c[1]-PW2(a[0])*b
    [0]*c[1]*d[0]-(PW2(a[0])*b[0]*c[0]-PW3(a[0])*d[0])*d[1])*d[2]+3*(a
    [0]*PW2(b[0])*PW2(c[0])-2*PW2(a[0])*b[0]*c[0]*d[0]+PW3(a[0])*PW2(d
    [0]))*d[3];
cf[4]=-3*PW3(b[0])*PW2(c[1])*c[2]-3*PW3(b[0])*c[0]*PW2(c[2])-6*PW3(b
    [0])*c[0]*c[1]*c[3]-3*PW2(a[0])*b[0]*c[2]*PW2(d[1])+2*(3*a[3]*PW3(b
    [0])-a[2]*PW2(b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*
    PW2(b[0])*b[3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*c[0]*c
    [1]-3*(PW2(a[0])*b[0]*c[0]-PW3(a[0])*d[0])*PW2(d[2])+(3*a[0]*PW2(b
    [0])*PW2(c[2])+6*a[0]*PW2(b[0])*c[1]*c[3]+(2*a[0]*a[1]*PW2(b[1])+6*
    PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a
    [1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c
    [1])*d[0]+(6*a[0]*PW2(b[0])*c[1]*c[2]+6*a[0]*PW2(b[0])*c[0]*c
    [3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a
    [0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b
    [0]-3*PW2(a[0])*b[1])*b[2])*c[0]+2*(PW2(a[0])*a[1]*b[2]-3*PW3(a[0])
    *b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(a[0])
    *a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*d[0])*d[1]+3*(a
    [0]*PW2(b[0])*PW2(c[1])+2*a[0]*PW2(b[0])*c[0]*c[2]-2*PW2(a[0])*b
    [0]*c[2]*d[0]-2*PW2(a[0])*b[0]*c[1]*d[1]+PW3(a[0])*PW2(d[1]))*d
    [2]+6*(a[0]*PW2(b[0])*c[0]*c[1]-PW2(a[0])*b[0]*c[1]*d[0]-(PW2(a[0])
    *b[0]*c[0]-PW3(a[0])*d[0])*d[1])*d[3];
cf[5]=-3*PW3(b[0])*c[1]*PW2(c[2])+(3*a[3]*PW3(b[0])-a[2]*PW2(b[0])*b
    [1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(b[0])*b[3]-(2*a
    [1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*PW2(c[1])+2*(3*a[3]*PW3(b[0])
    -a[2]*PW2(b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(
    b[0])*b[3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*c[0]*c[2]+(PW2
    (a[0])*a[1]*b[2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])
    -3*a[0]*a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a
    [0])*a[2])*b[1])*PW2(d[1])-3*(PW2(a[0])*b[0]*c[1]-PW3(a[0])*d[1])*
    PW2(d[2])-3*(PW3(b[0])*PW2(c[1])+2*PW3(b[0])*c[0]*c[2])*c[3]+(6*a
    [0]*PW2(b[0])*c[2]*c[3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b
    [3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b
    [0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[2])*d[0]+(3*a
    [0]*PW2(b[0])*PW2(c[2])+6*a[0]*PW2(b[0])*c[1]*c[3]+(2*a[0]*a[1]*PW2
    (b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])
    -(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b
    [1])*b[2])*c[1])*d[1]+(6*a[0]*PW2(b[0])*c[1]*c[2]+6*a[0]*PW2(b[0])*
    c[0]*c[3]-6*PW2(a[0])*b[0]*c[2]*d[1]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a
    [0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a
    [0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c
    [0]+2*(PW2(a[0])*a[1]*b[2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(
    PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])
    -2*PW2(a[0])*a[2])*b[1])*d[0])*d[2]+3*(a[0]*PW2(b[0])*PW2(c[1])+2*a
    [0]*PW2(b[0])*c[0]*c[2]-2*PW2(a[0])*b[0]*c[2]*d[0]-2*PW2(a[0])*b
    [0]*c[1]*d[1]+PW3(a[0])*PW2(d[1])-2*(PW2(a[0])*b[0]*c[0]-PW3(a[0])*
    d[0])*d[2])*d[3];
cf[6]=-PW3(b[0])*PW3(c[2])-3*PW3(b[0])*c[0]*PW2(c[3])-3*PW2(a[0])*b
    [0]*c[2]*PW2(d[2])+PW3(a[0])*PW3(d[2])+2*(3*a[3]*PW3(b[0])-a[2]*PW2
    (b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(b[0])*b
    [3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*c[1]*c[2]-3*(PW2(a
```

```
    [0])*b[0]*c[0]-PW3(a[0])*d[0])*PW2(d[3])-(3*PW2(a[3])*PW3(b[0])-2*a
    [2]*a[3]*PW2(b[0])*b[1]+2*a[1]*a[3]*b[0]*PW2(b[1])-2*a[0]*a[3]*PW3(
    b[1])+PW2(a[0])*PW3(b[2])+3*PW2(a[0])*b[0]*PW2(b[3])-(a[0]*a[1]*b
    [1]-(PW2(a[1])-2*a[0]*a[2])*b[0])*PW2(b[2])+(a[0]*a[2]*PW2(b[1])+(
    PW2(a[2])-4*a[1]*a[3])*PW2(b[0])-(a[1]*a[2]-6*a[0]*a[3])*b[0]*b[1])
    *b[2]+(2*a[0]*a[1]*PW2(b[1])+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])
    -(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b
    [1])*b[2])*b[3])*c[0]-2*(3*PW3(b[0])*c[1]*c[2]-(3*a[3]*PW3(b[0])-a
    [2]*PW2(b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(b
    [0])*b[3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*c[0])*c[3]+(PW2
    (a[0])*a[2]*PW2(b[2])+3*PW3(a[0])*PW2(b[3])+3*a[0]*PW2(b[0])*PW2(c
    [3])+(PW3(a[2])-3*a[1]*a[2]*a[3]+3*a[0]*PW2(a[3]))*PW2(b[0])-(a[1]*
    PW2(a[2])-(2*PW2(a[1])+a[0]*a[2])*a[3])*b[0]*b[1]+(a[0]*PW2(a[2])
    -2*a[0]*a[1]*a[3])*PW2(b[1])+((PW2(a[1])*a[2]-2*a[0]*PW2(a[2])-a
    [0]*a[1]*a[3])*b[0]-(a[0]*a[1]*a[2]-3*PW2(a[0])*a[3])*b[1])*b
    [2]-2*(PW2(a[0])*a[1]*b[2]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(a[0])*
    a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*b[3]+(2*a[0]*a
    [1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b
    [0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*
    b[1])*b[2])*c[3])*d[0]+(6*a[0]*PW2(b[0])*c[2]*c[3]+(2*a[0]*a[1]*PW2
    (b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])
    -(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b
    [1])*b[2])*c[2])*d[1]+(3*a[0]*PW2(b[0])*PW2(c[2])+6*a[0]*PW2(b[0])*
    c[1]*c[3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a
    [2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*
    a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[1]+2*(PW2(a[0])*a[1]*b[2]-3*PW3
    (a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2
    (a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*d[1])*d
    [2]+(6*a[0]*PW2(b[0])*c[1]*c[2]+6*a[0]*PW2(b[0])*c[0]*c[3]-6*PW2(a
    [0])*b[0]*c[2]*d[1]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b
    [3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b
    [0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[0]+2*(PW2(a[0])*
    a[1]*b[2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])-3*a[0]*
    a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*
    b[1])*d[0]-6*(PW2(a[0])*b[0]*c[1]-PW3(a[0])*d[1])*d[2])*d[3];
cf[7]=-3*PW3(b[0])*c[1]*PW2(c[3])+(3*a[3]*PW3(b[0])-a[2]*PW2(b[0])*b
    [1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(b[0])*b[3]-(2*a
    [1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*PW2(c[2])+(PW2(a[0])*a[1]*b
    [2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])-3*a[0]*a[1]*a
    [2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*
    PW2(d[2])-3*(PW2(a[0])*b[0]*c[1]-PW3(a[0])*d[1])*PW2(d[3])-(3*PW2(a
    [3])*PW3(b[0])-2*a[2]*a[3]*PW2(b[0])*b[1]+2*a[1]*a[3]*b[0]*PW2(b
    [1])-2*a[0]*a[3]*PW3(b[1])+PW2(a[0])*PW3(b[2])+3*PW2(a[0])*b[0]*PW2
    (b[3])-(a[0]*a[1]*b[1]-(PW2(a[1])-2*a[0]*a[2])*b[0])*PW2(b[2])+(a
    [0]*a[2]*PW2(b[1])+(PW2(a[2])-4*a[1]*a[3])*PW2(b[0])-(a[1]*a[2]-6*a
    [0]*a[3])*b[0]*b[1])*b[2]+(2*a[0]*a[1]*PW2(b[1])+3*(a[1]*a[2]-2*a
    [0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b
    [0]-3*PW2(a[0])*b[1])*b[2])*b[3])*c[1]-(3*PW3(b[0])*PW2(c[2])-2*(3*
    a[3]*PW3(b[0])-a[2]*PW2(b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b
    [1])-3*a[0]*PW2(b[0])*b[3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b
    [2])*c[1])*c[3]+(PW2(a[0])*a[2]*PW2(b[2])+3*PW3(a[0])*PW2(b[3])+3*a
```

```
    [0]*PW2(b[0])*PW2(c[3])+(PW3(a[2])-3*a[1]*a[2]*a[3]+3*a[0]*PW2(a
    [3]))*PW2(b[0])-(a[1]*PW2(a[2])-(2*PW2(a[1])+a[0]*a[2])*a[3])*b[0]*
    b[1]+(a[0]*PW2(a[2])-2*a[0]*a[1]*a[3])*PW2(b[1])+((PW2(a[1])*a
    [2]-2*a[0]*PW2(a[2])-a[0]*a[1]*a[3])*b[0]-(a[0]*a[1]*a[2]-3*PW2(a
    [0])*a[3])*b[1])*b[2]-2*(PW2(a[0])*a[1]*b[2]+(PW3(a[1])-3*a[0]*a
    [1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b
    [1])*b[3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a
    [2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*
    a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[3])*d[1]+(6*a[0]*PW2(b[0])*c
    [2]*c[3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a
    [2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*
    a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[2])*d[2]+(3*a[0]*PW2(b[0])*PW2(
    c[2])+6*a[0]*PW2(b[0])*c[1]*c[3]-6*PW2(a[0])*b[0]*c[2]*d[2]+3*PW3(a
    [0])*PW2(d[2])+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a
    [1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(
    a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[1]+2*(PW2(a[0])*a[1]*b
    [2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])-3*a[0]*a[1]*a
    [2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*
    d[1])*d[3];
cf[8]=-3*PW3(b[0])*c[2]*PW2(c[3])+2*(3*a[3]*PW3(b[0])-a[2]*PW2(b[0])*b
    [1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(b[0])*b[3]-(2*a
    [1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*c[2]*c[3]-3*(PW2(a[0])*b[0]*c
    [2]-PW3(a[0])*d[2])*PW2(d[3])-(3*PW2(a[3])*PW3(b[0])-2*a[2]*a[3]*
    PW2(b[0])*b[1]+2*a[1]*a[3]*b[0]*PW2(b[1])-2*a[0]*a[3]*PW3(b[1])+PW2
    (a[0])*PW3(b[2])+3*PW2(a[0])*b[0]*PW2(b[3])-(a[0]*a[1]*b[1]-(PW2(a
    [1])-2*a[0]*a[2])*b[0])*PW2(b[2])+(a[0]*a[2]*PW2(b[1])+(PW2(a[2])
    -4*a[1]*a[3])*PW2(b[0])-(a[1]*a[2]-6*a[0]*a[3])*b[0]*b[1])*b[2]+(2*
    a[0]*a[1]*PW2(b[1])+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a
    [1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*b
    [3])*c[2]+(PW2(a[0])*a[2]*PW2(b[2])+3*PW3(a[0])*PW2(b[3])+3*a[0]*
    PW2(b[0])*PW2(c[3])+(PW3(a[2])-3*a[1]*a[2]*a[3]+3*a[0]*PW2(a[3]))*
    PW2(b[0])-(a[1]*PW2(a[2])-(2*PW2(a[1])+a[0]*a[2])*a[3])*b[0]*b[1]+(
    a[0]*PW2(a[2])-2*a[0]*a[1]*a[3])*PW2(b[1])+((PW2(a[1])*a[2]-2*a[0]*
    PW2(a[2])-a[0]*a[1]*a[3])*b[0]-(a[0]*a[1]*a[2]-3*PW2(a[0])*a[3])*b
    [1])*b[2]-2*(PW2(a[0])*a[1]*b[2]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(
    a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*b[3]+(2*a
    [0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*
    PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a
    [0])*b[1])*b[2])*c[3])*d[2]+(6*a[0]*PW2(b[0])*c[2]*c[3]+(2*a[0]*a
    [1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b
    [0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*
    b[1])*b[2])*c[2]+2*(PW2(a[0])*a[1]*b[2]-3*PW3(a[0])*b[3]-3*PW2(a
    [0])*b[0]*c[3]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(
    a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*d[2])*d[3];
cf[9]=PW3(a[3])*PW3(b[0])-a[2]*PW2(a[3])*PW2(b[0])*b[1]+a[1]*PW2(a[3])
    *b[0]*PW2(b[1])-a[0]*PW2(a[3])*PW3(b[1])+PW2(a[0])*a[3]*PW3(b[2])-
    PW3(a[0])*PW3(b[3])-PW3(b[0])*PW3(c[3])+PW3(a[0])*PW3(d[3])-(a[0]*a
    [1]*a[3]*b[1]-(PW2(a[1])-2*a[0]*a[2])*a[3]*b[0])*PW2(b[2])+(PW2(a
    [0])*a[1]*b[2]+(PW3(a[1])-3*a[0]*a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(
    a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b[1])*PW2(b[3])+(3*a[3]*PW3(b[0])-
    a[2]*PW2(b[0])*b[1]+a[1]*b[0]*PW2(b[1])-a[0]*PW3(b[1])-3*a[0]*PW2(b
```

```
        [0])*b[3]-(2*a[1]*PW2(b[0])-3*a[0]*b[0]*b[1])*b[2])*PW2(c[3])+(PW2(
        a[0])*a[1]*b[2]-3*PW3(a[0])*b[3]-3*PW2(a[0])*b[0]*c[3]+(PW3(a[1])
        -3*a[0]*a[1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a
        [0])*a[2])*b[1])*PW2(d[3])+(a[0]*a[2]*a[3]*PW2(b[1])+(PW2(a[2])*a
        [3]-2*a[1]*PW2(a[3]))*PW2(b[0])-(a[1]*a[2]*a[3]-3*a[0]*PW2(a[3]))*b
        [0]*b[1])*b[2]-(PW2(a[0])*a[2]*PW2(b[2])+(PW3(a[2])-3*a[1]*a[2]*a
        [3]+3*a[0]*PW2(a[3]))*PW2(b[0])-(a[1]*PW2(a[2])-(2*PW2(a[1])+a[0]*a
        [2])*a[3])*b[0]*b[1]+(a[0]*PW2(a[2])-2*a[0]*a[1]*a[3])*PW2(b[1])+((
        PW2(a[1])*a[2]-2*a[0]*PW2(a[2])-a[0]*a[1]*a[3])*b[0]-(a[0]*a[1]*a
        [2]-3*PW2(a[0])*a[3])*b[1])*b[2])*b[3]-(3*PW2(a[3])*PW3(b[0])-2*a
        [2]*a[3]*PW2(b[0])*b[1]+2*a[1]*a[3]*b[0]*PW2(b[1])-2*a[0]*a[3]*PW3(
        b[1])+PW2(a[0])*PW3(b[2])+3*PW2(a[0])*b[0]*PW2(b[3])-(a[0]*a[1]*b
        [1]-(PW2(a[1])-2*a[0]*a[2])*b[0])*PW2(b[2])+(a[0]*a[2]*PW2(b[1])+(
        PW2(a[2])-4*a[1]*a[3])*PW2(b[0])-(a[1]*a[2]-6*a[0]*a[3])*b[0]*b[1])
        *b[2]+(2*a[0]*a[1]*PW2(b[1])+3*(a[1]*a[2]-2*a[0]*a[3])*PW2(b[0])
        -(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*a[1]*b[0]-3*PW2(a[0])*b
        [1])*b[2])*b[3])*c[3]+(PW2(a[0])*a[2]*PW2(b[2])+3*PW3(a[0])*PW2(b
        [3])+3*a[0]*PW2(b[0])*PW2(c[3])+(PW3(a[2])-3*a[1]*a[2]*a[3]+3*a[0]*
        PW2(a[3]))*PW2(b[0])-(a[1]*PW2(a[2])-(2*PW2(a[1])+a[0]*a[2])*a[3])*
        b[0]*b[1]+(a[0]*PW2(a[2])-2*a[0]*a[1]*a[3])*PW2(b[1])+((PW2(a[1])*a
        [2]-2*a[0]*PW2(a[2])-a[0]*a[1]*a[3])*b[0]-(a[0]*a[1]*a[2]-3*PW2(a
        [0])*a[3])*b[1])*b[2]-2*(PW2(a[0])*a[1]*b[2]+(PW3(a[1])-3*a[0]*a
        [1]*a[2]+3*PW2(a[0])*a[3])*b[0]-(a[0]*PW2(a[1])-2*PW2(a[0])*a[2])*b
        [1])*b[3]+(2*a[0]*a[1]*PW2(b[1])+6*PW2(a[0])*b[0]*b[3]+3*(a[1]*a
        [2]-2*a[0]*a[3])*PW2(b[0])-(2*PW2(a[1])+a[0]*a[2])*b[0]*b[1]+(a[0]*
        a[1]*b[0]-3*PW2(a[0])*b[1])*b[2])*c[3])*d[3];
}else if(abs(a[1])>=EZERO||abs(b[1])>=EZERO){
   // Curve 0 is quadratic
   cf[0]=0;
   cf[1]=0;
   cf[2]=0;
   cf[3]=PW2(b[1])*PW2(c[0])-2*a[1]*b[1]*c[0]*d[0]+PW2(a[1])*PW2(d[0]);
   cf[4]=2*PW2(b[1])*c[0]*c[1]-2*a[1]*b[1]*c[1]*d[0]-2*(a[1]*b[1]*c[0]-
      PW2(a[1])*d[0])*d[1];
   cf[5]=PW2(b[1])*PW2(c[1])+2*PW2(b[1])*c[0]*c[2]-2*a[1]*b[1]*c[2]*d
      [0]-2*a[1]*b[1]*c[1]*d[1]+PW2(a[1])*PW2(d[1])-2*(a[1]*b[1]*c[0]-PW2
      (a[1])*d[0])*d[2];
   cf[6]=2*PW2(b[1])*c[1]*c[2]+2*PW2(b[1])*c[0]*c[3]-2*a[1]*b[1]*c[2]*d
      [1]-(2*a[3]*PW2(b[1])-a[2]*b[1]*b[2]+a[1]*PW2(b[2])-2*a[1]*b[1]*b
      [3])*c[0]+(a[1]*a[2]*b[2]-2*PW2(a[1])*b[3]-2*a[1]*b[1]*c[3]-(PW2(a
      [2])-2*a[1]*a[3])*b[1])*d[0]-2*(a[1]*b[1]*c[1]-PW2(a[1])*d[1])*d
      [2]-2*(a[1]*b[1]*c[0]-PW2(a[1])*d[0])*d[3];
   cf[7]=PW2(b[1])*PW2(c[2])+2*PW2(b[1])*c[1]*c[3]-2*a[1]*b[1]*c[2]*d[2]+
      PW2(a[1])*PW2(d[2])-(2*a[3]*PW2(b[1])-a[2]*b[1]*b[2]+a[1]*PW2(b[2])
      -2*a[1]*b[1]*b[3])*c[1]+(a[1]*a[2]*b[2]-2*PW2(a[1])*b[3]-2*a[1]*b
      [1]*c[3]-(PW2(a[2])-2*a[1]*a[3])*b[1])*d[1]-2*(a[1]*b[1]*c[1]-PW2(a
      [1])*d[1])*d[3];
   cf[8]=2*PW2(b[1])*c[2]*c[3]-(2*a[3]*PW2(b[1])-a[2]*b[1]*b[2]+a[1]*PW2(
      b[2])-2*a[1]*b[1]*b[3])*c[2]+(a[1]*a[2]*b[2]-2*PW2(a[1])*b[3]-2*a
      [1]*b[1]*c[3]-(PW2(a[2])-2*a[1]*a[3])*b[1])*d[2]-2*(a[1]*b[1]*c[2]-
      PW2(a[1])*d[2])*d[3];
```

```
      cf[9]=PW2(a[3])*PW2(b[1])-a[2]*a[3]*b[1]*b[2]+a[1]*a[3]*PW2(b[2])+PW2(
         a[1])*PW2(b[3])+PW2(b[1])*PW2(c[3])+PW2(a[1])*PW2(d[3])-(a[1]*a[2]*
         b[2]-(PW2(a[2])-2*a[1]*a[3])*b[1])*b[3]-(2*a[3]*PW2(b[1])-a[2]*b
         [1]*b[2]+a[1]*PW2(b[2])-2*a[1]*b[1]*b[3])*c[3]+(a[1]*a[2]*b[2]-2*
         PW2(a[1])*b[3]-2*a[1]*b[1]*c[3]-(PW2(a[2])-2*a[1]*a[3])*b[1])*d[3];
   }else{
      // Curve 0 is linear
      cf[0]=0;
      cf[1]=0;
      cf[2]=0;
      cf[3]=0;
      cf[4]=0;
      cf[5]=0;
      cf[6]=b[2]*c[0]-a[2]*d[0];
      cf[7]=b[2]*c[1]-a[2]*d[1];
      cf[8]=b[2]*c[2]-a[2]*d[2];
      cf[9]=-a[3]*b[2]+a[2]*b[3]+b[2]*c[3]-a[2]*d[3];
   }
}

// Compute the coefficients of the implicit equation F1(t) for the
   intersection of two cubic parametric curves
void coefImp1(
  double *a, // IN Coefficients of curve 0 (x,y)=f(t); a[0], b[0] are cubic
     terms
  double *b,
  double *c, // IN Coefficients of curve 1 (x,y)=g(u); c[0], d[0] are cubic
     terms
  double *d,
  double *cf) // OUT Coefficients of F1(t); cf[0] is the 9th-degree term
{
   if(abs(c[0])>=EZERO||abs(d[0])>=EZERO){
      // Curve 1 is cubic
      cf[0]=PW3(b[0])*PW3(c[0])-3*a[0]*PW2(b[0])*PW2(c[0])*d[0]+3*PW2(a[0])*
         b[0]*c[0]*PW2(d[0])-PW3(a[0])*PW3(d[0]);
      cf[1]=3*PW2(b[0])*b[1]*PW3(c[0])-3*PW2(a[0])*a[1]*PW3(d[0])-3*(a[1]*
         PW2(b[0])+2*a[0]*b[0]*b[1])*PW2(c[0])*d[0]+3*(2*a[0]*a[1]*b[0]+PW2(
         a[0])*b[1])*c[0]*PW2(d[0]);
      cf[2]=3*(b[0]*PW2(b[1])+PW2(b[0])*b[2])*PW3(c[0])-3*(a[2]*PW2(b[0])+2*
         a[1]*b[0]*b[1]+a[0]*PW2(b[1])+2*a[0]*b[0]*b[2])*PW2(c[0])*d
         [0]+3*(2*a[0]*a[1]*b[1]+PW2(a[0])*b[2]+(PW2(a[1])+2*a[0]*a[2])*b
         [0])*c[0]*PW2(d[0])-3*(a[0]*PW2(a[1])+PW2(a[0])*a[2])*PW3(d[0]);
      cf[3]=-PW2(a[0])*c[0]*PW3(d[1])+(PW3(b[1])+6*b[0]*b[1]*b[2]+3*PW2(b
         [0])*b[3])*PW3(c[0])-(PW3(a[1])+6*a[0]*a[1]*a[2]+3*PW2(a[0])*a
         [3]-3*PW2(a[0])*c[3])*PW3(d[0])+3*(a[0]*b[0]*c[1]*c[2]-2*a[0]*b[0]*
         c[0]*c[3]+(2*a[0]*a[1]*b[2]+PW2(a[0])*b[3]+2*(a[1]*a[2]+a[0]*a[3])*
         b[0]+(PW2(a[1])+2*a[0]*a[2])*b[1])*c[0])*PW2(d[0])+(2*a[0]*b[0]*c
         [0]*c[1]+PW2(a[0])*c[1]*d[0])*PW2(d[1])+(PW2(b[0])*PW3(c[1])-3*PW2(
         b[0])*c[0]*c[1]*c[2]+3*PW2(b[0])*PW2(c[0])*c[3]-3*(a[3]*PW2(b[0])
         +2*a[2]*b[0]*b[1]+a[1]*PW2(b[1])+2*a[0]*b[0]*b[3]+2*(a[1]*b[0]+a
         [0]*b[1])*b[2])*PW2(c[0]))*d[0]-(PW2(b[0])*c[0]*PW2(c[1])-2*PW2(b
```

```
    [0])*PW2(c[0])*c[2]+PW2(a[0])*c[2]*PW2(d[0])+(2*a[0]*b[0]*PW2(c[1])
    +a[0]*b[0]*c[0]*c[2])*d[0])*d[1]+(PW2(b[0])*PW2(c[0])*c[1]+a[0]*b
    [0]*c[0]*c[1]*d[0]-2*PW2(a[0])*c[1]*PW2(d[0])-3*(a[0]*b[0]*PW2(c
    [0])-PW2(a[0])*c[0]*d[0])*d[1])*d[2]-3*(PW2(b[0])*PW3(c[0])-2*a[0]*
    b[0]*PW2(c[0])*d[0]+PW2(a[0])*c[0]*PW2(d[0]))*d[3];
cf[4]=-2*a[0]*a[1]*c[0]*PW3(d[1])+3*(PW2(b[1])*b[2]+b[0]*PW2(b[2])+2*b
    [0]*b[1]*b[3])*PW3(c[0])-3*(PW2(a[1])*a[2]+a[0]*PW2(a[2])+2*a[0]*a
    [1]*a[3]-2*a[0]*a[1]*c[3])*PW3(d[0])+3*((a[1]*b[0]+a[0]*b[1])*c[1]*
    c[2]-2*(a[1]*b[0]+a[0]*b[1])*c[0]*c[3]+(2*a[0]*a[1]*b[3]+(PW2(a[2])
    +2*a[1]*a[3])*b[0]+2*(a[1]*a[2]+a[0]*a[3])*b[1]+(PW2(a[1])+2*a[0]*a
    [2])*b[2])*c[0])*PW2(d[0])+2*(a[0]*a[1]*c[1]*d[0]+(a[1]*b[0]+a[0]*b
    [1])*c[0]*c[1])*PW2(d[1])+(2*b[0]*b[1]*PW3(c[1])-6*b[0]*b[1]*c[0]*c
    [1]*c[2]+6*b[0]*b[1]*PW2(c[0])*c[3]-3*(2*a[3]*b[0]*b[1]+a[2]*PW2(b
    [1])+a[0]*PW2(b[2])+2*(a[2]*b[0]+a[1]*b[1])*b[2]+2*(a[1]*b[0]+a[0]*
    b[1])*b[3])*PW2(c[0]))*d[0]-(2*b[0]*b[1]*c[0]*PW2(c[1])-4*b[0]*b
    [1]*PW2(c[0])*c[2]+2*a[0]*a[1]*c[2]*PW2(d[0])+(2*(a[1]*b[0]+a[0]*b
    [1])*PW2(c[1])+(a[1]*b[0]+a[0]*b[1])*c[0]*c[2])*d[0])*d[1]+(2*b[0]*
    b[1]*PW2(c[0])*c[1]-4*a[0]*a[1]*c[1]*PW2(d[0])+(a[1]*b[0]+a[0]*b
    [1])*c[0]*c[1]*d[0]+3*(2*a[0]*a[1]*c[0]*d[0]-(a[1]*b[0]+a[0]*b[1])*
    PW2(c[0]))*d[1])*d[2]-6*(b[0]*b[1]*PW3(c[0])+a[0]*a[1]*c[0]*PW2(d
    [0])-(a[1]*b[0]+a[0]*b[1])*PW2(c[0])*d[0])*d[3];
cf[5]=-(PW2(a[1])+2*a[0]*a[2])*c[0]*PW3(d[1])+3*(b[1]*PW2(b[2])+(PW2(b
    [1])+2*b[0]*b[2])*b[3])*PW3(c[0])-3*(a[1]*PW2(a[2])+(PW2(a[1])+2*a
    [0]*a[2])*a[3]-(PW2(a[1])+2*a[0]*a[2])*c[3])*PW3(d[0])+3*((a[2]*b
    [0]+a[1]*b[1]+a[0]*b[2])*c[1]*c[2]-2*(a[2]*b[0]+a[1]*b[1]+a[0]*b
    [2])*c[0]*c[3]+(2*a[2]*a[3]*b[0]+(PW2(a[2])+2*a[1]*a[3])*b[1]+2*(a
    [1]*a[2]+a[0]*a[3])*b[2]+(PW2(a[1])+2*a[0]*a[2])*b[3])*c[0])*PW2(d
    [0])+(2*(a[2]*b[0]+a[1]*b[1]+a[0]*b[2])*c[0]*c[1]+(PW2(a[1])+2*a
    [0]*a[2])*c[1]*d[0])*PW2(d[1])+((PW2(b[1])+2*b[0]*b[2])*PW3(c[1])
    -3*(PW2(b[1])+2*b[0]*b[2])*c[0]*c[1]*c[2]+3*(PW2(b[1])+2*b[0]*b[2])
    *PW2(c[0])*c[3]-3*(a[3]*PW2(b[1])+a[1]*PW2(b[2])+2*(a[3]*b[0]+a[2]*
    b[1])*b[2]+2*(a[2]*b[0]+a[1]*b[1]+a[0]*b[2])*b[3])*PW2(c[0]))*d
    [0]-((PW2(b[1])+2*b[0]*b[2])*c[0]*PW2(c[1])-2*(PW2(b[1])+2*b[0]*b
    [2])*PW2(c[0])*c[2]+(PW2(a[1])+2*a[0]*a[2])*c[2]*PW2(d[0])+(2*(a
    [2]*b[0]+a[1]*b[1]+a[0]*b[2])*PW2(c[1])+(a[2]*b[0]+a[1]*b[1]+a[0]*b
    [2])*c[0]*c[2])*d[0])*d[1]+((PW2(b[1])+2*b[0]*b[2])*PW2(c[0])*c
    [1]+(a[2]*b[0]+a[1]*b[1]+a[0]*b[2])*c[0]*c[1]*d[0]-2*(PW2(a[1])+2*a
    [0]*a[2])*c[1]*PW2(d[0])-3*((a[2]*b[0]+a[1]*b[1]+a[0]*b[2])*PW2(c
    [0])-(PW2(a[1])+2*a[0]*a[2])*c[0]*d[0])*d[1])*d[2]-3*((PW2(b[1])+2*
    b[0]*b[2])*PW3(c[0])-2*(a[2]*b[0]+a[1]*b[1]+a[0]*b[2])*PW2(c[0])*d
    [0]+(PW2(a[1])+2*a[0]*a[2])*c[0]*PW2(d[0]))*d[3];
cf[6]=-a[0]*PW2(c[0])*PW3(d[2])+(PW3(b[2])+6*b[1]*b[2]*b[3]+3*b[0]*PW2
    (b[3]))*PW3(c[0])-(PW3(a[2])+6*a[1]*a[2]*a[3]+3*a[0]*PW2(a[3])+3*a
    [0]*PW2(c[3])-6*(a[1]*a[2]+a[0]*a[3])*c[3])*PW3(d[0])+2*(a[0]*c[0]*
    c[3]-(a[1]*a[2]+a[0]*a[3])*c[0])*PW3(d[1])+(b[0]*PW3(c[2])+3*b[0]*c
    [0]*PW2(c[3])+3*(a[3]*b[0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*c[1]*c
    [2]+3*(PW2(a[3])*b[0]+2*a[2]*a[3]*b[1]+(PW2(a[2])+2*a[1]*a[3])*b
    [2]+2*(a[1]*a[2]+a[0]*a[3])*b[3])*c[0]-3*(b[0]*c[1]*c[2]+2*(a[3]*b
    [0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*c[0])*c[3])*PW2(d[0])+(b[0]*c
    [0]*PW2(c[2])-2*b[0]*c[0]*c[1]*c[3]+2*(a[3]*b[0]+a[2]*b[1]+a[1]*b
    [2]+a[0]*b[3])*c[0]*c[1]-2*(a[0]*c[1]*c[3]-(a[1]*a[2]+a[0]*a[3])*c
```

```
    [1])*d[0])*PW2(d[1])+(b[0]*PW2(c[0])*c[2]+a[0]*c[0]*c[1]*d[1]-(a
    [0]*PW2(c[1])-2*a[0]*c[0]*c[2])*d[0])*PW2(d[2])+3*(b[0]*PW3(c[0])-a
    [0]*PW2(c[0])*d[0])*PW2(d[3])+(2*(b[1]*b[2]+b[0]*b[3])*PW3(c[1])
    -6*(b[1]*b[2]+b[0]*b[3])*c[0]*c[1]*c[2]+6*(b[1]*b[2]+b[0]*b[3])*PW2
    (c[0])*c[3]-3*(2*a[3]*b[1]*b[2]+a[2]*PW2(b[2])+a[0]*PW2(b[3])+2*(a
    [3]*b[0]+a[2]*b[1]+a[1]*b[2])*b[3])*PW2(c[0]))*d[0]-(2*(b[1]*b[2]+b
    [0]*b[3])*c[0]*PW2(c[1])-4*(b[1]*b[2]+b[0]*b[3])*PW2(c[0])*c[2]-2*(
    a[0]*c[2]*c[3]-(a[1]*a[2]+a[0]*a[3])*c[2])*PW2(d[0])+(b[0]*c[1]*PW2
    (c[2])+2*(a[3]*b[0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*PW2(c[1])+(a[3]*
    b[0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*c[0]*c[2]-(2*b[0]*PW2(c[1])+b
    [0]*c[0]*c[2])*c[3])*d[0])*d[1]-(a[0]*c[0]*c[2]*PW2(d[1])-2*(b[1]*b
    [2]+b[0]*b[3])*PW2(c[0])*c[1]+(a[0]*PW2(c[2])-4*a[0]*c[1]*c[3]+4*(a
    [1]*a[2]+a[0]*a[3])*c[1])*PW2(d[0])-(b[0]*PW2(c[1])*c[2]-2*b[0]*c
    [0]*PW2(c[2])-b[0]*c[0]*c[1]*c[3]+(a[3]*b[0]+a[2]*b[1]+a[1]*b[2]+a
    [0]*b[3])*c[0]*c[1])*d[0]+(b[0]*c[0]*c[1]*c[2]-3*b[0]*PW2(c[0])*c
    [3]+3*(a[3]*b[0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*PW2(c[0])-(a[0]*c
    [1]*c[2]-6*a[0]*c[0]*c[3]+6*(a[1]*a[2]+a[0]*a[3])*c[0])*d[0])*d[1])
    *d[2]-(2*a[0]*c[0]*c[1]*PW2(d[1])+6*(b[1]*b[2]+b[0]*b[3])*PW3(c[0])
    +3*(a[0]*c[1]*c[2]-2*a[0]*c[0]*c[3]+2*(a[1]*a[2]+a[0]*a[3])*c[0])*
    PW2(d[0])+2*(b[0]*PW3(c[1])-3*b[0]*c[0]*c[1]*c[2]+3*b[0]*PW2(c[0])*
    c[3]-3*(a[3]*b[0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*PW2(c[0]))*d
    [0]-(2*b[0]*c[0]*PW2(c[1])-4*b[0]*PW2(c[0])*c[2]+(2*a[0]*PW2(c[1])+
    a[0]*c[0]*c[2])*d[0])*d[1]+(2*b[0]*PW2(c[0])*c[1]+a[0]*c[0]*c[1]*d
    [0]-3*a[0]*PW2(c[0])*d[1])*d[2])*d[3];
cf[7]=-a[1]*PW2(c[0])*PW3(d[2])+3*(PW2(b[2])*b[3]+b[1]*PW2(b[3]))*PW3(
    c[0])-3*(PW2(a[2])*a[3]+a[1]*PW2(a[3])+a[1]*PW2(c[3])-(PW2(a[2])+2*
    a[1]*a[3])*c[3])*PW3(d[0])+(2*a[1]*c[0]*c[3]-(PW2(a[2])+2*a[1]*a
    [3])*c[0])*PW3(d[1])+(b[1]*PW3(c[2])+3*b[1]*c[0]*PW2(c[3])+3*(a[3]*
    b[1]+a[2]*b[2]+a[1]*b[3])*c[1]*c[2]+3*(PW2(a[3])*b[1]+2*a[2]*a[3]*b
    [2]+(PW2(a[2])+2*a[1]*a[3])*b[3])*c[0]-3*(b[1]*c[1]*c[2]+2*(a[3]*b
    [1]+a[2]*b[2]+a[1]*b[3])*c[0])*c[3])*PW2(d[0])+(b[1]*c[0]*PW2(c[2])
    -2*b[1]*c[0]*c[1]*c[3]+2*(a[3]*b[1]+a[2]*b[2]+a[1]*b[3])*c[0]*c
    [1]-(2*a[1]*c[1]*c[3]-(PW2(a[2])+2*a[1]*a[3])*c[1])*d[0])*PW2(d[1])
    +(b[1]*PW2(c[0])*c[2]+a[1]*c[0]*c[1]*d[1]-(a[1]*PW2(c[1])-2*a[1]*c
    [0]*c[2])*d[0])*PW2(d[2])+3*(b[1]*PW3(c[0])-a[1]*PW2(c[0])*d[0])*
    PW2(d[3])+((PW2(b[2])+2*b[1]*b[3])*PW3(c[1])-3*(PW2(b[2])+2*b[1]*b
    [3])*c[0]*c[1]*c[2]+3*(PW2(b[2])+2*b[1]*b[3])*PW2(c[0])*c[3]-3*(a
    [3]*PW2(b[2])+a[1]*PW2(b[3])+2*(a[3]*b[1]+a[2]*b[2])*b[3])*PW2(c
    [0]))*d[0]-((PW2(b[2])+2*b[1]*b[3])*c[0]*PW2(c[1])-2*(PW2(b[2])+2*b
    [1]*b[3])*PW2(c[0])*c[2]-(2*a[1]*c[2]*c[3]-(PW2(a[2])+2*a[1]*a[3])*
    c[2])*PW2(d[0])+(b[1]*c[1]*PW2(c[2])+2*(a[3]*b[1]+a[2]*b[2]+a[1]*b
    [3])*PW2(c[1])+(a[3]*b[1]+a[2]*b[2]+a[1]*b[3])*c[0]*c[2]-(2*b[1]*
    PW2(c[1])+b[1]*c[0]*c[2])*c[3])*d[0])*d[1]-(a[1]*c[0]*c[2]*PW2(d
    [1])-(PW2(b[2])+2*b[1]*b[3])*PW2(c[0])*c[1]+(a[1]*PW2(c[2])-4*a[1]*
    c[1]*c[3]+2*(PW2(a[2])+2*a[1]*a[3])*c[1])*PW2(d[0])-(b[1]*PW2(c[1])
    *c[2]-2*b[1]*c[0]*PW2(c[2])-b[1]*c[0]*c[1]*c[3]+(a[3]*b[1]+a[2]*b
    [2]+a[1]*b[3])*c[0]*c[1])*d[0]+(b[1]*c[0]*c[1]*c[2]-3*b[1]*PW2(c
    [0])*c[3]+3*(a[3]*b[1]+a[2]*b[2]+a[1]*b[3])*PW2(c[0])-(a[1]*c[1]*c
    [2]-6*a[1]*c[0]*c[3]+3*(PW2(a[2])+2*a[1]*a[3])*c[0])*d[0])*d[1])*d
    [2]-(2*a[1]*c[0]*c[1]*PW2(d[1])+3*(PW2(b[2])+2*b[1]*b[3])*PW3(c[0])
    +3*(a[1]*c[1]*c[2]-2*a[1]*c[0]*c[3]+(PW2(a[2])+2*a[1]*a[3])*c[0])*
```

```
    PW2(d[0])+2*(b[1]*PW3(c[1])-3*b[1]*c[0]*c[1]*c[2]+3*b[1]*PW2(c[0])*
    c[3]-3*(a[3]*b[1]+a[2]*b[2]+a[1]*b[3])*PW2(c[0]))*d[0]-(2*b[1]*c
    [0]*PW2(c[1])-4*b[1]*PW2(c[0])*c[2]+(2*a[1]*PW2(c[1])+a[1]*c[0]*c
    [2])*d[0])*d[1]+(2*b[1]*PW2(c[0])*c[1]+a[1]*c[0]*c[1]*d[0]-3*a[1]*
    PW2(c[0])*d[1])*d[2])*d[3];
cf[8]=3*b[2]*PW2(b[3])*PW3(c[0])-a[2]*PW2(c[0])*PW3(d[2])-3*(a[2]*PW2(
    a[3])-2*a[2]*a[3]*c[3]+a[2]*PW2(c[3]))*PW3(d[0])-2*(a[2]*a[3]*c[0]-
    a[2]*c[0]*c[3])*PW3(d[1])+(b[2]*PW3(c[2])+3*b[2]*c[0]*PW2(c[3])+3*(
    a[3]*b[2]+a[2]*b[3])*c[1]*c[2]+3*(PW2(a[3])*b[2]+2*a[2]*a[3]*b[3])*
    c[0]-3*(b[2]*c[1]*c[2]+2*(a[3]*b[2]+a[2]*b[3])*c[0])*c[3])*PW2(d
    [0])+(b[2]*c[0]*PW2(c[2])-2*b[2]*c[0]*c[1]*c[3]+2*(a[3]*b[2]+a[2]*b
    [3])*c[0]*c[1]+2*(a[2]*a[3]*c[1]-a[2]*c[1]*c[3])*d[0])*PW2(d[1])+(b
    [2]*PW2(c[0])*c[2]+a[2]*c[0]*c[1]*d[1]-(a[2]*PW2(c[1])-2*a[2]*c[0]*
    c[2])*d[0])*PW2(d[2])+3*(b[2]*PW3(c[0])-a[2]*PW2(c[0])*d[0])*PW2(d
    [3])+(2*b[2]*b[3]*PW3(c[1])-6*b[2]*b[3]*c[0]*c[1]*c[2]+6*b[2]*b[3]*
    PW2(c[0])*c[3]-3*(2*a[3]*b[2]*b[3]+a[2]*PW2(b[3]))*PW2(c[0]))*d
    [0]-(2*b[2]*b[3]*c[0]*PW2(c[1])-4*b[2]*b[3]*PW2(c[0])*c[2]+2*(a[2]*
    a[3]*c[2]-a[2]*c[2]*c[3])*PW2(d[0])+(b[2]*c[1]*PW2(c[2])+2*(a[3]*b
    [2]+a[2]*b[3])*PW2(c[1])+(a[3]*b[2]+a[2]*b[3])*c[0]*c[2]-(2*b[2]*
    PW2(c[1])+b[2]*c[0]*c[2])*c[3])*d[0])*d[1]+(2*b[2]*b[3]*PW2(c[0])*c
    [1]-a[2]*c[0]*c[2]*PW2(d[1])-(4*a[2]*a[3]*c[1]+a[2]*PW2(c[2])-4*a
    [2]*c[1]*c[3])*PW2(d[0])+(b[2]*PW2(c[1])*c[2]-2*b[2]*c[0]*PW2(c[2])
    -b[2]*c[0]*c[1]*c[3]+(a[3]*b[2]+a[2]*b[3])*c[0]*c[1])*d[0]-(b[2]*c
    [0]*c[1]*c[2]-3*b[2]*PW2(c[0])*c[3]+3*(a[3]*b[2]+a[2]*b[3])*PW2(c
    [0])-(6*a[2]*a[3]*c[0]+a[2]*c[1]*c[2]-6*a[2]*c[0]*c[3])*d[0])*d[1])
    *d[2]-(6*b[2]*b[3]*PW3(c[0])+2*a[2]*c[0]*c[1]*PW2(d[1])+3*(2*a[2]*a
    [3]*c[0]+a[2]*c[1]*c[2]-2*a[2]*c[0]*c[3])*PW2(d[0])+2*(b[2]*PW3(c
    [1])-3*b[2]*c[0]*c[1]*c[2]+3*b[2]*PW2(c[0])*c[3]-3*(a[3]*b[2]+a[2]*
    b[3])*PW2(c[0]))*d[0]-(2*b[2]*c[0]*PW2(c[1])-4*b[2]*PW2(c[0])*c
    [2]+(2*a[2]*PW2(c[1])+a[2]*c[0]*c[2])*d[0])*d[1]+(2*b[2]*PW2(c[0])*
    c[1]+a[2]*c[0]*c[1]*d[0]-3*a[2]*PW2(c[0])*d[1])*d[2])*d[3];
cf[9]=PW3(b[3])*PW3(c[0])-PW3(c[0])*PW3(d[3])-(PW3(a[3])-3*PW2(a[3])*c
    [3]+3*a[3]*PW2(c[3])-PW3(c[3]))*PW3(d[0])-(PW2(a[3])*c[0]-2*a[3]*c
    [0]*c[3]+c[0]*PW2(c[3]))*PW3(d[1])-(a[3]*PW2(c[0])-PW2(c[0])*c[3])*
    PW3(d[2])+(3*PW2(a[3])*b[3]*c[0]+3*a[3]*b[3]*c[1]*c[2]+b[3]*PW3(c
    [2])+3*b[3]*c[0]*PW2(c[3])-3*(2*a[3]*b[3]*c[0]+b[3]*c[1]*c[2])*c
    [3])*PW2(d[0])+(2*a[3]*b[3]*c[0]*c[1]+b[3]*c[0]*PW2(c[2])-2*b[3]*c
    [0]*c[1]*c[3]+(PW2(a[3])*c[1]-2*a[3]*c[1]*c[3]+c[1]*PW2(c[3]))*d
    [0])*PW2(d[1])+(b[3]*PW2(c[0])*c[2]-(a[3]*PW2(c[1])-2*a[3]*c[0]*c
    [2]-(PW2(c[1])-2*c[0]*c[2])*c[3])*d[0]+(a[3]*c[0]*c[1]-c[0]*c[1]*c
    [3])*d[1])*PW2(d[2])+(3*b[3]*PW3(c[0])+PW2(c[0])*c[1]*d[2]-(3*a[3]*
    PW2(c[0])-PW3(c[1])+3*c[0]*c[1]*c[2]-3*PW2(c[0])*c[3])*d[0]-(c[0]*
    PW2(c[1])-2*PW2(c[0])*c[2])*d[1])*PW2(d[3])-(3*a[3]*PW2(b[3])*PW2(c
    [0])-PW2(b[3])*PW3(c[1])+3*PW2(b[3])*c[0]*c[1]*c[2]-3*PW2(b[3])*PW2
    (c[0])*c[3])*d[0]-(PW2(b[3])*c[0]*PW2(c[1])-2*PW2(b[3])*PW2(c[0])*c
    [2]+(PW2(a[3])*c[2]-2*a[3]*c[2]*c[3]+c[2]*PW2(c[3]))*PW2(d[0])+(2*a
    [3]*b[3]*PW2(c[1])+a[3]*b[3]*c[0]*c[2]+b[3]*c[1]*PW2(c[2])-(2*b[3]*
    PW2(c[1])+b[3]*c[0]*c[2])*c[3])*d[0])*d[1]+(PW2(b[3])*PW2(c[0])*c
    [1]-(2*PW2(a[3])*c[1]+a[3]*PW2(c[2])+2*c[1]*PW2(c[3])-(4*a[3]*c[1]+
    PW2(c[2]))*c[3])*PW2(d[0])-(a[3]*c[0]*c[2]-c[0]*c[2]*c[3])*PW2(d
    [1])+(a[3]*b[3]*c[0]*c[1]+b[3]*PW2(c[1])*c[2]-2*b[3]*c[0]*PW2(c[2])
```

```
      -b[3]*c[0]*c[1]*c[3])*d[0]-(3*a[3]*b[3]*PW2(c[0])+b[3]*c[0]*c[1]*c
      [2]-3*b[3]*PW2(c[0])*c[3]-(3*PW2(a[3])*c[0]+a[3]*c[1]*c[2]+3*c[0]*
      PW2(c[3])-(6*a[3]*c[0]+c[1]*c[2])*c[3])*d[0])*d[1])*d[2]-(3*PW2(b
      [3])*PW3(c[0])+PW2(c[0])*c[2]*PW2(d[2])+(3*PW2(a[3])*c[0]+3*a[3]*c
      [1]*c[2]+PW3(c[2])+3*c[0]*PW2(c[3])-3*(2*a[3]*c[0]+c[1]*c[2])*c[3])
      *PW2(d[0])+(2*a[3]*c[0]*c[1]+c[0]*PW2(c[2])-2*c[0]*c[1]*c[3])*PW2(d
      [1])-2*(3*a[3]*b[3]*PW2(c[0])-b[3]*PW3(c[1])+3*b[3]*c[0]*c[1]*c
      [2]-3*b[3]*PW2(c[0])*c[3])*d[0]-(2*b[3]*c[0]*PW2(c[1])-4*b[3]*PW2(c
      [0])*c[2]+(2*a[3]*PW2(c[1])+a[3]*c[0]*c[2]+c[1]*PW2(c[2])-(2*PW2(c
      [1])+c[0]*c[2])*c[3])*d[0])*d[1]+(2*b[3]*PW2(c[0])*c[1]+(a[3]*c[0]*
      c[1]+PW2(c[1])*c[2]-2*c[0]*PW2(c[2])-c[0]*c[1]*c[3])*d[0]-(3*a[3]*
      PW2(c[0])+c[0]*c[1]*c[2]-3*PW2(c[0])*c[3])*d[1])*d[2])*d[3];
}else if(abs(c[1])>=EZERO||abs(d[1])>=EZERO){
   // Curve 1 is quadratic
   cf[0]=0;
   cf[1]=0;
   cf[2]=0;
   cf[3]=PW2(b[0])*PW2(c[1])-2*a[0]*b[0]*c[1]*d[1]+PW2(a[0])*PW2(d[1]);
   cf[4]=2*b[0]*b[1]*PW2(c[1])+2*a[0]*a[1]*PW2(d[1])-2*(a[1]*b[0]+a[0]*b
      [1])*c[1]*d[1];
   cf[5]=(PW2(b[1])+2*b[0]*b[2])*PW2(c[1])-2*(a[2]*b[0]+a[1]*b[1]+a[0]*b
      [2])*c[1]*d[1]+(PW2(a[1])+2*a[0]*a[2])*PW2(d[1]);
   cf[6]=-a[0]*c[1]*PW2(d[2])+2*(b[1]*b[2]+b[0]*b[3])*PW2(c[1])+2*(a[1]*a
      [2]+a[0]*a[3]-a[0]*c[3])*PW2(d[1])-(b[0]*PW2(c[2])-2*b[0]*c[1]*c
      [3]+2*(a[3]*b[0]+a[2]*b[1]+a[1]*b[2]+a[0]*b[3])*c[1])*d[1]+(b[0]*c
      [1]*c[2]+a[0]*c[2]*d[1])*d[2]-2*(b[0]*PW2(c[1])-a[0]*c[1]*d[1])*d
      [3];
   cf[7]=-a[1]*c[1]*PW2(d[2])+(PW2(b[2])+2*b[1]*b[3])*PW2(c[1])+(PW2(a
      [2])+2*a[1]*a[3]-2*a[1]*c[3])*PW2(d[1])-(b[1]*PW2(c[2])-2*b[1]*c
      [1]*c[3]+2*(a[3]*b[1]+a[2]*b[2]+a[1]*b[3])*c[1])*d[1]+(b[1]*c[1]*c
      [2]+a[1]*c[2]*d[1])*d[2]-2*(b[1]*PW2(c[1])-a[1]*c[1]*d[1])*d[3];
   cf[8]=2*b[2]*b[3]*PW2(c[1])-a[2]*c[1]*PW2(d[2])+2*(a[2]*a[3]-a[2]*c
      [3])*PW2(d[1])-(b[2]*PW2(c[2])-2*b[2]*c[1]*c[3]+2*(a[3]*b[2]+a[2]*b
      [3])*c[1])*d[1]+(b[2]*c[1]*c[2]+a[2]*c[2]*d[1])*d[2]-2*(b[2]*PW2(c
      [1])-a[2]*c[1]*d[1])*d[3];
   cf[9]=PW2(b[3])*PW2(c[1])+PW2(c[1])*PW2(d[3])+(PW2(a[3])-2*a[3]*c[3]+
      PW2(c[3]))*PW2(d[1])-(a[3]*c[1]-c[1]*c[3])*PW2(d[2])-(2*a[3]*b[3]*c
      [1]+b[3]*PW2(c[2])-2*b[3]*c[1]*c[3])*d[1]+(b[3]*c[1]*c[2]+(a[3]*c
      [2]-c[2]*c[3])*d[1])*d[2]-(2*b[3]*PW2(c[1])+c[1]*c[2]*d[2]-(2*a[3]*
      c[1]+PW2(c[2])-2*c[1]*c[3])*d[1])*d[3];
}else{
   // Curve 1 is linear
   cf[0]=0;
   cf[1]=0;
   cf[2]=0;
   cf[3]=0;
   cf[4]=0;
   cf[5]=0;
   cf[6]=-b[0]*c[2]+a[0]*d[2];
   cf[7]=-b[1]*c[2]+a[1]*d[2];
   cf[8]=-b[2]*c[2]+a[2]*d[2];
```

```
      cf[9]=-b[3]*c[2]+(a[3]-c[3])*d[2]+c[2]*d[3];
   }
}

// Evaluate the function value
double p_(
  double *ca, // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
  double x)  // IN x
{
   return (((((((ca[0]*x+ca[1])*x+ca[2])*x+ca[3])*x+ca[4])*x+ca[5])*x+ca
      [6])*x+ca[7])*x+ca[8])*x+ca[9];
}

// Evaluate the derivative value
double pd(
  double *ca, // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
  double x)  // IN x
{
   return (((((((9*ca[0]*x+8*ca[1])*x+7*ca[2])*x+6*ca[3])*x+5*ca[4])*x+4*ca
      [5])*x+3*ca[6])*x+2*ca[7])*x+ca[8];
}

// Count the number of sign variations in the coefficient sequence
int nSignVariation(
  double *a, // IN Coefficient array
  int n)  // IN Size of the coefficient array minus one
{
   int rv=0;
   int sign=0;
   for(int i=0;i<=n;i++){
      if((sign==1&&*a<0)||(sign==-1&&*a>0))
         ++rv;
      if(*a>0)
         sign=1;
      else if(*a<0)
         sign=-1;
      ++a;
   }
   return rv;
}

// Compute the coefficients of p(x+1) for Budan's theorem
void coefBudan(
  double *ca, // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
  double *cb) // OUT Coefficient array of p(x+1); cb[0] is the 9th-degree
     coefficient
{
   cb[0]=ca[0];
```

```
   cb[1]=9*ca[0]+ca[1];
   cb[2]=36*ca[0]+8*ca[1]+ca[2];
   cb[3]=84*ca[0]+28*ca[1]+7*ca[2]+ca[3];
   cb[4]=126*ca[0]+56*ca[1]+21*ca[2]+6*ca[3]+ca[4];
   cb[5]=126*ca[0]+70*ca[1]+35*ca[2]+15*ca[3]+5*ca[4]+ca[5];
   cb[6]=84*ca[0]+56*ca[1]+35*ca[2]+20*ca[3]+10*ca[4]+4*ca[5]+ca[6];
   cb[7]=36*ca[0]+28*ca[1]+21*ca[2]+15*ca[3]+10*ca[4]+6*ca[5]+3*ca[6]+ca[7];
   cb[8]=9*ca[0]+8*ca[1]+7*ca[2]+6*ca[3]+5*ca[4]+4*ca[5]+3*ca[6]+2*ca[7]+ca
      [8];
   cb[9]=ca[0]+ca[1]+ca[2]+ca[3]+ca[4]+ca[5]+ca[6]+ca[7]+ca[8]+ca[9];
}

// Compute the coefficients for the VAG method
void coefVAG(
  double *ca, // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
  double a,  // IN Interval endpoints
  double b,
  double *cv) // OUT Coefficient array for the VAG method; cv[0] is the 9th-
     degree coefficient
{
   cv[0]=(((((((ca[0]*b+ca[1])*b+ca[2])*b+ca[3])*b+ca[4])*b+ca[5])*b+ca[6])
      *b+ca[7])*b+ca[8])*b+ca[9];
   cv[1]=(((((((9*a*ca[0]+ca[1])*b+8*a*ca[1]+2*ca[2])*b+7*a*ca[2]+3*ca[3])*
      b+6*a*ca[3]+4*ca[4])*b+5*a*ca[4]+5*ca[5])*b+4*a*ca[5]+6*ca[6])*b+3*a*
      ca[6]+7*ca[7])*b+2*a*ca[7]+8*ca[8])*b+a*ca[8]+9*ca[9];
   cv[2]=(((((((36*a*ca[0]+8*ca[1])*a+ca[2])*b+(28*a*ca[1]+14*ca[2])*a+3*ca
      [3])*b+(21*a*ca[2]+18*ca[3])*a+6*ca[4])*b+(15*a*ca[3]+20*ca[4])*a+10*
      ca[5])*b+(10*a*ca[4]+20*ca[5])*a+15*ca[6])*b+(6*a*ca[5]+18*ca[6])*a
      +21*ca[7])*b+(3*a*ca[6]+14*ca[7])*a+28*ca[8])*b+(a*ca[7]+8*ca[8])*a
      +36*ca[9];
   cv[3]=(((((((84*a*ca[0]+28*ca[1])*a+7*ca[2])*a+ca[3])*b+((56*a*ca[1]+42*
      ca[2])*a+18*ca[3])*a+4*ca[4])*b+((35*a*ca[2]+45*ca[3])*a+30*ca[4])*a
      +10*ca[5])*b+((20*a*ca[3]+40*ca[4])*a+40*ca[5])*a+20*ca[6])*b+((10*a*
      ca[4]+30*ca[5])*a+45*ca[6])*a+35*ca[7])*b+((4*a*ca[5]+18*ca[6])*a+42*
      ca[7])*a+56*ca[8])*b+((a*ca[6]+7*ca[7])*a+28*ca[8])*a+84*ca[9];
   cv[4]=(((((((126*a*ca[0]+56*ca[1])*a+21*ca[2])*a+6*ca[3])*a+ca[4])*b
      +(((70*a*ca[1]+70*ca[2])*a+45*ca[3])*a+20*ca[4])*a+5*ca[5])*b+(((35*a
      *ca[2]+60*ca[3])*a+60*ca[4])*a+40*ca[5])*a+15*ca[6])*b+(((15*a*ca
      [3]+40*ca[4])*a+60*ca[5])*a+60*ca[6])*a+35*ca[7])*b+(((5*a*ca[4]+20*
      ca[5])*a+45*ca[6])*a+70*ca[7])*a+70*ca[8])*b+(((a*ca[5]+6*ca[6])*a
      +21*ca[7])*a+56*ca[8])*a+126*ca[9];
   cv[5]=(((((((126*ca[0]*b+56*ca[1])*b+21*ca[2])*b+6*ca[3])*b+ca[4])*a
      +(((70*ca[1]*b+70*ca[2])*b+45*ca[3])*b+20*ca[4])*b+5*ca[5])*a+(((35*
      ca[2]*b+60*ca[3])*b+60*ca[4])*b+40*ca[5])*b+15*ca[6])*a+(((15*ca[3]*b
      +40*ca[4])*b+60*ca[5])*b+60*ca[6])*b+35*ca[7])*a+(((5*ca[4]*b+20*ca
      [5])*b+45*ca[6])*b+70*ca[7])*b+70*ca[8])*a+(((ca[5]*b+6*ca[6])*b+21*
      ca[7])*b+56*ca[8])*b+126*ca[9];
   cv[6]=(((((((84*ca[0]*b+28*ca[1])*b+7*ca[2])*b+ca[3])*a+((56*ca[1]*b+42*
      ca[2])*b+18*ca[3])*b+4*ca[4])*a+((35*ca[2]*b+45*ca[3])*b+30*ca[4])*b
      +10*ca[5])*a+((20*ca[3]*b+40*ca[4])*b+40*ca[5])*b+20*ca[6])*a+((10*ca
```

```
      [4]*b+30*ca[5])*b+45*ca[6])*b+35*ca[7])*a+((4*ca[5]*b+18*ca[6])*b+42*
      ca[7])*b+56*ca[8])*a+((ca[6]*b+7*ca[7])*b+28*ca[8])*b+84*ca[9];
   cv[7]=(((((((36*ca[0]*b+8*ca[1])*b+ca[2])*a+(28*ca[1]*b+14*ca[2])*b+3*ca
      [3])*a+(21*ca[2]*b+18*ca[3])*b+6*ca[4])*a+(15*ca[3]*b+20*ca[4])*b+10*
      ca[5])*a+(10*ca[4]*b+20*ca[5])*b+15*ca[6])*a+(6*ca[5]*b+18*ca[6])*b
      +21*ca[7])*a+(3*ca[6]*b+14*ca[7])*b+28*ca[8])*a+(ca[7]*b+8*ca[8])*b
      +36*ca[9];
   cv[8]=(((((((9*ca[0]*b+ca[1])*a+8*ca[1]*b+2*ca[2])*a+7*ca[2]*b+3*ca[3])*
      a+6*ca[3]*b+4*ca[4])*a+5*ca[4]*b+5*ca[5])*a+4*ca[5]*b+6*ca[6])*a+3*ca
      [6]*b+7*ca[7])*a+2*ca[7]*b+8*ca[8])*a+ca[8]*b+9*ca[9];
   cv[9]=(((((((ca[0]*a+ca[1])*a+ca[2])*a+ca[3])*a+ca[4])*a+ca[5])*a+ca[6])
      *a+ca[7])*a+ca[8])*a+ca[9];
}

// Test for the existence of real roots using Budan's theorem
int vBudan( // RETURN Difference in the number of sign variations between p(
   x) and p(x+1)
 double *ca) // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
{
   double cb[10];
   coefBudan(ca,cb);
   return nSignVariation(ca,9)-nSignVariation(cb,9);
}

// Find intervals containing exactly one real root using the VAG method
// Notes:
// This function is called recursively.
// Depending on the value of ABMIN (minimum interval length) and the
   execution environment, stack overflow may occur.
// Adjust the stack size if necessary.
void vVAG(
 double *ca,   // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
 Interval &iv,   // IN Target interval
 vector<Interval> &ov) // OUT Intervals containing exactly one real root
{
   Interval nv;
   double cv[10];
   coefVAG(ca,iv.a,iv.b,cv);
   int var=nSignVariation(cv,9);
   double m=(iv.a+iv.b)/2;
   if(var==0){
      return;
   }else if(var==1){
      nv.a=iv.a;
      nv.b=iv.b;
      nv.pa=p_(ca,iv.a);
      nv.pb=p_(ca,iv.b);
      nv.nr=var;
      ov.push_back(nv);
```

```
      return;
   }else if(iv.b-iv.a<ABMIN){
      // Multiple root
      // Expand the interval to ABADJ*2 and determine the multiplicity
      double a=m-ABADJ;
      double b=m+ABADJ;
      double pa=p_(ca,a);
      double pb=p_(ca,b);
      coefVAG(ca,a,b,cv);
      var=nSignVariation(cv,9);
      nv.a=a;
      nv.b=b;
      nv.pa=pa;
      nv.pb=pb;
      nv.nr=var;
      ov.push_back(nv);
      return;
   }
   double pm=p_(ca,m);
   if(abs(pm)<EZERO){
      double a=m-ABADJ;
      double b=m+ABADJ;
      double pa=p_(ca,a);
      double pb=p_(ca,b);
      nv.a=iv.a;
      nv.b=a;
      nv.pa=iv.pa;
      nv.pb=pa;
      vVAG(ca,nv,ov);

      coefVAG(ca,a,b,cv);
      var=nSignVariation(cv,9);
      nv.nr=var;
      if(var==1){
         nv.a=nv.b=m;
         nv.pa=nv.pb=pm;
      }else{
         // Multiple root
         nv.a=a;
         nv.b=b;
         nv.pa=pa;
         nv.pb=pb;
      }
      ov.push_back(nv);

      nv.a=b;
      nv.b=iv.b;
      nv.pa=pb;
      nv.pb=iv.pb;
      vVAG(ca,nv,ov);
   }else{
```

```
      nv.a=iv.a;
      nv.b=m;
      nv.pa=iv.pa;
      nv.pb=pm;
      vVAG(ca,nv,ov);

      nv.a=m;
      nv.b=iv.b;
      nv.pa=pm;
      nv.pb=iv.pb;
      vVAG(ca,nv,ov);
   }
}

// Shrink an interval containing one real root by half using the bisection
   method
void vBisection(
 double *ca,  // IN Coefficient array of the function; ca[0] is the 9th-
    degree coefficient
 Interval &v)  // IN/OUT Target interval
{
   double m=(v.a+v.b)/2;
   double pm=p_(ca,m);
   if(abs(pm)<EZERO){
      v.a=v.b=m;
      v.pa=v.pb=0.0;
      return;
   }else if(((v.pa>=EZERO||(abs(v.pa)<EZERO&&pd(ca,v.a)>0.0))&&pm<=-EZERO)||
         ((v.pa<=-EZERO||(abs(v.pa)<EZERO&&pd(ca,v.a)<0.0))&&pm>=EZERO)){
      v.b=m;
      v.pb=pm;
   }else{
      v.a=m;
      v.pa=pm;
   }
   return;
}

// Find a real root of a 9th-degree function using Ridder's method
// Source: Kiusalaas, Jaan (2010). Numerical Methods in Engineering with
   Python (2nd ed.). Cambridge University Press. pp. 147-148. ISBN
   978-0-521-19132-6
double ridder( // RETURN Approximate root
 double *ca,  // IN Coefficient array of the function; ca[0] is the 9th-
    degree coefficient
 Interval &v,  // IN Target interval
 double tol)  // IN Convergence tolerance
{
   double a=v.a;
   double b=v.b;
   double fa=v.pa;
```

```
   double fb=v.pb;
   if(fa==0.0)
      return a;
   if(fb==0.0)
      return b;
   if(fa*fb>0.0){
      return nan(""); // root is not bracketed
   }
   double xOld;
   for(int i=0;i<30;i++){
      // Compute the improved root x from Ridder's formula
      double c=0.5*(a+b);
      double fc=p_(ca,c);
      double s=sqrt(fc*fc-fa*fb);
      if(s==0)
         return nan("");
      double dx=(c-a)*fc/s;
      if(fa-fb<0.0)
         dx=-dx;
      double x=c+dx;
      double fx=p_(ca,x);
      // Test for convergence
      if(i>0&&abs(x-xOld)<tol)
         return x;
      xOld=x;
      // Re-bracket the root as tightly as possible
      if(fc*fx>0.0){
         if(fa*fx<0.0){
            b=x;
            fb=fx;
         }else{
            a=x;
            fa=fx;
         }
      }else{
         a=c;
         b=x;
         fa=fc;
         fb=fx;
      }
   }
   return nan(""); // Too many iterations
}

// Find real roots of a 9th-degree function in the interval [0,1}
void vRoots(
  double *ca,   // IN Coefficient array of the function; ca[0] is the 9th-
     degree coefficient
  vector<double> &rt) // OUT OUT Real roots
{
   rt.clear();
```

```
Interval iv;   // Interval for finding root
iv.a=0.0;
iv.b=1.0;
iv.pa=p_(ca,0.0);
iv.pb=p_(ca,1.0);

vector<Interval> rv; // Intervals containing exactly one real root
rv.reserve(9);

if(abs(iv.pa)<EZERO){
   // Since the function value at the start point is zero, treat 0 as a
      root
   Interval nv;
   nv.a=nv.b=0.0;
   nv.pa=nv.pb=iv.pa;
   rv.push_back(nv);
}

// Find real roots in the interval {0,1}
int nb=vBudan(ca); // Test for the existence of real roots using Budan's
   theorem
if(nb==1){
   // There is exactly one real root in {0,1}
   rv.push_back(iv);
}else if(nb>=2){
   // There is more than one real root in {0,1}
   vVAG(ca,iv,rv);
}
if(rv.size()==0){
   // No intersection found; terminate
   return;
}
// Shrink intervals containing one real root to 1/8 of their length using
    the bisection method
// Skip intervals with multiple roots
for(Interval& v:rv){
   if(v.nr==1&&v.a<v.b){
      vBisection(ca,v);
      vBisection(ca,v);
      vBisection(ca,v);
   }
}
for(Interval& v:rv){
   if(v.a==v.b){
      rt.push_back(v.a);
   }else if(v.nr==1){
      // No multiplicity: compute the real root using Ridder's method
      double r=ridder(ca,v,ABMIN);
      if(!isnan(r)){
         rt.push_back(r);
```

```
         }
      }else{
         // Multiple roots: use the midpoint of the interval as the root
         rt.push_back((v.a+v.b)/2);
      }
   }
}

// Compute intersection points of two cubic Bezier curves
bool vBezierIntersection( // RETURN true: success
  QList<QPointF> &cp0, // IN Control points of cubic B^^c3^^a9zier curve #0
  QList<QPointF> &cp1, // IN Control points of cubic B^^c3^^a9zier curve #1
  QList<QPointF> &op) // OUT Intersection point coordinates
{
   op.clear();
   if(cp0.size()!=4||cp1.size()!=4)
      return false;

   // Compute normalized coordinates of control points
   QList<QPointF> ncp0;
   QList<QPointF> ncp1;
   normCP(cp0, cp1, ncp0, ncp1);

   // Compute coefficients of the normalized Bezier curves
   double na[4],nb[4],nc[4],nd[4];
   coefBezier(ncp0,na,nb);
   coefBezier(ncp1,nc,nd);

   // Compute coefficients of the implicit intersection equations F0(u) and
      F1(t)
   double cf0[10],cf1[10];
   coefImp0(na,nb,nc,nd,cf0);
   coefImp1(na,nb,nc,nd,cf1);

   vector<double> rt0,rt1;  // Roots
   rt0.reserve(9);
   rt1.reserve(9);

   // Find roots of F0(u) and F1(t)
   vRoots(cf0,rt0);
   vRoots(cf1,rt1);

   QList<QPointF> nip0,nip1;  // Intersection candidates (normalized
      coordinates)
   for(double u:rt0)
      nip0<<pointBezier(nc,nd,u);
   for(double t:rt1)
      nip1<<pointBezier(na,nb,t);

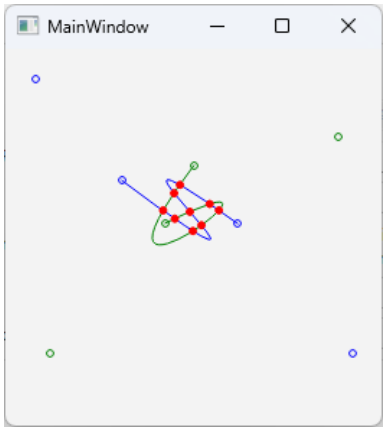
   // Match intersection candidate coordinates
   double c[4],d[4];
```

```
   coefBezier(cp1,c,d);
   int i0=0;
   foreach(QPointF p0,nip0){
      foreach(QPointF p1,nip1){
         double dx=p1.x()-p0.x();
         double dy=p1.y()-p0.y();
         if(sqrt(PW2(dx)+PW2(dy))<=DSTMIN){
            // Output the intersection coordinate (non-normalized)
            op<<pointBezier(c,d,rt0[i0]);
         }
      }
      ++i0;
   }
   return true;
}
```

Appendix F Qt Sample Application

When you start the program, **Ex.3** (with the coordinate values multiplied by 10 and upside down) will be displayed. Dragging and moving the control points will update the curve and intersection points.

Example of display:



Listing 16: mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QMouseEvent>
#include <QGraphicsScene>
#include <QGraphicsItem>
#include "scene.h"

QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE
```

```
class MainWindow : public QMainWindow
{
      Q_OBJECT

public:
      MainWindow(QWidget *parent = nullptr);
      ~MainWindow();

private slots:
      void mouseMove(QGraphicsSceneMouseEvent *mouseEvent);
      void mousePress(QGraphicsSceneMouseEvent *mouseEvent);

private:
      Scene scene;
   QGraphicsEllipseItem *rc0[4];
   QGraphicsEllipseItem *rc1[4];
   QGraphicsPathItem *ci0;
   QGraphicsPathItem *ci1;
   QList<QGraphicsEllipseItem *> ri;
   QList<QPointF> cp0;
   QList<QPointF> cp1;
   int scp;

private:
      Ui::MainWindow *ui;
};
#endif // MAINWINDOW_H
```

Listing 17: mainwindow.cpp

```
#include "mainwindow.h"
#include "ui_mainwindow.h"

bool vBezierIntersection(
   QList<QPointF> &cp0, // IN Control points of cubic Bezier curve 0
   QList<QPointF> &cp1, // IN Control points of cubic Bezier curve 1
   QList<QPointF> &op); // OUT Intersection points

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
      ui->setupUi(this);
   scene.setSceneRect(0,0,ui->graphicsView->width()-2,ui->graphicsView->
      height()-2);
   ui->graphicsView->setScene(&scene);
   ui->graphicsView->setRenderHint(QPainter::Antialiasing);

   // Ex.3 with the coordinate values multiplied by 10
   cp0<<QPointF(70,80);
   cp0<<QPointF(230,200);
```

```
    cp0<<QPointF(10,10);
    cp0<<QPointF(150,110);
    cp1<<QPointF(100,110);
    cp1<<QPointF(220,50);
    cp1<<QPointF(20,200);
    cp1<<QPointF(120,70);

    for(int i=0;i<4;i++){
        rc0[i]=scene.addEllipse(cp0[i].x()-2.5,cp0[i].y()-2.5,5,5,QPen(Qt::
            blue));
        rc1[i]=scene.addEllipse(cp1[i].x()-2.5,cp1[i].y()-2.5,5,5,QPen(Qt::
            darkGreen));
    }
    {
        QPainterPath curve(cp0[0]);
        curve.cubicTo(cp0[1],cp0[2],cp0[3]);
        ci0=scene.addPath(curve,QPen(QBrush(Qt::blue),1));
    }
    {
        QPainterPath curve(cp1[0]);
        curve.cubicTo(cp1[1],cp1[2],cp1[3]);
        ci1=scene.addPath(curve,QPen(QBrush(Qt::darkGreen),1));
    }
    QList<QPointF> op;
    vBezierIntersection(cp0,cp1,op);
    foreach (QPointF p,op)
        ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::red),QBrush(Qt::
            red));
    scp=-1;

    connect(&scene,SIGNAL(mouseMove(QGraphicsSceneMouseEvent*)),this,SLOT(
        mouseMove(QGraphicsSceneMouseEvent*)));
    connect(&scene,SIGNAL(mousePress(QGraphicsSceneMouseEvent*)),this,SLOT(
        mousePress(QGraphicsSceneMouseEvent*)));
}

MainWindow::~MainWindow()
{
      delete ui;
}

void MainWindow::mouseMove(QGraphicsSceneMouseEvent *mouseEvent)
{
    if((mouseEvent->buttons()&Qt::LeftButton)&&scp>=0){
        QPointF pos=mouseEvent->scenePos();
        if(scp<=3){
            cp0[scp]=pos;
            rc0[scp]->setRect(pos.x()-2.5,pos.y()-2.5,5,5);
            QPainterPath curve(cp0[0]);
            curve.cubicTo(cp0[1],cp0[2],cp0[3]);
            ci0->setPath(curve);
```

```
      }else{
         cp1[scp-4]=pos;
         rc1[scp-4]->setRect(pos.x()-2.5,pos.y()-2.5,5,5);
         QPainterPath curve(cp1[0]);
         curve.cubicTo(cp1[1],cp1[2],cp1[3]);
         ci1->setPath(curve);
      }
      foreach(QGraphicsEllipseItem *gi,ri){
         scene.removeItem(gi);
         delete gi;
      }
      ri.clear();
      QList<QPointF> op;
      vBezierIntersection(cp0,cp1,op);
      foreach(QPointF p,op)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::red),QBrush(
            Qt::red));
   }
}

void MainWindow::mousePress(QGraphicsSceneMouseEvent *mouseEvent)
{
      if(mouseEvent->buttons() & Qt::LeftButton){
      QPointF pos = mouseEvent->scenePos();
      scp=-1;
      for(int i=0;i<4;i++){
         if(QLineF(pos,rc0[i]->rect().center()).length()<=5)
            scp=i;
      }
      for(int i=0;i<4;i++){
         if(QLineF(pos,rc1[i]->rect().center()).length()<=5)
            scp=i+4;
      }
   }
}
```

Listing 18: scene.h

```
#ifndef SCENE_H
#define SCENE_H

#include <QObject>
#include <QGraphicsScene>
#include <QGraphicsSceneMouseEvent>

class Scene : public QGraphicsScene
{
      Q_OBJECT
public:
      explicit Scene(QObject *parent = nullptr);

signals:
```

```
      void mouseMove(QGraphicsSceneMouseEvent *mouseEvent);
      void mousePress(QGraphicsSceneMouseEvent *mouseEvent);

protected:
      virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent)
         override;
      virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent)
         override;
};

#endif // SCENE_H
```

Listing 19: scene.cpp

```
#include "scene.h"

Scene::Scene(QObject *parent)
: QGraphicsScene{parent}
{
}

void Scene::mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent)
{
      emit mouseMove(mouseEvent);
}

void Scene::mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent)
{
      emit mousePress(mouseEvent);
}
```

Listing 20: mainwindow.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>MainWindow</class>
 <widget class="QMainWindow" name="MainWindow">
  <property name="geometry">
   <rect>
    <x>0</x>
    <y>0</y>
    <width>261</width>
    <height>261</height>
   </rect>
  </property>
  <property name="windowTitle">
   <string>MainWindow</string>
  </property>
  <widget class="QWidget" name="centralwidget">
   <widget class="QGraphicsView" name="graphicsView">
    <property name="geometry">
     <rect>
```

```
     <x>10</x>
     <y>10</y>
     <width>241</width>
     <height>241</height>
    </rect>
   </property>
   <property name="minimumSize">
    <size>
     <width>100</width>
     <height>100</height>
    </size>
   </property>
   <property name="mouseTracking">
    <bool>true</bool>
   </property>
   <property name="alignment">
    <set>Qt::AlignmentFlag::AlignLeading|Qt::AlignmentFlag::AlignLeft|Qt::
        AlignmentFlag::AlignTop</set>
   </property>
  </widget>
 </widget>
</widget>
<resources/>
<connections/>
</ui>
```